

1.1数据库系统概述

2022年1月24日

21:16

★数据库四个基本概念:

1. 数据 (Data) : 数据库中存储的基本对象

定义: 描述事物的符号记录

种类: 数字、文字、图形、图像、音频、视频、学生的档案记录等。

数据的含义:称为数据的语

- 冗余度较小
 - 数据独立性较高
 - 易扩展

存储在计算机上有
结构的数据集

★2. 数据库管理系统DBMS

定义:

- 位于用户与操作系统之间的一层数据管理软件
- 是基础软件, 是一个大型复杂的软件系统

用途: ■ 科学地组织和存储数据、高效地获取和维护数据

1. ■ 数据定义功能

- 提供数据定义语言 (DDL) ●定义数据库中的数据对象

2. ■ 数据组织、存储和管理

- 分类组织、存储和管理各种数据
- 确定组织数据的文件结构和存取方式
- 实现数据之间的联系
- 提供多种存取方法提高存取效率

3. ■ 数据操纵功能

DataBase Definition Language

- 提供数据操纵语言 (DML)
- 实现对数据库的基本操作 (查询、插入、删除和修改)

4. ■ 数据库的事务管理和运行管理

- 数据库在建立、运行和维护时由数据库管理系统统一管理和控制
- 保证数据的安全性、完整性、多用户对数据的并发使用
- 发生故障后的系统恢复

5. 数据库建立和维护功能

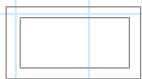
6. 其他功能

3. 数据库系统DBS : 是由数据库、数据库管理系统 (及其应用开发工具、应用程序、数据库管理员组成的存储、管理、处理和维护数据的系统。

)

DataBase System





1.2数据库管理技术的产生和发展

2022年1月25日 9:18

数据库
管理三
个阶
段:

1. 什么是数据管理

1. 对数据进行分类、组织、编码、存储、检索和维护

2. 数据处理的中心问题

数据的管理指对各种数据进行收集、存储、加工和传递等一系列活动的总称

2. 数据管理技术的发展过程

1. 人工管理阶段 (20世纪50年代中之前)

a. 背景:

应用背景	硬件背景	软件背景	处理方式
科学计算	无直接存取存储设备	没有操作系统	批处理

b. 特点

- i. ■ 数据的管理者: 用户 (程序员), 数据不保存
- ii. ■ 数据面向的对象: 某一应用程序
- iii. ■ 数据的共享程度: 无共享、冗余度极大
- iv. ■ 数据的独立性: 不独立, 完全依赖于程序
- v. ■ 数据的结构化: 无结构
- vi. ■ 数据控制能力: 应用程序自己控制

2. 文件系统阶段 (20世纪50年代末--60年代中)

a. 背景:

应用背景	硬件背景	软件背景	处理方式
科学计算、数据处理	磁盘、磁鼓	有文件系统	联机实时处理、批处理

b. 特点:

- i. ■ 数据的管理者: 文件系统, 数据可长期保存
- ii. ■ 数据面向的对象: 某一应用
- iii. ■ 数据的共享程度: 共享性差、冗余度大
- iv. ■ 数据的结构化: 记录内有结构, 整体无结构
- v. ■ 数据的独立性: 独立性差
- vi. ■ 数据控制能力: 应用程序自己控制

3. 数据库系统阶段 (20世纪60年代末--现在)

a. 背景:

应用背景	硬件背景	软件背景	处理方式
------	------	------	------

大规模数据管理	大容量磁盘、磁盘阵列	有数据库管理系统	联机实时处理 批处理	分布处理
---------	------------	----------	---------------	------

数据库系统实现整体数据的结构化

★b. 特点:

i. ❖数据结构化

1) 数据的整体结构化是数据库的主要特征之一，也是数据库系统与文件系统的本质区别。

2) 整体结构化

- a) ■ 不再仅仅针对某一个应用，而是面向全组织
- b) ■ 不仅数据内部结构化，整体是结构化的，数据之间具有联系
- c) ■ 数据记录可以变长
- d) ■ 数据的最小存取单位是数据项
- e) ■ 数据的用数据模型描述，无需应用程序定义

ii. ❖数据的共享性高，冗余度低且易扩充

1) 数据面向整个系统，可以被多个用户、多个应用共享使用。

2) 数据共享的好处

- a) ■ 减少数据冗余，节约存储空间
- b) ■ 避免数据之间的不相容性与不一致性
- c) ■ 使系统易于扩充

★iii. ❖数据独立性高，是数据库的显著优点

1) 物理独立性

■ 指用户的应用程序与数据库中数据的物理存储是相互独立的。当数据的物理存储改变了，应用程序不用改变。

2) 逻辑独立性

■ 指用户的应用程序与数据库的逻辑结构是相互独立的。数据的逻辑结构改变了，应用程序不用改变。

数据独立性由数据库管理系统的二级映像功能来保证。

iv. ❖数据由数据库管理系统统一管理和控制:

(1) 数据的安全性 (Security) 保护

保护数据以防止不合法的使用造成的数据的泄密和破坏。

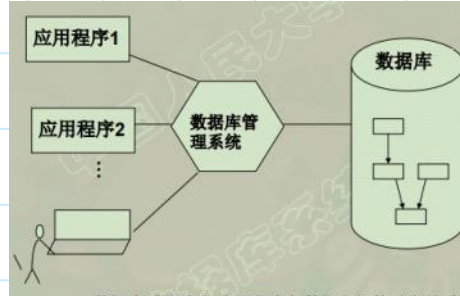
(2) 数据的完整性 (Integrity) 检查

保证数据的正确性、有效性和相容性。

(3) 并发 (Concurrency) 控制

对多用户的并发操作加以控制和协调，防止相互干扰而得到错误的结果。

(4) 数据库恢复 (Recovery) 将数据库从错误状态恢复到某一已知的正确状态。



总结

文件系统到数据库系统，标志着数据管理技术的飞跃。
数据库的出现使信息系统从以加工数据的程序为中心转向围绕共享的数据库为中心的新阶段。

1.3数据库系统的特点

2022年1月25日

11:38

数据库
管理四
个阶
段：

1. 什么是数据管理

1. 对数据进行分类、组织、编码、存储、检索和维护
2. 数据处理的中心问题

2. 数据管理技术的发展过程

1. 人工管理阶段（20世纪50年代中之前）

a. 背景：

应用背景	硬件背景	软件背景	处理方式
科学计算	无直接存取存储设备	没有操作系统	批处理

b. 特点

- i. 数据的管理者：用户（程序员），数据不保存
- ii. 数据面向的对象：某一应用程序
- iii. 数据的共享程度：无共享、冗余度极大
- iv. 数据的独立性：不独立，完全依赖于程序
- v. 数据的结构化：无结构
- vi. 数据控制能力：应用程序自己控制

2. 文件系统阶段（20世纪50年代末--60年代中）

a. 背景：

应用背景	硬件背景	软件背景	处理方式
科学计算、数据处理	磁盘、磁鼓	有文件系统	联机实时处理、批处理

b. 特点：

- i. 数据的管理者：文件系统，数据可长期保存
- ii. 数据面向的对象：某一应用
- iii. 数据的共享程度：共享性差、冗余度大
- iv. 数据的结构化：记录内有结构，整体无结构
- v. 数据的独立性：独立性差
- vi. 数据控制能力：应用程序自己控制

3. 数据库系统阶段（20世纪60年代末--现在）

a. 背景：

应用背景	硬件背景	软件背景	处理方式

		大规模数据管理	大容量磁盘、磁盘阵列	有数据库管理系统	联机实时处理,分布处理,批处理
总结	文件系统到数据库系统，标志着数据管理技术的飞跃				

1.4数据模型

2022年1月25日

13:35

数据模型定义:	数据模型是对现实世界数据特征的抽象。 通俗地讲数据模型就是现实世界的模拟,用来描述数据组织数据和对数据
数据模型的要求:	能比较真实地模拟现实世界 容易为人所理解 便于在计算机上实现
数据模型的意义:	数据模型是数据库系统的核心和基础
★数据模型的分类:	(1) 概念模型 也称信息模型,它是按用户的观点来对数据和信息建模,用于数据库设计。 (2) 逻辑模型和物理模型 ■逻辑模型主要包括网状模型、层次模型、关系模型、面向对象数据模型、对象关系数据模型、半结构化数据模型等。按计算机系统的观点对数据建模,用于DBMS实现。 ■物理模型是对数据最底层的抽象,描述数据在系统内部的表示方式和存取方法,在磁盘或磁带上的存储方式和存取方法
客观对象的抽象过程---两步抽象	■ 现实世界中的客观对象抽象为概念模型; ●将现实世界抽象为信息世界 ■ 把概念模型转换为某一数据库管理系统支持的数据模型。 ●将信息世界转换为机器世界 现实世界 ↓ 信息世界 ↓ 概念模型 ↓ 机器世界 数据库管理系统支持的数据模型
★数据模型的组成三要素	❖数据结构 (最重要) 描述数据库的组成对象,以及对象之间的联系 ☆ 数据结构是对系统静态特性的描述。是所描述对象类型的集合 描述的内容 两类 { 1. 与对象的类型、内容、性质有关 如网状模型中的数据项、记录等。 2. 与数据之间联系有关 ❖数据操作 定义: 对数据库中各种对象(型)的实例(值)允许执行的操作的集合,包括操作及有关的操作规则 数据操作的类型 ■ 查询 ■ 更新(包括插入、删除、修改) 数据模型对操作的定义

	☆ ■操作的确切含义 ■操作符号 ■操作规则（如优先级） ■实现操作的语言 ❖数据操作是对系统动态特性的描述
	❖数据的完整性约束条件 定义：一组完整性规则的集合 ■完整性规则：给定的数据模型中数据及其联系所具有制约和依存规则，用以限定符合数据模型的数据库状态以及状态的变化，以保证数据的正确、有效和相容
数据模型“型”与“值”	型指某一类数据的结构和属性的说明，值是型的一个具体赋值。

1.5概念模型

2022年1月25日

14:32

概念模型的意义:	概念模型是现实世界到机器世界的一个中间层次 ①概念模型用于信息世界的建模; ②现实世界到信息世界的第一层抽象; ③数据库设计人员进行数据库设计的有力工具; ④数据库设计人员和用户之间进行交流的语言。
概念模型基本概念:	(1) 实体 (Entity) <i>[-组记录]</i> 客观存在并可相互区别的事物称为实体。可以是具体的人、事、物或抽象的概念。 (2) 属性 (Attribute) 实体所具有的某一特性称为属性。一个实体可以由若干个属性来刻画 (3) 码 (Key) 唯一标识实体的属性集称为码。 (4) 域(Domain) 域是一组具有相同数据类型的值的集合。属性的取值范围来自某个域 (性别男女为一个域)。 (5) 实体型 (Entity Type) <i>类</i> 用实体名及其属性名集合来抽象和刻画同类实体称为实体型 (6) 实体集 (Entity Set) 同一类型实体的集合称为实体集 ★ (7) 联系 (Relationship) ■ 现实世界中事物内部以及事物之间的联系在信息世界中反映为实体 (型) 内部的联系和实体 (型) 之间的联系。 ■ 实体内部的联系通常是指组成实体的各属性之间的联系 ■ 实体之间的联系通常是指不同实体集之间的联系 ■ 实体之间的联系有一对一 (班级-班长)、一对多 (1: n学生-班级) 和多对多 (n:m课程: 学生) 种类型。同一个实体集可以存在多种联系。
★ 实体-联系方法	<i>是概念模型的一种表示方法, 该方法用E-R图来描述现实世界的概念模型</i> ■ 用E-R图来描述现实世界的概念模型 ■ E-R方法也称为E-R模型 E-R图提供了表示实体型, 属性和联系的方法。 实体型: 用矩形表示, 矩形框内写明实体名。 属性: 用椭圆形表示, 并用无向边将其与相应的实体形连接起来。 联系: 用菱形表示, 菱形框内写明联系名, 并用无向边分别与有关实体型连接起来, 同

时,在无向边旁标上联系的类型(1:1, 1:n,n:m)。

1.6关系模型

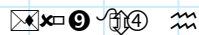
2022年1月25日

16:27

关系模型的数据结构

关系数据库系统采用关系模型作为数据的组织方式。

Relation



Tuple



Attribute



Key



域 (Domain)

- 是一组具有相同数据类型的值的集合。属性的取值范围来自某个域。

分量

- 元组中的一个属性值。

关系模式

- 对关系的描述

一般表示为：关系名（属性1，属性2，...，属性n）

例如：学生（学号，姓名，年龄，性别，系名，年级）

关系术语	一般表格的术语
关系名	表名
关系模式	表头（表格的描述）
关系	（一张）二维表
元组	记录或行
属性	列
属性名	列名
属性值	列值
分量	一条记录中的一个列值
非规范关系	表中有表（大表中嵌有小表）

★ 关系的完整性约束条件

实体完整性

必须能去确定唯一的实体

	参照完整性 参照其他表的属性不能改 用户定义的完整性 例如：年龄范围，根据实际需要自己定义。
关系的数据操作 操作的对象和结果都是关系	查询、插入、删除、更新。 关系模型中的数据操作都是集合操作，操作对象和操作结果都是关系。
关系模型的优点	■ 建立在严格的数学概念的基础上 ■ 概念单一 ● 实体和各类联系都用关系来表示 ● 对数据的检索结果也是关系 ■ 关系模型的存取路径对用户透明，不用管怎么放，放哪个位置。 ● 具有更高的数据独立性，更好的安全保密性 ● 简化了程序员的工作和数据库开发建立的工作
关系模型的缺点	■ 存取路径对用户透明，查询效率往往不如格式化数据模型 ■ 为提高性能，必须对用户的查询请求进行优化，增加了开发数据库管理系统的难度

1.7 关系型数据库的模式

2022年1月25日

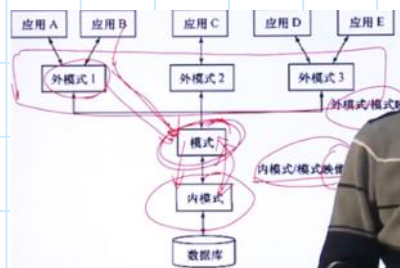
16:56

★ 数据库系统的三级模式结构

模式 (Schema) 怎么存

外模式 (External Schema) 应用读数据

内模式 (Internal Schema) 计算机硬盘怎么存



❖ 外模式的地位：介于模式与应用之间

❖ 模式与外模式的关系：一对多

- 外模式通常是模式的子集

- 一个数据库可以有多个外模式。反映了不同的用户的应

用需求、看待数据的方式、对数据保密的要求，用户只能看见和访问外模式的数据

- 对模式中同一数据，在外模式中的结构、类型、长度、保密级别等都可以不同

❖ 外模式与应用的关系：一对多

- 同一外模式也可以为某一用户的多个应用系统所使用

- 但一个应用程序只能使用一个外模式

模式 (也称逻辑模式)

★ 数据库中全体数据的逻辑结构和特征的描述

所有用户的公共数据视图，数据库系统模式结构的中间层。不涉及物理和硬件，也不涉及具体的应用程序。

它仅仅涉及到型的描述，不涉及具体的值。模式的一个具体的值称为模式的一个实例。

一个数据库只有一个模式，数据库模式以某一种数据模型为基础。

外模式 (称子模式或用户模式)

定义：数据库用户 (包括应用程序员和最终用户) 使用的局部数据的逻辑结构和特征的描述，数据库用户的数据视图，是与某一应用有关的数据的逻辑表示

内模式 (也称存储模式)

- 是数据物理结构和存储方式的描述

- 是数据在数据库内部的表示方式

- 记录的存储方式 (例如，顺序存储，按照B树结构存储，

	按hash方法存储等) ●索引的组织方式 ●数据是否压缩存储 ●数据是否加密 ●数据存储记录结构的规定 ❖一个数据库只有一个内模式
★数据库的二级映像功能与数据独立性	二级映象在数据库管理系统内部实现这三个抽象层次的联系和转换 外模式 / 模式映像 模式 / 内模式映像 当模式改变时,由数据库管理员对各个外模式/模式的映像作相应改变,可以使应用程序不变,从而保证了数据与程序的逻辑独立性,简称数据的逻辑独立性。
数据库系统的组成	硬件平台及数据库 ■ 足够大的内存 ■ 足够的大的磁盘或磁盘阵列等设备 ■ 较高的通道能力, 提高数据传送率 当数据库的存储结构改变时,由数据库管理员对模式/内模式映像作相应改变,可以使模式保持不变,从而应用程序也不会改变,保证了数据与程序的物理独立性。 软件 1. 数据库管理系统 2. 支持数据库管理系统运行的操作系统 3. 与数据库接口的高级语言及其编译系统 4. 以数据库管理系统为核心的应用开发工具 5. 为特定应用环境开发的数据库应用系统 人员 1. 数据库管理员 DBA database adminnistrator: 管理数据库 2. 系统分析员和数据库设计人员: 设计er图,确定数据库系统的设计。 3. 应用程序员: 开发软件 4. 最终用户

1.8常见模型

2022年1月2日 8:48

数据模型	概念 计算机是不可能直接处理现实世界中的具体事物的，所以人们必须事先把具体事物转化为计算机能够处理的数据。因此数据模型是对现实世界数据特征的抽象，也即数据模型是用来描述数据、组织数据和对数据进行操作的。它应该满足下面三点要求 • 比较真实地模拟现实世界 • 容易被理解 • 便于在计算机上实现 • 从现实世界到概念模型的转换是由数据库设计人员完成的 • 从概念模型到逻辑模型的转换可由数据库设计人员完成 • 从逻辑模型到物理模型的转换主要是由数据库管理系统完成 分类 <table><tr><td rowspan="7">概念模型（信息模型）</td><td>定义</td><td>是按用户的观点来对数据和信息建模，用于数据库设计</td></tr><tr><td rowspan="6">相关概念(术语)</td><td>实体</td><td>客观存在并可以相互区别的事物。实体可以是具体的人、事、物，也可以是抽象的概念或联系</td></tr><tr><td>属性</td><td>实体所具有的某一特性。一个实体可以由若干属性刻画</td></tr><tr><td>码</td><td>唯一标识实体的属性</td></tr><tr><td>实体型</td><td>用实体名及其属性集合来抽象和刻画同类实体</td></tr><tr><td>实体集</td><td>同一类型实体的集合</td></tr><tr><td>联系</td><td>实体内部的联系通常是指组成实体的各属性之间的联系，实体之间的联系通常是指不同实体集之间的联系</td></tr><tr><td colspan="2">逻辑模型和物理模型</td></tr><tr><td>逻辑模型</td><td colspan="2">是按计算机系统的观点对数据建模，主要用于数据库管理系统的实现</td></tr><tr><td>物理模型</td><td colspan="2">对数据最底层的抽象，它描述数据在系统内部的表示方法和存取方法，或在磁盘或磁带上的存储方式和存取方法，是面向计算机系统的</td></tr></table>	概念模型（信息模型）	定义	是按 用户的观点 来对数据和信息建模，用于 数据库设计	相关概念(术语)	实体	客观存在并可以相互区别的事物。实体可以是具体的人、事、物，也可以是抽象的概念或联系	属性	实体所具有的某一特性。一个实体可以由若干属性刻画	码	唯一标识实体的属性	实体型	用实体名及其属性集合来抽象和刻画同类实体	实体集	同一类型实体的集合	联系	实体内部的联系通常是指组成实体的各属性之间的联系，实体之间的联系通常是指不同实体集之间的联系	逻辑模型和物理模型		逻辑模型	是按 计算机系统的观点 对数据建模，主要用于 数据库管理系统的实现		物理模型	对数据最底层的抽象，它描述数据在系统内部的表示方法和存取方法，或在磁盘或磁带上的存储方式和存取方法，是面向计算机系统的		数据模型组成要素	<table><tr><td>数据结构</td><td>数据结构是所研究的对象类型的集合</td></tr><tr><td>数据操作</td><td>是指对数据库中各种对象(型)的实例(值)允许执行的操作，及有关的操作规则 • 查询 • 更新 </td></tr><tr><td>数据完整性约束</td><td>数据的完整性约束条件是一组完整性规则的集合，规定数据库状态及状态变化所应满足的条件，以保证数据的正确性、有效性和相容性。 • 实体完整性 • 参照完整性 </td></tr></table>	数据结构	数据结构是所研究的对象类型的集合	数据操作	是指对数据库中各种对象(型)的实例(值)允许执行的操作，及有关的操作规则 • 查询 • 更新	数据完整性约束	数据的完整性约束条件是一组 完整性规则的集合 ，规定数据库 状态及状态变化所应满足的条件 ，以保证数据的正确性、有效性和相容性。 • 实体完整性 • 参照完整性	层次模型、网状模型和关系模型	<table><tr><td rowspan="3">层次模型</td><td>概念</td><td>在数据库中定义满足下面两个条件的基本层次联系的集合就为层次模型 • 有且只有一个结点没有双亲结点，此结点为根节点 • 根结点以外的所有结点有且只有一个双亲结点 在层次模型中每个结点表示一个记录类型，记录类型之间的联系用结点之间的连线（有向边）表示，这种联系是父子之间的一对多的联系</td></tr><tr><td rowspan="2">优缺点（了解）</td><td>优点</td><td> • 层次模型的数据结构较为简单清晰 • 层次模型的查询效率高 • 层次模型提供了良好的完整性支持 </td></tr><tr><td>缺点</td><td> • 层次模型不能很好的表示多对多联系 • 插入，删除麻烦，编写程序复杂 • 插叙子女结点必须通过双亲结点 • 由于结构严密，层次命令趋于程序化 </td></tr><tr><td>网状模型</td><td>概念</td><td>在数据库中定义满足下面两个条件的基本层次联系的集合就为网状模型 - **允许一个以上的结点无双亲** - **一个结点可以有多个的双亲** </td></tr><tr><td></td><td>优缺点（了解）</td><td>优点</td><td> • 能够更为直接地描述现实世界 • 具有良好的性能，存取效率较高 </td></tr></table>	层次模型	概念	在数据库中定义满足下面两个条件的基本层次联系的集合就为 层次模型 • 有且只有一个结点没有双亲结点，此结点为根节点 • 根结点以外的所有结点有且只有一个双亲结点 在层次模型中 每个结点表示一个记录类型 ，记录类型之间的联系用结点之间的连线（有向边）表示，这种联系是父子之间的一对多的联系	优缺点（了解）	优点	• 层次模型的数据结构较为简单清晰 • 层次模型的查询效率高 • 层次模型提供了良好的完整性支持	缺点	• 层次模型不能很好的表示多对多联系 • 插入，删除麻烦，编写程序复杂 • 插叙子女结点必须通过双亲结点 • 由于结构严密，层次命令趋于程序化	网状模型	概念	在数据库中定义满足下面两个条件的基本层次联系的集合就为 网状模型 - **允许一个以上的结点无双亲** - **一个结点可以有多个的双亲**		优缺点（了解）	优点	• 能够更为直接地描述现实世界 • 具有良好的性能，存取效率较高
概念模型（信息模型）	定义		是按 用户的观点 来对数据和信息建模，用于 数据库设计																																															
	相关概念(术语)		实体	客观存在并可以相互区别的事物。实体可以是具体的人、事、物，也可以是抽象的概念或联系																																														
			属性	实体所具有的某一特性。一个实体可以由若干属性刻画																																														
			码	唯一标识实体的属性																																														
			实体型	用实体名及其属性集合来抽象和刻画同类实体																																														
			实体集	同一类型实体的集合																																														
		联系	实体内部的联系通常是指组成实体的各属性之间的联系，实体之间的联系通常是指不同实体集之间的联系																																															
逻辑模型和物理模型																																																		
逻辑模型	是按 计算机系统的观点 对数据建模，主要用于 数据库管理系统的实现																																																	
物理模型	对数据最底层的抽象，它描述数据在系统内部的表示方法和存取方法，或在磁盘或磁带上的存储方式和存取方法，是面向计算机系统的																																																	
数据模型组成要素	<table><tr><td>数据结构</td><td>数据结构是所研究的对象类型的集合</td></tr><tr><td>数据操作</td><td>是指对数据库中各种对象(型)的实例(值)允许执行的操作，及有关的操作规则 • 查询 • 更新 </td></tr><tr><td>数据完整性约束</td><td>数据的完整性约束条件是一组完整性规则的集合，规定数据库状态及状态变化所应满足的条件，以保证数据的正确性、有效性和相容性。 • 实体完整性 • 参照完整性 </td></tr></table>	数据结构	数据结构是所研究的对象类型的集合	数据操作	是指对数据库中各种对象(型)的实例(值)允许执行的操作，及有关的操作规则 • 查询 • 更新	数据完整性约束	数据的完整性约束条件是一组 完整性规则的集合 ，规定数据库 状态及状态变化所应满足的条件 ，以保证数据的正确性、有效性和相容性。 • 实体完整性 • 参照完整性	层次模型、网状模型和关系模型	<table><tr><td rowspan="3">层次模型</td><td>概念</td><td>在数据库中定义满足下面两个条件的基本层次联系的集合就为层次模型 • 有且只有一个结点没有双亲结点，此结点为根节点 • 根结点以外的所有结点有且只有一个双亲结点 在层次模型中每个结点表示一个记录类型，记录类型之间的联系用结点之间的连线（有向边）表示，这种联系是父子之间的一对多的联系</td></tr><tr><td rowspan="2">优缺点（了解）</td><td>优点</td><td> • 层次模型的数据结构较为简单清晰 • 层次模型的查询效率高 • 层次模型提供了良好的完整性支持 </td></tr><tr><td>缺点</td><td> • 层次模型不能很好的表示多对多联系 • 插入，删除麻烦，编写程序复杂 • 插叙子女结点必须通过双亲结点 • 由于结构严密，层次命令趋于程序化 </td></tr><tr><td>网状模型</td><td>概念</td><td>在数据库中定义满足下面两个条件的基本层次联系的集合就为网状模型 - **允许一个以上的结点无双亲** - **一个结点可以有多个的双亲** </td></tr><tr><td></td><td>优缺点（了解）</td><td>优点</td><td> • 能够更为直接地描述现实世界 • 具有良好的性能，存取效率较高 </td></tr></table>	层次模型	概念	在数据库中定义满足下面两个条件的基本层次联系的集合就为 层次模型 • 有且只有一个结点没有双亲结点，此结点为根节点 • 根结点以外的所有结点有且只有一个双亲结点 在层次模型中 每个结点表示一个记录类型 ，记录类型之间的联系用结点之间的连线（有向边）表示，这种联系是父子之间的一对多的联系	优缺点（了解）	优点	• 层次模型的数据结构较为简单清晰 • 层次模型的查询效率高 • 层次模型提供了良好的完整性支持	缺点	• 层次模型不能很好的表示多对多联系 • 插入，删除麻烦，编写程序复杂 • 插叙子女结点必须通过双亲结点 • 由于结构严密，层次命令趋于程序化	网状模型	概念	在数据库中定义满足下面两个条件的基本层次联系的集合就为 网状模型 - **允许一个以上的结点无双亲** - **一个结点可以有多个的双亲**		优缺点（了解）	优点	• 能够更为直接地描述现实世界 • 具有良好的性能，存取效率较高																										
数据结构	数据结构是所研究的对象类型的集合																																																	
数据操作	是指对数据库中各种对象(型)的实例(值)允许执行的操作，及有关的操作规则 • 查询 • 更新																																																	
数据完整性约束	数据的完整性约束条件是一组 完整性规则的集合 ，规定数据库 状态及状态变化所应满足的条件 ，以保证数据的正确性、有效性和相容性。 • 实体完整性 • 参照完整性																																																	
层次模型、网状模型和关系模型	<table><tr><td rowspan="3">层次模型</td><td>概念</td><td>在数据库中定义满足下面两个条件的基本层次联系的集合就为层次模型 • 有且只有一个结点没有双亲结点，此结点为根节点 • 根结点以外的所有结点有且只有一个双亲结点 在层次模型中每个结点表示一个记录类型，记录类型之间的联系用结点之间的连线（有向边）表示，这种联系是父子之间的一对多的联系</td></tr><tr><td rowspan="2">优缺点（了解）</td><td>优点</td><td> • 层次模型的数据结构较为简单清晰 • 层次模型的查询效率高 • 层次模型提供了良好的完整性支持 </td></tr><tr><td>缺点</td><td> • 层次模型不能很好的表示多对多联系 • 插入，删除麻烦，编写程序复杂 • 插叙子女结点必须通过双亲结点 • 由于结构严密，层次命令趋于程序化 </td></tr><tr><td>网状模型</td><td>概念</td><td>在数据库中定义满足下面两个条件的基本层次联系的集合就为网状模型 - **允许一个以上的结点无双亲** - **一个结点可以有多个的双亲** </td></tr><tr><td></td><td>优缺点（了解）</td><td>优点</td><td> • 能够更为直接地描述现实世界 • 具有良好的性能，存取效率较高 </td></tr></table>	层次模型	概念	在数据库中定义满足下面两个条件的基本层次联系的集合就为 层次模型 • 有且只有一个结点没有双亲结点，此结点为根节点 • 根结点以外的所有结点有且只有一个双亲结点 在层次模型中 每个结点表示一个记录类型 ，记录类型之间的联系用结点之间的连线（有向边）表示，这种联系是父子之间的一对多的联系	优缺点（了解）	优点	• 层次模型的数据结构较为简单清晰 • 层次模型的查询效率高 • 层次模型提供了良好的完整性支持	缺点	• 层次模型不能很好的表示多对多联系 • 插入，删除麻烦，编写程序复杂 • 插叙子女结点必须通过双亲结点 • 由于结构严密，层次命令趋于程序化	网状模型	概念	在数据库中定义满足下面两个条件的基本层次联系的集合就为 网状模型 - **允许一个以上的结点无双亲** - **一个结点可以有多个的双亲**		优缺点（了解）	优点	• 能够更为直接地描述现实世界 • 具有良好的性能，存取效率较高																																		
层次模型	概念		在数据库中定义满足下面两个条件的基本层次联系的集合就为 层次模型 • 有且只有一个结点没有双亲结点，此结点为根节点 • 根结点以外的所有结点有且只有一个双亲结点 在层次模型中 每个结点表示一个记录类型 ，记录类型之间的联系用结点之间的连线（有向边）表示，这种联系是父子之间的一对多的联系																																															
	优缺点（了解）		优点	• 层次模型的数据结构较为简单清晰 • 层次模型的查询效率高 • 层次模型提供了良好的完整性支持																																														
		缺点	• 层次模型不能很好的表示多对多联系 • 插入，删除麻烦，编写程序复杂 • 插叙子女结点必须通过双亲结点 • 由于结构严密，层次命令趋于程序化																																															
网状模型	概念	在数据库中定义满足下面两个条件的基本层次联系的集合就为 网状模型 - **允许一个以上的结点无双亲** - **一个结点可以有多个的双亲**																																																
	优缺点（了解）	优点	• 能够更为直接地描述现实世界 • 具有良好的性能，存取效率较高																																															

		缺点 • 结构复杂，不利于用户掌握 • 其DDL、DML复杂，嵌入高级语言中也难以让用户掌握 • 用户需要了解结构细节，加重了编写程序的负担																																													
关系模型	概念	从 用户 的角度看，关系模型由一组 关系 组成，每个关系的数据结构是一张规范化的 二维表																																													
	相关术语	学生登记表 属性 <table><tr><td>学号</td><td>姓名</td><td>年龄</td><td>性别</td><td>系名</td><td>年级</td></tr><tr><td>2005004</td><td>王小明</td><td>19</td><td>女</td><td>社会学</td><td>2005</td></tr><tr><td>2005006</td><td>黄大鹏</td><td>20</td><td>男</td><td>商品学</td><td>2005</td></tr><tr><td>2005008</td><td>张文斌</td><td>18</td><td>女</td><td>法律</td><td>2005</td></tr><tr><td>...</td><td>...</td><td>...</td><td>...</td><td>...</td><td>...</td></tr></table> 元组 <table><tr><td>关系</td><td>一个关系对应一张表</td></tr><tr><td>元组</td><td>表中一行或者是一条记录叫做一个元组</td></tr><tr><td>属性</td><td>表中一列叫做一个属性，给每一个属性起一个名称叫做属性名</td></tr><tr><td>码</td><td>用于唯一确定一个元组</td></tr><tr><td>域</td><td>属性的取值范围</td></tr><tr><td>分量</td><td>元组的每一个属性值，关系的每一个分量都不可以再分</td></tr><tr><td>关系模式</td><td>对关系的描述，必须是规范化的</td></tr></table>	学号	姓名	年龄	性别	系名	年级	2005004	王小明	19	女	社会学	2005	2005006	黄大鹏	20	男	商品学	2005	2005008	张文斌	18	女	法律	2005	...	...	...	...	...	...	关系	一个关系对应一张表	元组	表中一行或者是一条记录叫做一个元组	属性	表中一列叫做一个属性，给每一个属性起一个名称叫做属性名	码	用于唯一确定一个元组	域	属性的取值范围	分量	元组的每一个属性值，关系的每一个分量都不可以再分	关系模式	对关系的描述，必须是规范化的	
学号	姓名	年龄	性别	系名	年级																																										
2005004	王小明	19	女	社会学	2005																																										
2005006	黄大鹏	20	男	商品学	2005																																										
2005008	张文斌	18	女	法律	2005																																										
...	...	...	...	...	...																																										
关系	一个关系对应一张表																																														
元组	表中一行或者是一条记录叫做一个元组																																														
属性	表中一列叫做一个属性，给每一个属性起一个名称叫做属性名																																														
码	用于唯一确定一个元组																																														
域	属性的取值范围																																														
分量	元组的每一个属性值，关系的每一个分量都不可以再分																																														
关系模式	对关系的描述，必须是规范化的																																														
	优缺点	优点 建立在严格的数学概念的基础上 关系模型的概念单一，无论实体还是联系都用关系表示，对数据的检索和更新结果也是关系。因此简单、清晰，易用 关系模型的存取路径对用户透明 缺点 查询效率往往不如格式化数据模型 开发相应数据库管理系统难度大																																													

3.1 SQL概述

2022年3月2日 12:32

导航

红色 重点 橙色
蓝色
会背
了解 绿色：可能

≡ ≧ ≡ ♀ ♀

SQL

SQL (Structured Query Language)：结构化查询语言，是关系数据库的标准语言。是一个通用的、功能极强的关系数据库语言。

SQL特点：

1. 综合统一

集数据定义语言 (DDL)，数据操纵语言 (DML)，数据控制语言 (DCL) 功能于一体。可以独立完成数据库生命周期中的全部活动：定义和修改、删除关系模式，定义和删除视图，插入数据，建立数据库；对数据库中的数据进行查询和更新；数据库重构和维护，数据库安全性、完整性控制，以及事务控制。嵌入式SQL和动态SQL定义，用户数据库投入运行后，可根据需要随时逐步修改模式，不影响数据库的运行。数据操作符统一。

2. 高度非过程化

非关系数据模型的数据操纵语言“面向过程”，必须指定存取路径。

SQL只要提出“做什么”，无须了解存取路径。

存取路径的选择以及SQL的操作过程由系统自动完成。

3. 面向集合的操作方式

非关系数据模型采用面向记录的操作方式，操作对象是一条记录。

SQL采用集合操作方式，操作对象、查找结果可以是元组的集合。一次插入、删除、更新操作的对象可以是元组的集合

4. 以同一种语法结构提供多种使用方式

SQL是独立的语言，能够独立地用于联机交互的使用方式。SQL又是嵌入式语言，SQL能够嵌入到高级语言（例如C，C++，Java）程序中，供程序员设计程序时使用

第一节：SQL概述(2.26)

2022年1月3日 12:02

SQL的产生与发展	SQL发展历程 SQL提出 1974 1986年 1989年 1999年 2010年 SQL/89 SQL/99 SQL2011 ANSI批准SQL作为关系数据库语言 公布SQL标准文本，简称SQL-86 CSDN @快乐江湖											
SQL特点	综合统一 SQL集数据定义语言、数据操纵语言、数据控制语言的功能于一体，语言风格统一，可以独立完成数据库生命周期中的全部活动，包括以下一系列操作要求 高度非过程化 用SQL进行数据操作时，只要提出“做什么”，而无须指明“怎么做”，因此无须了解存取路径。存取路径的选择以及SQL的操作过程由系统自动完成 面向集合的操作方式 而SQL采用集合操作方式，不仅操作对象、查找结果可以是元组的集合，而且一次插入、删除、更新操作的对象也可以是元组的集合 以同一种语法结构提供多种使用方式 SQL可作为独立语言：SQL既是独立的语言，又是嵌入式语言。作为独立的语言，它能够独立地用于联机交互的使用方式，用户可以在终端键盘上直接键入SQL命令对数据库进行操作 SQL可作为嵌入式语言：SQL语句可以嵌入到高级语言（例如C++、Java等）程序中，供程序员设计程序时使用 而且在这两种不同的使用方式下，其语法结构仍然基本是一致的 语言简洁，易于使用 完成表 功能只用9个动词 <table><tr><th>SQL 功能</th><th>动词</th></tr><tr><td>数据查询</td><td>SELECT</td></tr><tr><td>数据定义</td><td>CREATE, DROP, ALTER</td></tr><tr><td>数据操纵</td><td>INSERT, UPDATE, DELETE</td></tr><tr><td>数据控制</td><td>GRANT, REVOKE</td></tr></table> CSDN @快乐江湖	SQL 功能	动词	数据查询	SELECT	数据定义	CREATE, DROP, ALTER	数据操纵	INSERT, UPDATE, DELETE	数据控制	GRANT, REVOKE	
SQL 功能	动词											
数据查询	SELECT											
数据定义	CREATE, DROP, ALTER											
数据操纵	INSERT, UPDATE, DELETE											
数据控制	GRANT, REVOKE											
SQL的基本概念	SQL 视图1 视图2 基本表1 基本表2 基本表3 基本表4 存储文件1 存储文件2 外模式 模式 内模式 外模式包括若干视图和若干基本表。 数据库模式包括若干基本表。 内模式包括若干存储文件。 CSDN @快乐江湖											
基本数据类型	略											

第二、三节：数据定义(2.26)

2022年1月3日 15:00

概述	<table><tr><th rowspan="2">操作对象</th><th colspan="3">操作方式</th></tr><tr><th>创建</th><th>删除</th><th>修改</th></tr><tr><td>模式</td><td>CREATE SCHEMA</td><td>DROP SCHEMA</td><td></td></tr><tr><td>表</td><td>CREATE TABLE</td><td>DROP TABLE</td><td>ALTER TABLE</td></tr><tr><td>视图</td><td>CREATE VIEW</td><td>DROP VIEW</td><td></td></tr><tr><td>索引</td><td>CREATE INDEX</td><td>DROP INDEX</td><td>ALTER INDEX</td></tr></table> 一个数据库管理系统的实例中，可以建立多个数据库，一个数据库中可以有多个表，一个表下面通常包括多个表、视图和索引等数据库对象。 注意修改视图和修改模式时只能删除重建			操作对象	操作方式			创建	删除	修改	模式	CREATE SCHEMA	DROP SCHEMA		表	CREATE TABLE	DROP TABLE	ALTER TABLE	视图	CREATE VIEW	DROP VIEW		索引	CREATE INDEX	DROP INDEX	ALTER INDEX
操作对象	操作方式																									
	创建	删除	修改																							
模式	CREATE SCHEMA	DROP SCHEMA																								
表	CREATE TABLE	DROP TABLE	ALTER TABLE																							
视图	CREATE VIEW	DROP VIEW																								
索引	CREATE INDEX	DROP INDEX	ALTER INDEX																							
模式的定义和删除 (SCHEMA)	定义模式	语法： CREATE SCHEMA <模式名> AUTHORIZATION <用户名> （注意上面语法说明） - 如果没有指定 <模式名>，那么<模式名>隐含为<用户名> - 定义模式本质是定义了一个命名空间 - 创建模式的同时也可以创建其它东西： CREATE SCHEMA <模式名> AUTHORIZATION <用户名> [<表定义子句> <视图定义子句> <授权定义子句>]																								
	删除模式	语法： DROP SCHEMA <模式名> <CASCADE RESTRICT> （注意上面语法说明） - CASCADE：表示级联，也即删除模式时会删除该模式中所有数据库对象 - RESTRICT：表示限制，也即在删除时如果该模式下定义了其它对象，则拒绝																								
基本表的定义、删除和修改 (TABLE)	定义基本表	语法	<pre>CREATE TABLE <表名> (<列名><数据类型> [列级完整性约束条件] [,<列名><数据类型> [列级完整性约束条件]] ... [,<表级完整性约束条件>]);</pre> - 建表的同时通常还可以定义与该表有关的完整性约束条件（参照前几节内容） - 如果完整性约束条件涉及该表的多个属性列，则必须定义在表级上，否则既可以在列级上也可以在表级上																							
	例子	【例1】：建立学生表 Student，其中 Sno 是主码，且 Sname 不能重复 <pre>1 CREATE TABLE Student 2 (3 Sno CHAR(9) PRIMARY KEY, // 主码 4 Sname CHAR(20) UNIQUE, // 唯一值 5 Ssex CHAR(2), 6 Sage SMALLINT, 7 Sdept CHAR(20) 8);</pre>																								

【例2】：建立课程表 `Course`，其中 `Cno` 是主码，`Cname` 不能取空值，`Cpno` 代表先修课（意思是学习该课前必须先学习某个课程），它是外码，需要参照本表，参照列为 `Cno`

```
1 CREATE TABLE Course
2 (
3     Cno CHAR(4) PRIMARY KEY, // 主码
4     Cname CHAR(20) NOT NULL, // 非空
5     Cpno CHAR(4),
6     Credit SMALLINT,
7     FOREIGN KEY (Cpno) REFERENCES Course(Cno) // 表级完整性, Cpno 是外码, 被参照表示
8 );
```

【例3】：建立学生选课表 `SC`，其中 `Sno` 和 `Cno` 是外码，分别参照 `Student` 表的 `Sno` 列和 `Course` 表的 `Cno` 列

- `Sno` 和 `Cno` 是 `SC` 的主码，必须使用表级完整性定义

```
1 CREATE TABLE SC
2 (
3     Sno CHAR(9),
4     Cno CHAR(4),
5     Grade SMALLINT,
6
7     PRIMARY KEY (Sno, Cno), // 必须使用表级完整性定义
8     FOREIGN KEY (Sno) REFERENCES Student(Sno),
9     FOREIGN KEY (Cno) REFERENCES Course(Cno)
10 );
```

修改基本表

语法

```
ALTER TABLE <表名>
[ADD [COLUMN] <新列名><数据类型> [完整性约束]]
[ADD <表级完整性约束>]
[DROP [COLUMN] <列名> [CASCADE|RESTRICT]]
[DROP CONSTRAINT<完整性约束名> [RESTRICT|CASCADE]]
[ALTER COLUMN <列名><数据类型>];
```

- `ADD`：用于增加新列，新的列级完整性约束条件和新的表级完整性约束条件
- `DROP COLUMN`：用于删除表中的列
- `DROP CONSTRAINT`：用于删除指定的完整性约束条件
- `ALTER COLUMN`：用于修改原有的列定义

例子

【例4】：向 `Student` 表中增加“入学时间”列，数据类型为“日期”

```
1 ALTER TABLE Student ADD Sentrance DATE;
```

【例5】：将 `Student` 表“年龄”这一列的数据类型由改为 `INT`

```
1 ALTER TABLE Student ALTER COLUMN Sage CHAR;
```

【例6】：`Course` 表中的 `Cname` 必须取唯一值

```
1 ALTER TABLE Course ADD UNIQUE(Cname);
```

【例7】：删除 `Student` 表中的“入学时间”列

```
1 ALTER TABLE Student DROP COLUMN Sentrance CASCADE;
```

	<table border="1"> <tr> <td data-bbox="416 96 544 472">删除基本表</td><td data-bbox="544 96 1572 472"> 语法 语法： <pre>DROP TABLE <表名> [RESTRICT CASCADE];</pre> • 选择 RESTRICT：欲删除的基本表不能被其他表的约束所引用（比如CHECK、FOREIGN KEY等）、不能有视图、不能有触发器（trigger），不能有存储过程或函数等 • 选择 CASCADE：没有限制条件，所有相关依赖对象连同基本表一起删除 </td></tr> <tr> <td data-bbox="416 472 544 846">例子</td><td data-bbox="544 472 1572 846"> 【例9】：删除 Student 表，使用 RESTRICT • 由于 SC 表参照的是 Student，所以删除不成功 <pre>1 DROP TABLE student RESTRICT;</pre> 信息 概况 状态 [SQL]DROP TABLE student RESTRICT; [Err] 3730 - Cannot drop table 'student' referenced by a foreign key constraint 'sc_ibfk_1' on table 'sc'. </td></tr> </table>	删除基本表	语法 语法： <pre>DROP TABLE <表名> [RESTRICT CASCADE];</pre> • 选择 RESTRICT：欲删除的基本表不能被其他表的约束所引用（比如CHECK、FOREIGN KEY等）、不能有视图、不能有触发器（trigger），不能有存储过程或函数等 • 选择 CASCADE：没有限制条件，所有相关依赖对象连同基本表一起删除	例子	【例9】：删除 Student 表，使用 RESTRICT • 由于 SC 表参照的是 Student，所以删除不成功 <pre>1 DROP TABLE student RESTRICT;</pre> 信息 概况 状态 [SQL]DROP TABLE student RESTRICT; [Err] 3730 - Cannot drop table 'student' referenced by a foreign key constraint 'sc_ibfk_1' on table 'sc'.								
删除基本表	语法 语法： <pre>DROP TABLE <表名> [RESTRICT CASCADE];</pre> • 选择 RESTRICT：欲删除的基本表不能被其他表的约束所引用（比如CHECK、FOREIGN KEY等）、不能有视图、不能有触发器（trigger），不能有存储过程或函数等 • 选择 CASCADE：没有限制条件，所有相关依赖对象连同基本表一起删除												
例子	【例9】：删除 Student 表，使用 RESTRICT • 由于 SC 表参照的是 Student，所以删除不成功 <pre>1 DROP TABLE student RESTRICT;</pre> 信息 概况 状态 [SQL]DROP TABLE student RESTRICT; [Err] 3730 - Cannot drop table 'student' referenced by a foreign key constraint 'sc_ibfk_1' on table 'sc'.												
索引的建立与删除 (INDEX)	<table border="1"> <tr> <td data-bbox="416 846 504 1279">概述</td><td data-bbox="504 846 1572 1279"> 用户可以根据应用环境的需要在基本表上建立一个或多个索引，类型有 • 顺序文件上的索引 • B+树索引 • 散列索引 • 位图索引 索引虽然能加快查询速度，但也有缺点 • 需要占用一定的存储空间 • 会提高查询速度但是会降低更新速度 </td></tr> <tr> <td data-bbox="416 1279 504 2047">建立索引</td><td data-bbox="504 1279 1572 2047"> <table border="1"> <tr> <td data-bbox="512 1279 584 1682">语法</td><td data-bbox="584 1279 1564 1682"> <pre>CREATE [UNIQUE] [CLUSTER] INDEX <索引名> ON <表名>(<列名> [<次序>] [, <列名> [<次序>]] ...);</pre> • <表名>：要建立索引的基本表的名字 • 索引可以建立在该表的一列或多列上，各列之间使用逗号分隔 • 每个<列名>后面还可以用<次序>指定索引值的排列次序，可选ASC-升序（默认）或DESC-降序 • UNIQUE：表明此索引的每一个索引值只对应唯一的数据记录 • CLUSTER：表示需要建立聚簇索引（第七章会讲到） </td></tr> <tr> <td data-bbox="512 1682 584 2047">例子</td><td data-bbox="584 1682 1564 2047"> 【例10】：请按以下要求建立索引 • Student 表按学号升序建立唯一索引 • Course 表按课程号升序建立唯一索引 • SC 表按学号升序和课程号降序建立唯一索引 <pre>1 CREATE UNIQUE INDEX Stusno ON Student(Sno); 2 CREATE UNIQUE INDEX Coucno ON Course(Cno); 3 CREATE UNIQUE INDEX SCno ON SC(Sno ASC,Cno DESC);</pre> </td></tr> </table> </td></tr> <tr> <td data-bbox="416 2047 504 2139">修改索引</td><td data-bbox="504 2047 1572 2139"> <table border="1"> <tr> <td data-bbox="512 2047 584 2139">语法</td><td data-bbox="584 2047 1564 2139"> <pre>ALTER INDEX <旧索引名> RENAME TO <新索引名>;</pre> </td></tr> </table> </td></tr> </table>	概述	用户可以根据应用环境的需要在基本表上建立一个或多个索引，类型有 • 顺序文件上的索引 • B+树索引 • 散列索引 • 位图索引 索引虽然能加快查询速度，但也有缺点 • 需要占用一定的存储空间 • 会提高查询速度但是会降低更新速度	建立索引	<table border="1"> <tr> <td data-bbox="512 1279 584 1682">语法</td><td data-bbox="584 1279 1564 1682"> <pre>CREATE [UNIQUE] [CLUSTER] INDEX <索引名> ON <表名>(<列名> [<次序>] [, <列名> [<次序>]] ...);</pre> • <表名>：要建立索引的基本表的名字 • 索引可以建立在该表的一列或多列上，各列之间使用逗号分隔 • 每个<列名>后面还可以用<次序>指定索引值的排列次序，可选ASC-升序（默认）或DESC-降序 • UNIQUE：表明此索引的每一个索引值只对应唯一的数据记录 • CLUSTER：表示需要建立聚簇索引（第七章会讲到） </td></tr> <tr> <td data-bbox="512 1682 584 2047">例子</td><td data-bbox="584 1682 1564 2047"> 【例10】：请按以下要求建立索引 • Student 表按学号升序建立唯一索引 • Course 表按课程号升序建立唯一索引 • SC 表按学号升序和课程号降序建立唯一索引 <pre>1 CREATE UNIQUE INDEX Stusno ON Student(Sno); 2 CREATE UNIQUE INDEX Coucno ON Course(Cno); 3 CREATE UNIQUE INDEX SCno ON SC(Sno ASC,Cno DESC);</pre> </td></tr> </table>	语法	<pre>CREATE [UNIQUE] [CLUSTER] INDEX <索引名> ON <表名>(<列名> [<次序>] [, <列名> [<次序>]] ...);</pre> • <表名>：要建立索引的基本表的名字 • 索引可以建立在该表的一列或多列上，各列之间使用逗号分隔 • 每个<列名>后面还可以用<次序>指定索引值的排列次序，可选ASC-升序（默认）或DESC-降序 • UNIQUE：表明此索引的每一个索引值只对应唯一的数据记录 • CLUSTER：表示需要建立聚簇索引（第七章会讲到）	例子	【例10】：请按以下要求建立索引 • Student 表按学号升序建立唯一索引 • Course 表按课程号升序建立唯一索引 • SC 表按学号升序和课程号降序建立唯一索引 <pre>1 CREATE UNIQUE INDEX Stusno ON Student(Sno); 2 CREATE UNIQUE INDEX Coucno ON Course(Cno); 3 CREATE UNIQUE INDEX SCno ON SC(Sno ASC,Cno DESC);</pre>	修改索引	<table border="1"> <tr> <td data-bbox="512 2047 584 2139">语法</td><td data-bbox="584 2047 1564 2139"> <pre>ALTER INDEX <旧索引名> RENAME TO <新索引名>;</pre> </td></tr> </table>	语法	<pre>ALTER INDEX <旧索引名> RENAME TO <新索引名>;</pre>
概述	用户可以根据应用环境的需要在基本表上建立一个或多个索引，类型有 • 顺序文件上的索引 • B+树索引 • 散列索引 • 位图索引 索引虽然能加快查询速度，但也有缺点 • 需要占用一定的存储空间 • 会提高查询速度但是会降低更新速度												
建立索引	<table border="1"> <tr> <td data-bbox="512 1279 584 1682">语法</td><td data-bbox="584 1279 1564 1682"> <pre>CREATE [UNIQUE] [CLUSTER] INDEX <索引名> ON <表名>(<列名> [<次序>] [, <列名> [<次序>]] ...);</pre> • <表名>：要建立索引的基本表的名字 • 索引可以建立在该表的一列或多列上，各列之间使用逗号分隔 • 每个<列名>后面还可以用<次序>指定索引值的排列次序，可选ASC-升序（默认）或DESC-降序 • UNIQUE：表明此索引的每一个索引值只对应唯一的数据记录 • CLUSTER：表示需要建立聚簇索引（第七章会讲到） </td></tr> <tr> <td data-bbox="512 1682 584 2047">例子</td><td data-bbox="584 1682 1564 2047"> 【例10】：请按以下要求建立索引 • Student 表按学号升序建立唯一索引 • Course 表按课程号升序建立唯一索引 • SC 表按学号升序和课程号降序建立唯一索引 <pre>1 CREATE UNIQUE INDEX Stusno ON Student(Sno); 2 CREATE UNIQUE INDEX Coucno ON Course(Cno); 3 CREATE UNIQUE INDEX SCno ON SC(Sno ASC,Cno DESC);</pre> </td></tr> </table>	语法	<pre>CREATE [UNIQUE] [CLUSTER] INDEX <索引名> ON <表名>(<列名> [<次序>] [, <列名> [<次序>]] ...);</pre> • <表名>：要建立索引的基本表的名字 • 索引可以建立在该表的一列或多列上，各列之间使用逗号分隔 • 每个<列名>后面还可以用<次序>指定索引值的排列次序，可选ASC-升序（默认）或DESC-降序 • UNIQUE：表明此索引的每一个索引值只对应唯一的数据记录 • CLUSTER：表示需要建立聚簇索引（第七章会讲到）	例子	【例10】：请按以下要求建立索引 • Student 表按学号升序建立唯一索引 • Course 表按课程号升序建立唯一索引 • SC 表按学号升序和课程号降序建立唯一索引 <pre>1 CREATE UNIQUE INDEX Stusno ON Student(Sno); 2 CREATE UNIQUE INDEX Coucno ON Course(Cno); 3 CREATE UNIQUE INDEX SCno ON SC(Sno ASC,Cno DESC);</pre>								
语法	<pre>CREATE [UNIQUE] [CLUSTER] INDEX <索引名> ON <表名>(<列名> [<次序>] [, <列名> [<次序>]] ...);</pre> • <表名>：要建立索引的基本表的名字 • 索引可以建立在该表的一列或多列上，各列之间使用逗号分隔 • 每个<列名>后面还可以用<次序>指定索引值的排列次序，可选ASC-升序（默认）或DESC-降序 • UNIQUE：表明此索引的每一个索引值只对应唯一的数据记录 • CLUSTER：表示需要建立聚簇索引（第七章会讲到）												
例子	【例10】：请按以下要求建立索引 • Student 表按学号升序建立唯一索引 • Course 表按课程号升序建立唯一索引 • SC 表按学号升序和课程号降序建立唯一索引 <pre>1 CREATE UNIQUE INDEX Stusno ON Student(Sno); 2 CREATE UNIQUE INDEX Coucno ON Course(Cno); 3 CREATE UNIQUE INDEX SCno ON SC(Sno ASC,Cno DESC);</pre>												
修改索引	<table border="1"> <tr> <td data-bbox="512 2047 584 2139">语法</td><td data-bbox="584 2047 1564 2139"> <pre>ALTER INDEX <旧索引名> RENAME TO <新索引名>;</pre> </td></tr> </table>	语法	<pre>ALTER INDEX <旧索引名> RENAME TO <新索引名>;</pre>										
语法	<pre>ALTER INDEX <旧索引名> RENAME TO <新索引名>;</pre>												

修改索引	语法	ALTER INDEX <旧索引名> RENAME TO <新索引名>;
	例子	【例11】：将 SC 表的 SCno 索引改名为 SCSno 1 ALTER INDEX SCno RENAME TO SCSno
删除索引	语法	DROP INDEX <索引名>;
	例子	【例12】：删除 SC 表的SCSno 索引 1 DROP INDEX SCSno 2 3 // 如果失败，也可这样写 4 DROP INDEX SCSno on SC; 5 DROP INDEX SC.SCSno; 6

第四节：数据查询(2.27)

2022年1月4日 10:12

SELECT语句格式	SELECT语句含义：根据WHERE子句的条件表达式从FROM子句指定的表、视图中找出满足条件的元组，再按照SELECT子句中的目标列表式选出元组中的属性值形成结果表。如果有： • GROUP BY：结果按<列名1>的值进行分组，该属性列值相等的元组为一个组；通常会在每组中作用聚集函数；如果该子句还携带HAVING短语，则只有满足指定条件的组才予以输出 • ORDER BY：结果表还要按<列名2>的值的升序或降序排序 <pre>SELECT [ALL DISTINCT] <目标列表式> [,<目标列表式>] ... FROM <表名或视图名> [,<表名或视图名>...] [(<SELECT 语句>)[AS] <别名> [WHERE <条件表达式> [GROUP BY <列名 1> [HAVING <条件表达式>]] [ORDER BY <列名 2> [ASC DESC]] ;</pre> CSDN @快乐江湖																	
单表查询	选择表中的若干列	查询指定列	【例1】查询 <i>Student</i> 表中的学生及年龄 <pre>1 SELECT Sname,Sage from Student;</pre>															
		查询全部列	语法	• * 作为通配符表示全部														
		例子	【例2】查询 <i>Student</i> 表中全部列 <pre>1 SELECT * from Student;</pre> 复制															
		查询经过计算的 值	算数表达式	【例3】根据年龄计算学生的出生日期 <pre>1 SELECT Sname,2022-Sage from Student;</pre>														
		字符串常量及函数	【例4】使用小写字母展示所在系别 <pre>1 SELECT Sname,LOWER(Sdept) from student;</pre>															
		使用列别名改变查询结果标题	【例5】查询 <i>Student</i> 表中的 <i>Sname</i> 和 <i>Sage</i>，列标题起别名为“姓名”和“年龄” <pre>1 SELECT Sname `姓名`,Sage `年龄` from student;</pre>															
选择表中的若干行（元组）	消除取值重复的行（DISTINCT）	语法	语法：前面说过投影操作可能会导致相同的行出现所以其结果必须消除重复行。可以使用 DISTINCT 消除															
		例子	【例6】：查询 <i>SC</i> 表的 <i>Sno</i> 列，然后消除重复学号 <pre>1 SELECT DISTINCT Sno from SC;</pre>															
	查询满足条件的元组	语法	语法：通过WHERE子句实现，常用的查询条件如下 <table><tr><th>查询条件</th><th>谓词</th></tr><tr><td>比较</td><td>=, >, <, >=, <=, !=, <>, !>, !<; NOT+上述比较运算符</td></tr><tr><td>确定范围</td><td>BETWEEN AND, NOT BETWEEN AND</td></tr><tr><td>确定集合</td><td>IN, NOT IN</td></tr><tr><td>字符匹配</td><td>LIKE, NOT LIKE</td></tr><tr><td>空值</td><td>IS NULL, IS NOT NULL</td></tr><tr><td>多重条件（逻辑运算）</td><td>AND, OR, NOT</td></tr></table> CSDN @快乐江湖		查询条件	谓词	比较	=, >, <, >=, <=, !=, <>, !>, !<; NOT+上述比较运算符	确定范围	BETWEEN AND, NOT BETWEEN AND	确定集合	IN, NOT IN	字符匹配	LIKE, NOT LIKE	空值	IS NULL, IS NOT NULL	多重条件（逻辑运算）	AND, OR, NOT
		查询条件	谓词															
比较	=, >, <, >=, <=, !=, <>, !>, !<; NOT+上述比较运算符																	
确定范围	BETWEEN AND, NOT BETWEEN AND																	
确定集合	IN, NOT IN																	
字符匹配	LIKE, NOT LIKE																	
空值	IS NULL, IS NOT NULL																	
多重条件（逻辑运算）	AND, OR, NOT																	
例子	比较大小	【例7】在 <i>SC</i> 表中查询成绩大于85的同学的学号 <pre>1 SELECT Sno,Grade from SC 2 WHERE Grade > 85;</pre> 复制																
	确定范围	【例8】查血年龄在闭区间[19,20]的学生 <pre>1 SELECT Sname,Sage from student 2 WHERE Sage BETWEEN 19 AND 20;</pre>																

				【例9】查血年龄不在闭区间[19,20]的学生 <pre>1 SELECT Sname,Sage from student 2 WHERE Sage NOT BETWEEN 19 AND 20;</pre>
			确定集合	【例10】查询数字1是否在集合(1,2,3)中 由于满足，所以会返回1 <pre>1 SELECT 1 IN (1,2,3);</pre> 【例11】查询数学系（MA）和计算机科学系（CS）学生的姓名 <pre>1 SELECT Sname,Sdept from student 2 WHERE Sdept IN('MA','CS');</pre> 【例12】查询非数学系（MA）和非计算机科学系（CS）学生的姓名 <pre>1 SELECT Sname,Sdept from student 2 WHERE Sdept NOT IN('MA','CS');</pre>
			字符匹配	<pre>1 SELECT Sname from student 2 WHERE Ssex LIKE '男';</pre> 【例14】查询所有姓刘的学生（%代替多个） <pre>1 SELECT Sname FROM student 2 WHERE Sname like '刘%';</pre>
			转义字符	注意： - ESCAPE '\ ' 表示将“ \ ”翻译为转义字符 演示： 【例15】假设 Course 表中有一门课叫做 DB_Design - 如果不作处理，这里的 _ 代替某个字符，产生歧义 <pre>1 SELECT Cno,Ccredit 2 FROM Course 3 WHERE Cname LIKE 'DB_Design' ESCAPE '\';</pre>
			涉及空值的查询	【例16】查询缺少成绩的学生的学号和相应的课程号 某些学生选修课程后没有参加考试，所以有选课记录，但没有考试成绩 <pre>1 SELECT Sno,Cno FROM SC 2 WHERE Grade IS NULL;</pre>
			多重条件查询	【例17】查询计算机系年龄在20岁以下的学生姓名 <pre>1 SELECT * FROM student 2 WHERE Sdept='CS' AND Sage < 20;</pre>
ORDER BY子句	语法	语法：ORDER BY子句对查询结果按照一个或多个属性列进行排序 - ASC-升序（默认） - DESC-降序		
	例子	【例18】查询选修了3号课程的学生的学号及其成绩，查询结果按分数降序排列 <pre>1 SELECT Sno,Grade 2 from SC 3 WHERE Cno='3' 4 ORDER BY Grade DESC;</pre>		
聚集函数	语法	- 计数 <pre>COUNT ([DISTINCT ALL] *)</pre> <pre>COUNT ([DISTINCT ALL] <列名>)</pre> - 计算总和 <pre>SUM ([DISTINCT ALL] <列名>)</pre> - 计算平均值 <pre>AVG ([DISTINCT ALL] <列名>)</pre>		

聚集函数	语法 • 计数 COUNT ([DISTINCT ALL] *) COUNT ([DISTINCT ALL] <列名>) • 计算总和 SUM ([DISTINCT ALL] <列名>) • 计算平均值 AVG ([DISTINCT ALL] <列名>) • 最大最小值 MAX ([DISTINCT ALL] <列名>) MIN ([DISTINCT ALL] <列名>) CSDN @快乐江湖 例子 【例20】查询学生的总人数 <pre>1 SELECT COUNT(Sno) 2 FROM student;</pre> 复制 【例21】查询选修了课程的学生人数 <pre>1 SELECT COUNT(DISTINCT Sno) Num 2 FROM SC;</pre> 复制 【例22】计算2号课程的学生平均成绩 <pre>1 SELECT AVG(Grade) 2 FROM SC 3 WHERE Cno = '2';</pre> 【例23】查询选修2号课程的学生最高分数 <pre>1 SELECT Sno,MAX(Grade) 2 FROM SC 3 WHERE Cno='2';</pre>
GROUP BY子句	语法 语法：GROUP BY子句将查询结果按某一列或多列的值分组，值相等的分为一组 • 分组目的是为了细化聚集函数的作用对象：若未分组，聚集函数将会作用于整个查询结果；若分组，聚集函数将会作用于每一个组，也即每一个组都有一个函数值 • 需要注意：WHERE子句作用于整个表或视图，从中选择出满足条件的元组；HAVING短语作用于组，从中选择满足条件的组 例子 【例24】查询平均成绩大于等于80分的学生学号和平均成绩 <pre>1 SELECT Sno,AVG(Grade) 2 FROM SC 3 GROUP BY Sno 4 HAVING AVG(Grade) >= 80;</pre>
连接查询	等值连接和非等值连接 语法 语法：在WHERE子句中写入连接条件（又叫做连接谓词），其格式为 <pre>[<表名 1>.] <列名 1><比较运算符> [<表名 2>.] <列名 2></pre> 其中比较运算符有：=、>、<、>=、<=、!= • 当运算符为=时称之为等值连接 • 当运算符不为=时称之为非等值连接 例子 【例25】查询每个学生及其选修课程的情况 <pre>1 SELECT student.*,sc.* 2 FROM student,sc 3 WHERE student.Sno=sc.Sno;</pre> 【例26】使用自然连接（特殊的等值连接）完成【例25】 <pre>1 SELECT student.Sno,Sname,Ssex,Sage,Sdept,Cno,Grade 2 FROM student,sc 3 WHERE student.sno=sc.sno;</pre> 复制

		【例27】查询选修2号课程且成绩在80分以上的所有学生的学号和姓名 <pre>1 SELECT Student.Sno,Sname 2 FROM student,sc 3 WHERE student.Sno=sc.Sno AND //连接条件 4 Cno='2' AND Grade > 80; //其他限定条件</pre>																																																															
自身连接	语法 语法：所谓自身连接就是指一个表与自己连接 例子 【例28】查询每一门课的先修课的先修课 在 <code>Course</code> 表中有的只是每门课的直接先修课，要想得到先修课的先修课，那么就必须先找到一门课的先修课，然后再按此先修课的课程号查找它的先修课 因此，为 <code>Course</code> 表取两个别名，分别为 <code>ONE</code> 和 <code>TWO</code> 因此，为 <code>Course</code> 表取两个别名，分别为 <code>ONE</code> 和 <code>TWO</code> <table><tr><th>Cno</th><th>Cname</th><th>Cpno</th><th>Ccredit</th></tr><tr><td>1</td><td>数据库</td><td>5</td><td>4</td></tr><tr><td>2</td><td>数学</td><td></td><td>2</td></tr><tr><td>3</td><td>信息系统</td><td>1</td><td>4</td></tr><tr><td>4</td><td>操作系统</td><td>6</td><td>3</td></tr><tr><td>5</td><td>数据结构</td><td>7</td><td>4</td></tr><tr><td>6</td><td>数据处理</td><td>2</td><td>2</td></tr><tr><td>7</td><td>PASCAL 语言</td><td>6</td><td>4</td></tr></table><table><tr><th>Cno</th><th>Cname</th><th>Cpno</th><th>Ccredit</th></tr><tr><td>1</td><td>数据库</td><td>5</td><td>4</td></tr><tr><td>2</td><td>数学</td><td></td><td>2</td></tr><tr><td>3</td><td>信息系统</td><td>1</td><td>4</td></tr><tr><td>4</td><td>操作系统</td><td>6</td><td>3</td></tr><tr><td>5</td><td>数据结构</td><td>7</td><td>4</td></tr><tr><td>6</td><td>数据处理</td><td>2</td><td>2</td></tr><tr><td>7</td><td>PASCAL 语言</td><td>6</td><td>4</td></tr></table> <pre>1 SELECT ONE.Cno,TWO.Cpno 2 FROM Course ONE,Course TWO 3 WHERE ONE.Cpno=TWO.Cno;</pre> 当然，还可以继续找先修课的先修课的先修课 <pre>1 SELECT ONE.Cno,THREE.Cpno 2 FROM Course ONE,Course TWO,course THREE 3 WHERE ONE.Cpno=TWO.Cno AND TWO.Cpno=THREE.Cno;</pre>	Cno	Cname	Cpno	Ccredit	1	数据库	5	4	2	数学		2	3	信息系统	1	4	4	操作系统	6	3	5	数据结构	7	4	6	数据处理	2	2	7	PASCAL 语言	6	4	Cno	Cname	Cpno	Ccredit	1	数据库	5	4	2	数学		2	3	信息系统	1	4	4	操作系统	6	3	5	数据结构	7	4	6	数据处理	2	2	7	PASCAL 语言	6	4
Cno	Cname	Cpno	Ccredit																																																														
1	数据库	5	4																																																														
2	数学		2																																																														
3	信息系统	1	4																																																														
4	操作系统	6	3																																																														
5	数据结构	7	4																																																														
6	数据处理	2	2																																																														
7	PASCAL 语言	6	4																																																														
Cno	Cname	Cpno	Ccredit																																																														
1	数据库	5	4																																																														
2	数学		2																																																														
3	信息系统	1	4																																																														
4	操作系统	6	3																																																														
5	数据结构	7	4																																																														
6	数据处理	2	2																																																														
7	PASCAL 语言	6	4																																																														
连接JOIN	语法 语法：SQL JOIN用于把来自两个或多个表的行结合起来，其格式如下 <pre>1 SELECT column_name(s) 2 FROM TABLE1//左表 3 <某某 JOIN>TABLE2//右表 4 ON TABLE1.column_name=TABLE2.column_name</pre> 有如下几类 • <code>INNER JOIN (JOIN)</code> • <code>LEFT JOIN (LEFT OUTER JOIN)</code> • <code>RIGHT JOIN (RIGHT OUTER JOIN)</code> • <code>FULL JOIN (FULL OUTER JOIN)</code> 例子 【例29】以 <code>sc</code> 和 <code>course</code> 的 <code>Cno</code> 作为比对标准，将相同连接在一起 <pre>1 SELECT Sno,sc.Cno,Grade,course.Cno,Cname,Cpno,Ccredit 2 FROM sc INNER JOIN course ON(sc.Cno=course.Cno);</pre>																																																																
复合条件连接	语法 语法：没有什么新的东西，就是涉及多张表，多个条件的查询 例子 【例30】查询每个学生的学号、姓名、选修的课程名及成绩 <pre>1 SELECT student.Sno,Sname,Cname,Grade 2 FROM student,course,sc 3 WHERE student.Sno=sc.Sno AND sc.Cno =course.Cno;</pre>																																																																
嵌套查询	带有IN谓词的子查询 语法 语法：嵌套查询中，子查询的结果往往是一个集合，所以IN在嵌套查询中使用次数最为频繁 例子 【例31】查询与“刘晨”在同一个系学习的学生 考虑时可以由内向外，先查询出刘晨所在的系，然后在该集合中查询满足该集合的学生姓名 <pre>1 SELECT student.Sno,Sname,Sdept FROM student WHERE Sdept IN 2 (SELECT Sdept FROM student WHERE Sname='刘晨');</pre>																																																																

当然嵌套查询有时也可以转化为连接完成

```
1 SELECT S1.Sno,S1.Sname,S1.Sdept
2 FROM Student S1,Student S2
3 WHERE S1.Sdept=S2.Sdept AND S2.Sname='刘晨';
```

【例32】查询选修了课程名为“信息系统”的学生学号和姓名

- 首先在 **Course** 表中找出“信息系统”的 **Cno**，形成 **Cno** 的集合
- 然后在 **SC** 表中找出哪些记录的 **Cno** 在 **Cno** 集合内，形成 **Sno** 集合
- 最后在 **Student** 表中找出哪些记录的 **Sno** 在 **Sno** 集合内，形成最终的结果

```
1 SELECT Sno,Sname FROM Student WHERE Sno IN
2 (SELECT Sno FROM SC WHERE Cno IN
3 (SELECT Cno FROM Course WHERE Cname='信息系统'))
4 );
```

当然本例也可以使用连接完成

```
1 SELECT student.Sno,Sname
2 FROM student,course,sc
3 WHERE Student.Sno=sc.Sno AND sc.Cno=course.Cno AND Cname='信息系统';
```

带有比较运算符的子查询

语法

语法：带有比较运算符的子查询是指父查询与子查询之间用比较运算符进行连接。当用户能确切知道内层查询返回的是单个值时，可以使用 >、<、=、>=、<=、!= 等比较运算符

例子

【例33】查询与“刘晨”在同一个系学习的学生

- 和【例31】一样，只不过【例31】使用 **IN** 完成的

```
1 SELECT Sno,Sname,Sdept FROM student WHERE Sdept
2 =
3 (SELECT Sdept FROM student WHERE Sname='刘晨');
```

【例34】找出每个学生超过他选修课程平均成绩的课程号

- 首先内层循环要计算该学生的平均成绩
- 外层循环使用 >=
- 两个循环要用 **Sno** 关联在一起（相关子查询）

```
1 SELECT Sno,Cno FROM sc x WHERE Grade
2 >=
3 (SELECT AVG(Grade) from sc WHERE x.Sno=Sno);
```

带有ANY (SOME) 或ALL谓词的子查询

语法

语法：内层查询返回单个值时使用比较运算符。如果返回多个值要用 **ANY**（有的是SOME）或 **ALL**，然后同时使用比较运算符

> ANY	大于子查询结果中的某个值
> ALL	大于子查询结果中的所有值
< ANY	小于子查询结果中的某个值
< ALL	小于子查询结果中的所有值
>= ANY	大于等于子查询结果中的某个值
>= ALL	大于等于子查询结果中的所有值
<= ANY	小于等于子查询结果中的某个值
<= ALL	小于等于子查询结果中的所有值
= ANY	等于子查询结果中的某个值
= ALL	等于子查询结果中的所有值（通常没有实际意义）
!=（或<>）ANY	不等于子查询结果中的某个值
!=（或<>）ALL	不等于子查询结果中的任何一个值

另外 **ANY** 或 **ALL** 与聚集函数、**IN** 等谓词有等价关系。也就是说在一些情况下，他们的写法效果作用是一样的

	=	<>或!=	<	<=	>	>=
ANY	IN	--	<MAX	<=MAX	>MIN	>=MIN
ALL	--	NOT IN	<MIN	<=MIN	>MAX	>=MAX

例子

【例35】查询其他系比计算机科学系任意一个学生年龄小的学生姓名和年龄

```
1 SELECT Sname,Sage FROM student WHERE Sage < ANY
2 (SELECT Sage FROM student WHERE Sdept='CS')
3 AND Sdept!='CS';
```

		由于 <ANY 等价于 <MAX，所以其等价写法就是 <pre>1 SELECT Sname,Sage FROM student WHERE Sage < 2 (SELECT MAX(Sage) FROM student WHERE Sdept='CS') 3 AND Sdept!='CS';</pre>
带有EXISTS谓词的子查询	语法 语法：EXISTS代表“存在量词$\exists$”。带有EXISTS谓词的子查询不返回任何数据，只返回 true 或 false。另外，由EXISTS引出的子查询，其目标列表表达式通常都是 *，因为给出列名没有实际意义 • 若内层查询结果非空，则外层WHERE子句返回 true • 若内层查询结果为空，则外层WHERE子句返回 false 与EXISTS相对的便是NOT EXISTS • 若内层查询结果为空，则外层WHERE子句返回 true • 若内层查询结果非空，则外层WHERE子句返回 false 需要注意的是，一些带有EXISTS和NOT EXISTS谓词的子查询不能被其他形式的子查询等价替换；但是所有带IN谓词，比较运算符，ANY和ALL谓词的子查询都可以用带EXISTS谓词的子查询替换 例子 【例36】查询所有选修了1号课程的学生姓名 • 处理时，首先会取外层查询中 Student 表的第一个元组，根据它与内层查询相关的属性值（Sno）处理内层查询，若 WHERE 子句返回为 true 则取外层查询中该元组的 Sname 放入结果表 <pre>1 SELECT Sname FROM student WHERE 2 EXISTS 3 (SELECT * from sc where Sno=student.Sno AND Cno='1');</pre> 【例37】查询与“刘晨”在同一个系学习的学生 • 和【例31】一样，这里用EXISTS代替IN <pre>1 SELECT Sno,Sname Sdept FROM student s1 WHERE 2 EXISTS 3 (select * FROM student WHERE Sdept=s1.Sdept AND Sname='刘晨');</pre> 复制	
集合查询	语法 语法：SELECT语句的查询结果是元组的集合，所以多个SELECT语句的结果可进行集合操作。主要有： • 并操作： UNION • 交操作： INTERSECT • 差操作： EXCEPT 需要注意的是查询结果的列数必须相同且对应项的数据类型也必须相同 例子 【例39】查询计算机科学系的学生及年龄不大于19岁的学生 <pre>1 SELECT Sname,Sage,Sdept FROM student WHERE Sdept='CS' 2 UNION 3 SELECT Sname,Sage,Sdept FROM student WHERE Sage<=19;</pre>	

第五节：数据更新

2022年1月6日 9:39

插入数据	插入元组	语法：格式如下，用于将新元组插入指定表中。需要注意 INTO 子句中没有出现的属性列，新元组在这些列上将会取 NULL - 若 INTO 子句中没有指明任何属性列名，则新插入的元组必须在每个属性列上均有值 <pre>INSERT INTO <表名> [(<属性列 1> [,<属性列 2>] ...)] VALUES (<常量 1> [,<常量 2>] ...);</pre>
	例子	【例1】将一个新学生元组（学号：200215128；姓名：陈冬；性别：男；所在系：IS；年龄：18岁）插入到Student表中 <pre>1 INSERT 2 INTO student(Sno,Sname,Ssex,Sdept,Sage) 3 VALUES('201215128','陈冬','男','IS',18);</pre> - 注意顺序可以和表不一致 【例2】插入学生张成民 <pre>1 INSERT 2 INTO student 3 VALUES('201215126','张成民','男',18,'CS');</pre> - 注意由于没有指定顺序，所以按照必须按照表中属性列的顺序插入，否则会导致插入错误 【例3】插入一条选课记录（201215128,1） <pre>1 INSERT 2 INTO sc(Sno,Cno) 3 values('201215128','1');</pre> - 在这种情况下，Grade 属性列会设置为 NULL - 需要注意如果没有给出属性列，则必须执行 Grade 为 NULL 【例4】插入多条记录 <pre>1 INSERT 2 INTO student 3 VALUES 4 (201515000,'小赵','男',30,'IS'), 5 (201515001,'小钱','女',28,'MA'), 6 (201515002,'小孙','男',33,'MJ'), 7 (201515003,'小李','女',25,'CS'), 8 (201515004,'小周','男',41,'LI');</pre>
插入子查询结果	语法	语法：格式如下，子查询同样可以嵌套在 INSERT 语句中用于生成待插入的批量数据 <pre>INSERT INTO <表名> [(<属性列 1> [,<属性列 2>] ...)] 子查询;</pre>
	例子	【例5】对每一个系，求学生的平均年龄，并把结果存入数据库 <pre>1 CREATE TABLE dept_age(2 Sdept char(15), 3 Age SMALLINT 4); 5 6 INSERT 7 INTO dept_age(Sdept,Age) 8 SELECT Sdept,AVG(Sage) 9 FROM student 10 GROUP BY Sdept;</pre>
修改数据	修改某一个元组的值	【例6】将学生201215121的年龄改为22岁 <pre>1 UPDATE student 2 set Sage=22 3 WHERE Sno='201215121';</pre>

修改多个元组的值	【例7】将所有学生的年龄增加1岁 <pre>1 UPDATE student 2 set Sage=Sage+1;</pre>
带子查询的修改语句	【例8】将CS系所有学生的成绩置0 <pre>1 UPDATE sc 2 SET Grade=0 3 WHERE Sno IN 4 (5 SELECT Sno FROM student WHERE Sdept='CS' 6);</pre> 复制

删除数据（DELETE）	语法	语法：格式如下，其功能是从指定表中删除满足 WHERE 子句条件的所有元组，注意 • DELETE 删除的是表的数据，而不是表的定义 • 如果省略 WHERE 子句，那么就表示删除全部元组 <pre>DELETE FROM <表名> [WHERE <条件>];</pre>
	例子	【例9】删除学号为201215128的学生记录 <pre>1 DELETE 2 FROM student 3 WHERE Sno='201215128';</pre> 【例10】删除所有的学生选课记录 <pre>1 DELETE 2 FROM SC;</pre> 【例11】删除计算机科学系所有学生的选课记录 <pre>1 DELETE FROM sc 2 WHERE Sno IN 3 (SELECT Sno FROM student WHERE Sdept='CS');</pre>

关于视图	什么视图	视图：视图是一个虚表，其本质就是一条 SELECT 语句，而查询结果被赋予了一个名字，也即视图名字。或者说视图本身不包含任何数据，它只包含映射到基表的一个查询语句，当基表数据发生变化时，视图数据也随之变化。其目的就在于方便，简化数据操作
	什么时候会用到视图	简单点说，当我们感觉查询语句太过复杂且不好操控时就可以创建视图，直接 <pre>1 SELECT * FROM [view]</pre> 复制
	视图的作用	• 视图能够简化用户的操作 • 视图使用户能以多种角度看待同一数据 • 视图对重构数据库提供了一定程度的逻辑独立性 • 视图能够对机密数据提供安全保护 • 适当的利用视图可以更清晰的表达查询
视图的定义和删除	建立视图	语法：使用 CREATE VIEW 语句创建视图，格式如下 <pre>CREATE VIEW <视图名> [(<列名> [<列名>] ...)] AS <子查询> [WITH CHECK OPTION];</pre> • 子查询可以是任意的 SELECT 语句（是可以含有 ORDER BY 子句和 DISTINCT 短语取决于具体系统） • 组成视图的属性列名要么全部省略要么全部指定，不能有第三种情况 • 如果省略视图列名，则其列名默认由 SELECT 子句目标列诸字段组成
	简单创建	【例1】建立信息系IS学生的视图 <pre>1 CREATE VIEW IS_student 2 AS 3 SELECT Sno,Sname,Sage,Sdept 4 FROM student 5 WHERE Sdept='IS'</pre> 复制
	基于多个基表的视图	【例2】建立计算机科学系选修了1号课程的学生视图 <pre>1 CREATE VIEW CS1 2 AS 3 SELECT student.Sno Sno,Sname,Grade 4 FROM student,sc 5 WHERE Sdept='CS' AND student.Sno=sc.Sno AND sc.Cno='1';</pre>
	基于视图的视图	【例3】建立计算机科学系选修了1号课程且成绩大于95分的学生的视图 <pre>1 CREATE VIEW CS2 2 AS 3 SELECT * 4 FROM CS1 5 WHERE Grade > 95;</pre> 复制
	带表达式的视图	【例4】定义一个反映学生出生年份的视图 <pre>1 CREATE VIEW birthday(Sno,Sname,Syear) 2 AS 3 SELECT Sno,Sname,2022-Sage 4 FROM student;</pre> 复制

	分组视图 【例5】将学生的学号及他的平均成绩定义为一个视图 <pre>1 CREATE VIEW stu_grade_avg(Sno,Savg) 2 AS 3 SELECT Sno,AVG(Grade) 4 FROM sc 5 GROUP BY Sno;</pre>
	删除视图 语法：格式如下，需要注意 基本表删除之后，由该基本表导出的所有视图均无法使用，但是视图的定义没有从字典中清除 <pre>DROP VIEW <视图名> [CASCADE];</pre>
视图的查询	语法：从用户角度出发，查询视图和查询基本表相同；从DBMS角度出发，采用视图消解法，具体来讲 - 首先进行有效性检查 - 接着转换成等价的对基本表的查询 - 最后执行修正后的查询 这里具体就不再做过多演示了，一般来说转换是能成功进行的，更多细节大家可以了解课本（这部分内容不多）
视图的更新	语法：视图是虚表，所以对视图的更新最终会转化为对基本表的更新。为了防止用户通过视图对数据进行更新时，有意或无意地对不属于视图范围内的基本表数据进行操作，可以在定义视图时加上 WITH CHECK OPTION 子句。这样在更新时，如果不满足条件，DBMS会拒绝操作 UPDATE 【例6】将信息系学生视图 IS_Student 中学号201215125的学生姓名改为“刘辰” <pre>1 UPDATE is_student 2 SET Sname='刘辰' 3 WHERE Sno='201215125';</pre> 【例7】如果在定义视图 is_student 在定义时加入了 WITH CHECK OPTION 子句，接着再执行【例6】需求 那么在更新时如果将 Sdept 字段改为了'MA'或其他值，DBMS就会拒绝执行，并下面的错误反馈 <pre>1 UPDATE is_student 2 SET Sdept='MA' 3 WHERE Sno='201215125';</pre> 如果在更新时只修改名字，那么就没有问题 <pre>1 UPDATE is_student 2 SET Sname='德玛' 3 WHERE Sno='201215125';</pre> INSERT 【例8】向信息系学生视图 IS_Student 中插入一个新的学生记录：201215129，赵新，20岁 <pre>1 INSERT INTO is_student 2 VALUES('201215129','赵新',20);</pre> DELETE 【例9】删除学号为201215125的学生 <pre>1 DELETE 2 FROM is_student 3 WHERE Sno='201215125';</pre>

第一节：关系数据结构及其形式化定义(2.25)

2022年1月17日 13:04

关系	前面说过，关系模型建立在严格的数学概念之上，只包含单一的数据结构——关系，在用户看来关系就是一张 二维表				
	域	是一组具有相同数据类型的值的集合 • 如自然数、整数、实数			
	笛卡尔积	是一种域上面的 集合运算 ，得到 每个域中元素的所有可能的组合 ，它可以表示为一张二维表，在笛卡尔积中有 • 元组：每一行是一个元组，比如 (张清玫, 计算机专业, 李勇)、(张清玫, 计算机专业, 刘晨)等都是元组； • 分量：每一行中每一个元素都是一个分量，比如张清玫、计算机专业、李勇、刘晨等都是分量			
	关系	<table><tr><td>基本概述</td><td> 关系：笛卡尔积 $(D_1 \times D_2, \dots, D_n)$ 的子集叫做在域 $D_1、D_2, \dots, D_n$ 的关系，表示为 $R(D_1、D_2, \dots, D_n)$,其中 • R：关系名 • n：关系的目或度 (n=1时称为单元关系，n=2时称为二元关系) 关系既然是笛卡尔积的子集（有限子集），所以关系也是一张二维表 • 表的每一行对应一个元组，表的每一列对应一个域 • 由于域可以相同，为了区分，必须对每列起一个名字，称为属性（比如上面的表中研究生和导师都是人，为了区分，所以才取了不同的名字） </td></tr><tr><td>码相关概念</td><td> 以下面关系为例 有关系模式R（学号、姓名、性别、课程名、期末分数），假设姓名会重复，课程名不重复，忽略表设计。 候选码：若关系中的某一属性组（注意是组不是某个属性，当然有时属性组也可能只有一个属性）能唯一地标识一个元组，而其子集不能，则该属性组称之为候选码 • 上面关系中，学号是无法区分的，因为学号虽然不重复，但一个学生可能会对应多个课程，这就导致学号无法唯一标识一个元组。因此这里（学号，课程名）可以作为一个候选码 • 需要注意的是候选码不一定只有一个，可能有多个，只要满足条件即可，但在本例中确实只有一个 超码：能够唯一标识一条记录的属性或属性集，超码是候选码的扩充，候选码是最小的超码 • 上面关系中，（学号、课程名）是候选码，那么它的超集，例如（学号、课程名、姓名）、（学号、课程名、性别）就是超码 主码：某个能够唯一标识一条记录的最小属性集（候选码中的“人選之子”） • 候选码可能有多个，但是数据库设计者在设计时会根据实际需求选择一个候选码作为主码 外码：是本关系的属性且不是码，而是另一个关系的主码（后续会详细介绍） 全码：这是一种特殊情况：关系的所有属性组是这个关系模式的候选码 </td></tr></table>	基本概述	关系：笛卡尔积 $(D_1 \times D_2, \dots, D_n)$ 的子集叫做在域 $D_1、D_2, \dots, D_n$ 的关系，表示为 $R(D_1、D_2, \dots, D_n)$,其中 • R：关系名 • n：关系的目或度 (n=1时称为单元关系，n=2时称为二元关系) 关系既然是笛卡尔积的子集（有限子集），所以关系也是一张二维表 • 表的每一行对应一个元组，表的每一列对应一个域 • 由于域可以相同，为了区分，必须对每列起一个名字，称为属性（比如上面的表中研究生和导师都是人，为了区分，所以才取了不同的名字）	码相关概念
基本概述	关系：笛卡尔积 $(D_1 \times D_2, \dots, D_n)$ 的子集叫做在域 $D_1、D_2, \dots, D_n$ 的关系，表示为 $R(D_1、D_2, \dots, D_n)$,其中 • R：关系名 • n：关系的目或度 (n=1时称为单元关系，n=2时称为二元关系) 关系既然是笛卡尔积的子集（有限子集），所以关系也是一张二维表 • 表的每一行对应一个元组，表的每一列对应一个域 • 由于域可以相同，为了区分，必须对每列起一个名字，称为属性（比如上面的表中研究生和导师都是人，为了区分，所以才取了不同的名字）				
码相关概念	以下面关系为例 有关系模式R（学号、姓名、性别、课程名、期末分数），假设姓名会重复，课程名不重复，忽略表设计。 候选码：若关系中的某一属性组（注意是组不是某个属性，当然有时属性组也可能只有一个属性）能唯一地标识一个元组，而其子集不能，则该属性组称之为候选码 • 上面关系中，学号是无法区分的，因为学号虽然不重复，但一个学生可能会对应多个课程，这就导致学号无法唯一标识一个元组。因此这里（学号，课程名）可以作为一个候选码 • 需要注意的是候选码不一定只有一个，可能有多个，只要满足条件即可，但在本例中确实只有一个 超码：能够唯一标识一条记录的属性或属性集，超码是候选码的扩充，候选码是最小的超码 • 上面关系中，（学号、课程名）是候选码，那么它的超集，例如（学号、课程名、姓名）、（学号、课程名、性别）就是超码 主码：某个能够唯一标识一条记录的最小属性集（候选码中的“人選之子”） • 候选码可能有多个，但是数据库设计者在设计时会根据实际需求选择一个候选码作为主码 外码：是本关系的属性且不是码，而是另一个关系的主码（后续会详细介绍） 全码：这是一种特殊情况：关系的所有属性组是这个关系模式的候选码				

		主属性和非属性：包含在候选码中的属性（注意是集合，不是某个候选码）称为主属性；不包含在候选码中的属性称为非主属性 上面关系中，姓名、性别和期末分数都是非主属性
	关系的三种类型	基本关系（又称为基本表）：实际存在的表，是实际存储数据的逻辑表示 查询表：查询结果对应的表 视图表：由基本表或其他视图表导出的表，是虚表，不对应实际存储的数据
关系模式	关系模式：关系模式就是对关系的描述，可以表示为 $R(U, D, DOM, F)$ R：关系名 U：组成该关系的属性名集合 D：U中属性所来自的域 DOM：属性向域的映像集合 F：属性间数据的依赖关系集合（此部分属于第六章：关系数据理论的内容） 关系模式通常可以简记为（重点用） $R(U)$ 或 $R(A_1, A_2, ..., A_n)$ R：关系名 $A_1, A_2, ..., A_n$：属性名 “域名”及“属性向域的映像”常常直接说明为属性的类型和长度	
关系数据库	基本概念	关系数据库：在一个给定的应用领域中，所有关系的集合构成的一个关系数据库
	关系数据库的型与值	关系数据库的型：也称为关系数据库模式，是对关系数据库的描述，包括 - 若干域的定义 - 在这些域上定义的若干关系模式 关系数据库的值：这些关系模式在某一时刻对应的关系的和，通常就叫做关系数据库

第二、三节：关系操作和关系完整性(2.25)

2022年1月17日 15:02

关系操作	基本的关系操作	关系操作的特点是 集合操作方式 ，也即操作的对象和结果都是集合。也称为 一次一集合 （set-at-a-time）的方式									
		关系操作 查询 (query) 更新 (update) 选择 (select) 投影 (project) 并 (union) 差 (except) 笛卡尔积 连接 (join) 除 (divide) 交 (intersection) 插入 (insert) 删除 (delete) 修改 (modify) 5种基本操作 CSDN @快乐江湖									
	关系数据语言的分类	关系数据语言 关系代数语言 (例如ISBL) 关系演算语言 具有关系代数和关系演算双重特点的语言—SQL 元组关系演算语言 (例如ALPHA, QUEL) 域关系演算语言 (例如DQBE) 抽象语言 SQL是集数据定义、控制、查询、操纵于一体的关系数据库的标准语言 CSDN @快乐江湖									
关系完整性	实体完整	实体完整性：若属性A是基本关系R的主属性，则属性A不能取空值 例如“选修（学号,课程号,成绩）”关系中，若（学号,课程号）为主码，则学号和课程号都不能取空值									
	参照完整性	<table><tr><td>参照关系</td><td>(见博客)</td></tr><tr><td>外码</td><td> 外码：设F是基本关系R的一个或一组属性，但不是关系R的码【例如这里F是学生关系的专业号】，K_s是基本关系S的主码【例如这里K_s是专业关系的主码】。若F和K_s相对应，则称F是R的外码(foreign key)【则称专业号是学生关系的外码】，并称基本关系R为参照关系(referencing relation)【这里学生关系就是参照关系】，基本关系S为被参照关系(referenced relation)【这里专业关系就是被参照关系】。关系R和S不一定是不同的关系 - 注意：外码并不一定要与相应的主码同名，如上面第三个例子中，学生关系的主码为学号，外码为班长。不过，在实际应用中为了便于识别，当外码与相应的主码属于不同关系时，往往给它们取相同的名字。 学生关系 专业号 专业关系 学生关系 学号 选修关系 课程号 课程关系 (a) (b) 学生关系 学号 (c) CSDN @快乐江湖 </td></tr><tr><td>参照完整性规则</td><td> 参照完整性：若属性或属性组F是基本关系R的外码，它与基本S的主码相对应（关系R和S不一定是不同的关系），则对于R中每个元组在F上的值必须 - 要么取空值（此时F的每个属性值均为空值） - 要么等于S中某个元组的主码值 </td></tr><tr><td>用户自定义完整性</td><td colspan="2">用户自定义完整性针对某一具体关系数据库的约束条件，反映某一具体应用所涉及的数据必须满足的语义要求，例如某个属性必须取唯一值，某个非主属性不能取空值等等</td></tr></table>	参照关系	(见博客)	外码	外码：设F是基本关系R的一个或一组属性，但不是关系R的码【例如这里F是学生关系的专业号】，K_s是基本关系S的主码【例如这里K_s是专业关系的主码】。若F和K_s相对应，则称F是R的外码(foreign key)【则称专业号是学生关系的外码】，并称基本关系R为参照关系(referencing relation)【这里学生关系就是参照关系】，基本关系S为被参照关系(referenced relation)【这里专业关系就是被参照关系】。关系R和S不一定是不同的关系 注意：外码并不一定要与相应的主码同名，如上面第三个例子中，学生关系的主码为学号，外码为班长。不过，在实际应用中为了便于识别，当外码与相应的主码属于不同关系时，往往给它们取相同的名字。 学生关系 专业号 专业关系 学生关系 学号 选修关系 课程号 课程关系 (a) (b) 学生关系 学号 (c) CSDN @快乐江湖	参照完整性规则	参照完整性：若属性或属性组F是基本关系R的外码，它与基本S的主码相对应（关系R和S不一定是不同的关系），则对于R中每个元组在F上的值必须 - 要么取空值（此时F的每个属性值均为空值） - 要么等于S中某个元组的主码值	用户自定义完整性	用户自定义完整性针对某一具体关系数据库的约束条件，反映某一具体应用所涉及的数据必须满足的语义要求，例如某个属性必须取唯一值，某个非主属性不能取空值等等	
	参照关系	(见博客)									
外码	外码：设F是基本关系R的一个或一组属性，但不是关系R的码【例如这里F是学生关系的专业号】，K_s是基本关系S的主码【例如这里K_s是专业关系的主码】。若F和K_s相对应，则称F是R的外码(foreign key)【则称专业号是学生关系的外码】，并称基本关系R为参照关系(referencing relation)【这里学生关系就是参照关系】，基本关系S为被参照关系(referenced relation)【这里专业关系就是被参照关系】。关系R和S不一定是不同的关系 注意：外码并不一定要与相应的主码同名，如上面第三个例子中，学生关系的主码为学号，外码为班长。不过，在实际应用中为了便于识别，当外码与相应的主码属于不同关系时，往往给它们取相同的名字。 学生关系 专业号 专业关系 学生关系 学号 选修关系 课程号 课程关系 (a) (b) 学生关系 学号 (c) CSDN @快乐江湖										
参照完整性规则	参照完整性：若属性或属性组F是基本关系R的外码，它与基本S的主码相对应（关系R和S不一定是不同的关系），则对于R中每个元组在F上的值必须 - 要么取空值（此时F的每个属性值均为空值） - 要么等于S中某个元组的主码值										
用户自定义完整性	用户自定义完整性针对某一具体关系数据库的约束条件，反映某一具体应用所涉及的数据必须满足的语义要求，例如某个属性必须取唯一值，某个非主属性不能取空值等等										

第四节：关系代数(2.25)

2022年1月2日 10:15

关系代数的基本概念

关系代数：是一种抽象的查询语言，用于对**关系运算**来表达查询

关系代数的**运算对象**和**运算结果**都是**关系**，所用到的运算符可分为如下两种

- **传统的集合运算：**运算从行的角度进行
- **专门的关系运算：**运算同时涉及行和列

运算符		含义
集合运算符	$\cup$	并
	$-$	差
	$\cap$	交
	$\times$	笛卡儿积
专门的关系运算符	σ	选择
	π	投影
	$\bowtie$	连接
	$\div$	除

传统的集合运算

并 (union)

并：对于关系 R 和 S ，并操作就是将两个关系上下拼在一起形成一个新的关系，记为 $R \cup S$

- 所以两个关系具有**相同数目的属性**

R

A	B	C
3	8	1
5	4	2
7	3	6
9	7	4
2	3	5

S

A	B	C
1	5	7
4	6	8
2	3	5
5	4	2
3	4	6

$R \cup S$

A	B	C
3	8	1
5	4	2
7	3	6
9	7	4
2	3	5
1	5	7
4	6	8
2	3	5
5	4	2
3	4	6

CSDN @快乐江湖

差 (except)

差：对于关系 R 和 S ，求他们的差，就是在 R 中去掉两个关系中所有相同的行，形成一个新的关系，记为 $R - S$

R

A	B	C
3	8	1
5	4	2
7	3	6
9	7	4
2	3	5

S

A	B	C
1	5	7
9	7	4
2	7	5
5	4	2
3	4	6

$R - S$

A	B	C
3	8	1
7	3	6
2	3	5

CSDN @快乐江湖

交 (intersection)

交：对于关系 R 和 S ，求他们的交，就是选出两个关系中所有相同的行，形成一个新的关系，记为 $R \cap S$

- 所以 $R \cap S = R - (R - S)$

R			S			$R \cap S$		
A	B	C	A	B	C	A	B	C
3	8	1	1	5	7			
5	4	2	4	6	8			
7	3	6	2	3	5	2	3	5
9	7	4	5	4	2	5	4	2
2	3	5	3	4	6			

笛卡尔积 (cartesian product)

笛卡尔积：对于关系 R 和 S ，求他们的笛卡尔积，就是两个关系所有元组的不同排列组合，形成一个新的关系，记为 $R \times S$

R			S		$R \times S$				
A	B	C	D	E	A	B	C	D	E
1	2	3	a	b	1	2	3	a	b
4	5	6	a	b	1	2	3	c	d
7	8	9	c	d	4	5	6	a	b
					4	5	6	c	d
					7	8	9	a	b
					7	8	9	c	d

CSDN @快乐江湖

专门的关系运算

选择 (selection)

选择：从行的角度出发，在关系 R 中选择满足条件的元组然后组成新的关系。“满足条件”意味着有条件表达式，其运算符如下

投影
(projection)

选择：从列的角度出发，选择满足条件的若干属性列组成新的关系
6126 关系列属性选择符号
选择可以是属性名，列的序号，带数、运算符、真值谓词，逻辑运算符
6126 关系一列属性选择符号
• 投影之后取消了原关系中的某些列，当然也有可能取消某些元组（因为一旦取消了某些属性列后就极有可能出现重复行），所以一定注意消除完全相同的行

连接 (join)

等值连接和自然连接

等值连接

等值连接：可以按照如下步骤考虑，最终形成新的关系，记为

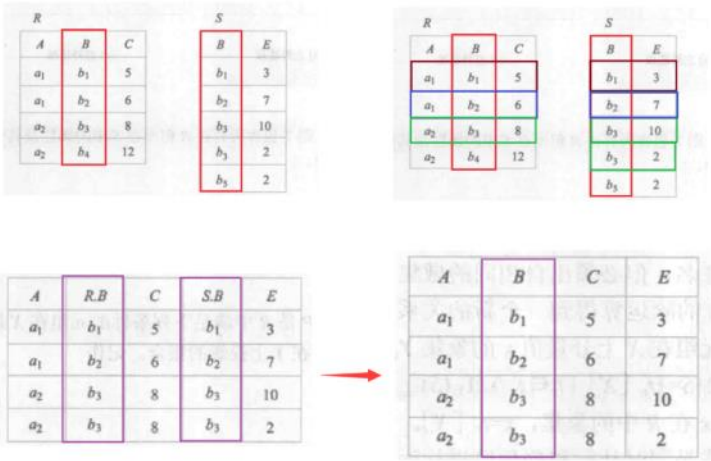
- 首先找到关系 R 和 S 中属性相同的列
- 然后找到两列中相同的元素
- 将相同元素所在的行组成新的一行，需要用【关系.属性】的格式区分两个关系中相同的属性

R			S		$R \bowtie S$				
A	B	C	B	E	A	R.B	C	S.B	E
a ₁	b ₁	5	b ₁	3	a ₁	b ₁	5	b ₁	3
a ₁	b ₂	6	b ₂	7	a ₁	b ₂	6	b ₂	7
a ₂	b ₃	8	b ₃	10	a ₂	b ₃	8	b ₃	10
a ₂	b ₄	12	b ₄	2	a ₂	b ₄	12	b ₄	2

CSDN @快乐江湖

自然连接

自然连接：它是一种特殊的等值连接，在等值连接的结果中去掉重复列即可



外连接

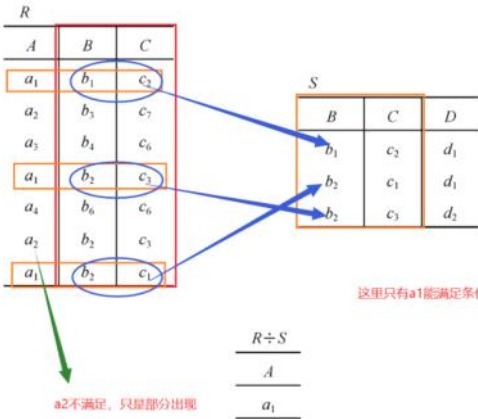
外连接：若将悬浮元组保留在自然连接的结果中，而在其他属性上填NULL，那么这种连接就叫做外连接，同时

- **左外连接：**只保留左边关系 R 中的悬浮元组
- **右外连接：**只保留右边关系 R 中的悬浮元组

除 (divide)

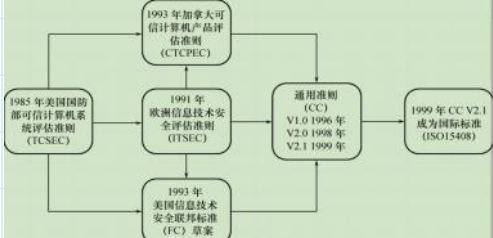
除：是笛卡尔积的逆运算，对于关系 R 和 S ，求 $R \div S$ 可按如下步骤考虑

- 研究对象是 R 和 S 中相同的属性列
- 在 R 中挑选元组，所挑选的元组一定满足它的属性均出现在 S 对应相同属性列的所有属性集合内



4.1数据库安全性概述

2022年1月26日 9:16

★ 数据库安全性	★ 数据库的安全性是指保护数据库以防止不合法使用所造成的数据泄露、更改或破坏。
数据库不安全因素	1. 非授权用户对数据库的恶意存取和破坏 手段：■ 一些黑客（Hacker）和犯罪分子在用户存取数据库时猎取用户名和用户口令，然后假冒合法用户偷取、修改甚至破坏用户数据。 防范：■ 数据库管理系统提供的安全措施主要包括用户身份鉴别、存取控制和视图等技术。 2. 数据库中重要或敏感的数据被泄露 手段：■ 黑客和敌对分子千方百计盗窃数据库中的重要数据，一些机密信息被暴露。 保护： ■ 数据库管理系统提供的主要技术有强制存取控制、数据加密存储和加密传输等。 ■ 审计日志分析 3. 安全环境的脆弱性
安全标准	 TCSEC标准 出现：1991年4月美国NCSC（国家计算机安全中心）颁布了《可信计算机系统评估标准关于可信数据库系统的解释》（Trusted Database Interpretation 简称TDI） ■ TDI又称紫皮书。它将TCSEC 扩展到数据库管理系统。 ■ TDI中定义了数据库管理系统的设计与实现中需满足和用以进行安全级别评估的标准TCSEC/TDI， 从四个方面来描述安全级别划分的指标 ●安全策略 ●责任 ●保证 ●文档 级别： 1. D级 ■ 将一切不符合更高标准的系统均归于D组 ■ 典型例子：DOS是安全标准为D的操作系统，DOS在安全性方面几乎没有什么专门的机制来保障。 2. C1级 ■ 非常初级的自主安全保护 ■ 能够实现对用户和数据的分离，进行自主存取控制（DAC），保护或限制用户权限的传播。 ■ 现有的商业系统稍作改进即可满足 3. C2级 ■ 安全产品的最低档次 ■ 提供受控的存取保护，将C1级的DAC进一步细化，以个人身份注册负责，并实施审计和资源隔离 ■ 达到C2级的产品在其名称中往往不突出“安全”（Security）这一特色 ■ 典型例子 ● Windows 2000 ● Oracle 7 4. B1级 ■ 标记安全保护。“安全”（Security）或“可信的”（Trusted）产品。真正意义的安全产品

- 对系统的数据加以标记，对标记的主体和客体实施强制存取控制（MAC）、审计等安全机制
- B1级典型例子
 - 操作系统
 - 惠普公司的HP-UX BLS release 9.09+
 - 数据库
 - Oracle公司的Trusted Oracle 7
 - Sybase公司的Secure SQL Server version 11.0.6

5. B2级 ■ 结构化保护

- 建立形式化的安全策略模型并对系统内的所有主体和客体实施DAC和MAC

6. B3级 ■ 安全域

- 该级的TCB必须满足访问监控器的要求，审计跟踪能力更强，并提供系统恢复过程

7. A1级 ■ 验证设计，即提供B3级保护的同时给出系统的形式化设计说明和验证以确信各安全保护真正实现。

CC

- 提出国际公认的表述信息技术安全性的结构
- 把信息产品的安全要求分为
 - 安全功能要求
 - 安全保证要求

CC文本组成

- 简介和一般模型
 - 有关术语、基本概念和一般模型以及与评估有关的一些框架
- 安全功能要求
 - 列出了一系列类、子类和组件
- 安全保证要求
 - 列出了一系列保证类、子类和组件
 - 提出了评估保证级（Evaluation Assurance Level, EAL），从EAL1至EAL7共分为七级

评估保证级	定 义	TCSEC安全级别 (近似相当)
EAL1	功能测试 (functionally tested)	
EAL2	结构测试 (structurally tested)	C1
EAL3	系统地测试和检查 (methodically tested and checked)	C2
EAL4	系统地设计、测试和复查 (methodically designed, tested, and reviewed)	B1
EAL5	半形式化设计和测试 (semiformally designed and tested)	B2
EAL6	半形式化验证的设计和测试 (semiformally verified design and tested)	B3
EAL7	形式化验证的设计和测试 (formally verified design and tested)	A1

4.2 数据安全性控制

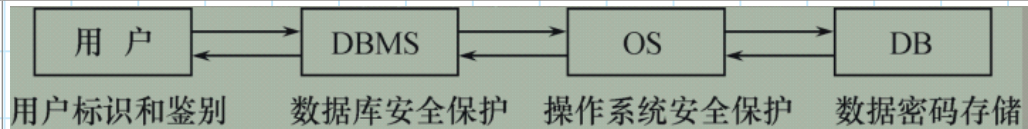
2022年1月26日

9:20

非法使用数据库的情况

1. 编写合法程序绕过数据库管理系统及其授权机制
2. 直接或编写应用程序执行非授权操作
3. 通过多次合法查询数据库从中推导出一些保密数据

计算机系统安全模型



数据安全性控制

★ 数据库安全性控制的常用方法

1. 用户标识和鉴定（用户身份鉴别）

- 系统提供的最外层安全保护措施

- 用户标识：由用户名和用户标识号组成（用户标识号在系统整个生命周期内唯一）

方法：

1. 静态口令鉴别

- 静态口令一般由用户自己设定，这些口令是静态不变的

2. 动态口令鉴别

- 口令是动态变化的，每次鉴别时均需使用动态产生的新口令登录数据库管理系统，即采用一次一密的方法

3. 生物特征鉴别

- 通过生物特征进行认证的技术，生物特征如指纹、虹膜和掌纹等

4. 智能卡鉴别

- 智能卡是一种不可复制的硬件，内置集成电路的芯片，具有硬件加密功能

2. 存取控制

a. 组成：

- 定义用户权限，并将用户权限登记到数据字典中

- 用户对某一数据对象的操作权力称为权限

- ★ ● DBMS提供适当的语言来定义用户权限，存放在数据字典中，称做安全规则或授权规则

- 合法权限检查

- 用户发出存取数据库操作请求

- DBMS查找数据字典，进行合法权限检查

a. 总结：用户权限定义和合法权检查机制一起组成了数据库管理系统的存取控制子系统

★ b. 常用存取控制方法：

- i. 自主存取控制（Discretionary Access Control，简称DAC）

- C2级

- 用户对不同的数据对象有不同的存取权限

- 不同的用户对同一对象也有不同的权限

- 用户还可将其拥有的存取权限转授给其他用户

ii. 强制存取控制 (Mandatory Access Control, 简称 MAC)

- B1级
- 每一个数据对象被标以一定的密级
- 每一个用户也被授予某一个级别的许可证
- 对于任意一个对象, 只有具有合法许可证的用户才可以存取

3. 视图

4. 审计

5. 数据加密

(3) 视图机制: 为不同的用户定义视图, 通过视图机制把要保密的数据对无权存取的用户隐藏起来, 从而自动地对数据提供一定程度的安全保护。

(4) 审计: 建立审计日志, 把用户对数据库的所有操作自动记录下来放入审计日志中, DBA 可以利用审计跟踪的信息, 重现导致数据库现有状况的一系列事件, 找出非法存取数据的人、时间和内容等。

(5) 数据加密: 对存储和传输的数据进行加密处理, 从而使得不知道解密算法的人无法获知数据的内容。

4.3 自动存取控制方法

2022年1月26日

10:36

自动存取控制方法

DAL

定义：用户可以自主的决定将数据的存储权限授予何人、决定是否将授权的权限授予别人。

通过 SQL 的 GRANT 语句和 REVOKE 语句实现

用户权限概念：定义用户可以在哪些数据库对象上进行哪些类型的操作。

❖ 用户权限组成

■ 数据对象

■ 操作类型

对象类型	对象	操作类型
数据库模式	模式	CREATE SCHEMA
	基本表	CREATE TABLE, ALTER TABLE
	视图	CREATE VIEW
	索引	CREATE INDEX
数据	基本表和视图	SELECT, INSERT, UPDATE, DELETE, REFERENCES, ALL PRIVILEGES
	属性列	SELECT, INSERT, UPDATE, REFERENCES, ALL PRIVILEGES

定义存取权限称为授权。

存取控制对象为数据本身和数据库模式

对象类型	对象	操作类型
数据库模式	模式	CREATE SCHEMA
	基本表	CREATE TABLE, ALTER TABLE
	视图	CREATE VIEW
	索引	CREATE INDEX
数据	基本表和视图	SELECT, INSERT, UPDATE, DELETE, REFERENCES, ALL PRIVILEGES
	属性列	SELECT, INSERT, UPDATE, REFERENCES, ALL PRIVILEGES

授权与回收

1. GRANT 授权

GRANT <权限> [, <权限>] ...

ON <对象类型> <对象名> [, <对象类型> <对象名>] ...

TO <用户> [, <用户>] ...

[WITH GRANT OPTION];

❖ 语义：将对指定操作对象的指定操作权限授予指定的用户

WITH GRANT OPTION 子句:

■ 指定：可以再授予

■ 没有指定：不能传播，不能给其他人不允许循环授权

举例：

[例4.1] 把查询Student表权限授给用户U1

GRANT SELECT ON TABLE Student TO U1;

[例4.2] 把对Student表和Course表的全部权限授予用户U2和U3

GRANT ALL PRIVILEGES
ON TABLE Student, Course
TO U2, U3

给所有用户即用PUBLIC

2. REVOKE

❖ 授予的权限可以由数据库管理员或其他授权者用 REVOKE 语句收回

❖ REVOKE 语句的一般格式为：

REVOKE <权限> [, <权限>] ...

ON <对象类型> <对象名> [, <对象类型> <对象名>] ...

FROM <用户> [, <用户>] ... [CASCADE | RESTRICT];

举例：

[例4.10] 把用户U5对SC表的INSERT权限收回

REVOKE INSERT ON TABLE SC FROM U5 CASCADE;

■ 将用户U5的INSERT权限收回的时候应该使用CASCADE（级联），否则拒绝执行该语句

■ 如果U6或U7还从其他用户处获得对SC表的INSERT权限，则他们仍具有此权限，系统只收回直接或间接从U5处获得的权限。

3. 创建数据库模式的权限

❖数据库管理员在创建用户时实现

❖CREATE USER语句格式

CREATE USER <username> [WITH][DBA|RESOURCE|CONNECT];

- 只有系统的超级用户才有权创建一个新的数据库用户
- 新创建的数据库用户有三种权限：CONNECT、RESOURCE和DBA

如没有指定创建的新用户的权限，默认该用户拥有CONNECT权限。拥有CONNECT权限的用户不能创建新用户，不能创建模式，也不能创建基本表，只能登录数据库。

拥有RESOURCE权限的用户能创建基本表和视图，成为所创建对象的属主。但不能创建模式，不能创建新的用户

拥有DBA权限的用户是系统中的超级用户，可以创建新的用户、创建模式、创建基本表和视图等；DBA拥有对所有数据库对象的存取权限，还可以把这些权限授予一般用户

拥有的权限	可否执行的操作			
	CREATE USER	CREATE SCHEMA	CREATE TABLE	登录数据库， 执行数据查询和 操纵
DBA	可以	可以	可以	可以
RESOURCE	不可以	不可以	不可以	不可以
CONNECT	不可以	不可以	不可以	可以，但必须拥有相应权限

4.4数据库角色

2022年1月26日

21:18

数据库角色

1. 数据库角色定义：被命名的一组与数据库操作相关的权限，角色是权限的集合

可以为一组具有相同权限的用户创建一个角色

简化授权的过程

2. 角色的创建

1. CREATE ROLE <角色名>

3. 给角色授权

GRANT <权限>[,<权限>]... ON <对象类型>对象名 TO <角色>[,<角色>]...

4. 将一个角色授予其他的角色或用户

GRANT <角色1>[,<角色2>]...TO <角色3>[,<用户1>]... WITH ADMIN OPTION]

5. 角色权限的收回

REVOKE <权限>[,<权限>] ON <对象类型> <对象名> FROM <角色>[,<角色>]...

■ 用户可以回收角色的权限，从而修改角色拥有的权限

■ REVOKE执行者是

●角色的创建者

●拥有在这个（些）角色上的ADMIN OPTION

6. [例4.11] 通过角色来实现将一组权限授予一个用户。

步骤如下：

(1) 首先创建一个角色 R1

CREATE ROLE R1;

(2) 然后使用GRANT语句，使角色R1拥有Student表的SELECT、UPDATE、INSERT权限

GRANT SELECT, UPDATE, INSERT ON TABLE Student TO R1;

(3) 将这个角色授予王平，张明，赵玲。使他们具有角色R1所包含的全部权限

GRANT R1 TO 王平,张明,赵玲;

(4) 可以一次性通过R1来回收王平的这3个权限

REVOKE R1 FROM 王平;

4.5强制存取控制方法

2022年1月28日

10:35

强制存取控制 (MAC)

保证更程度的安全性
用户不能直接感知或进行控制
适用于对数据有严格而固定密级分类的部门
军事部门
政府部门

数据库管理系统所管理的全部实体被分为主体和客体两大类

分类	主体	客体
定义	主体是系统中的活动实体	客体是系统中的被动实体，受主体操纵
作用	■ 数据库管理系统所管理的实际用户 ■ 代表用户的各进程	■ 文件、基本表、索引、视图

敏感度标记 (Label)

定义：对于主体和客体，DBMS为它们每个实例（值）指派一个敏感度标记（Label）

级别：

- 绝密 (Top Secret, TS) ●机密 (Secret, S) ●可信 (Confidential, C) ●公开 (Public, P)
- $TS \geq S \geq C \geq P$

主体的敏感度标记称为许可证级别 (Clearance Level)

客体的敏感度标记称为密级 (Classification Level)

强制存取控制规则

- (1) 仅当主体的许可证级别大于或等于客体的密级时，该主体才能读取相应的客体
- (2) 仅当主体的许可证级别小于或等于客体的密级时，该主体才能写相应的客体

说明（是自主控制之后的防线）

- ❖ 强制存取控制 (MAC) 是对数据本身进行密级标记，无论数据如何复制，标记与数据是一个不可分的整体，只有符合密级标记要求的用户才可以操纵数据。
- ❖ 实现强制存取控制时要首先实现自主存取控制
 - 原因：较高安全性级别提供的安全保护要包含较低级别的所有保护
- ❖ 自主存取控制与强制存取控制共同构成数据库管理系统的安全机制

总结 DAC + MAC安全 检查

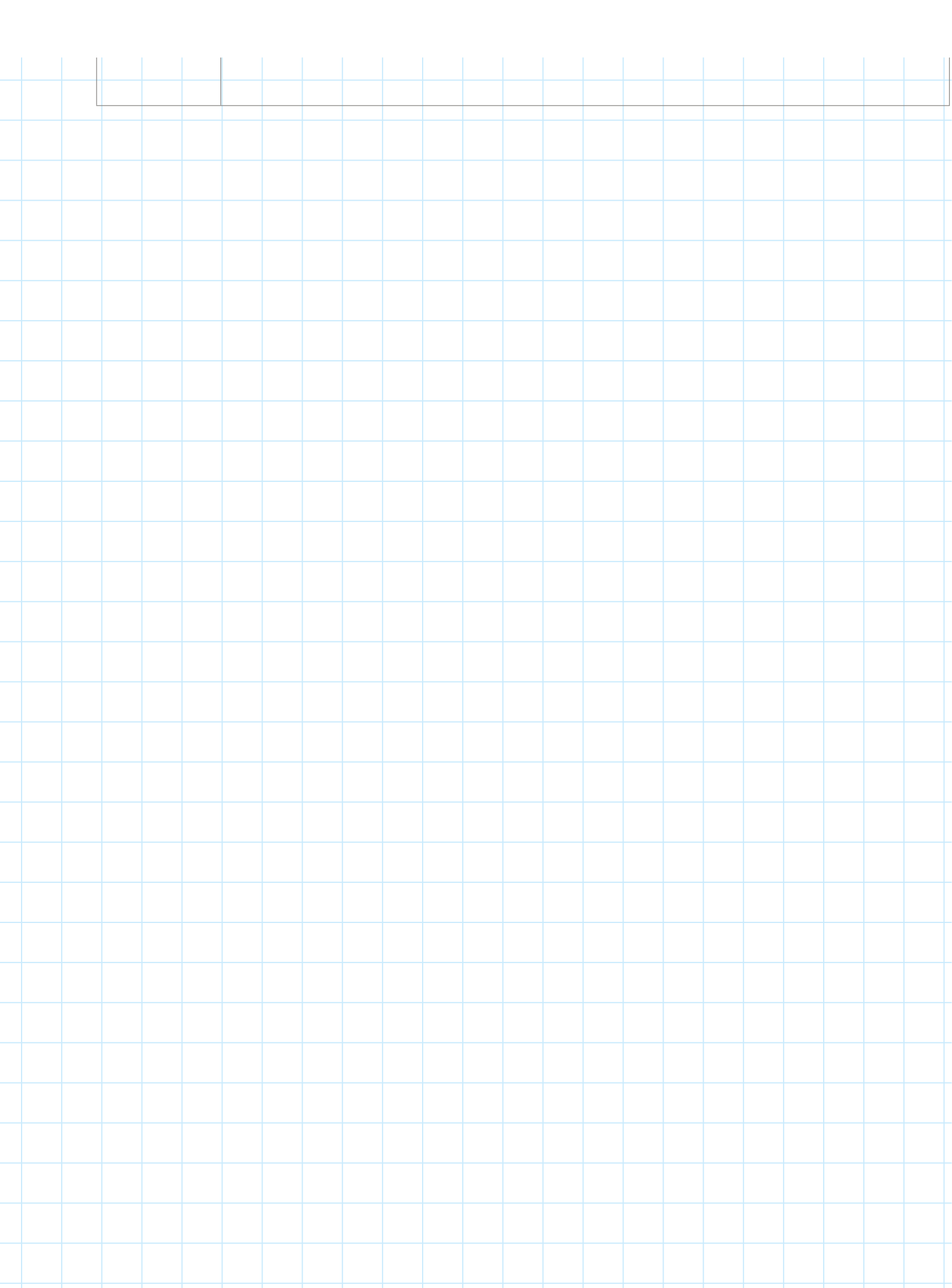


4.6视图和审计

2022年1月28日

10:54

视图作用	1. 把要保密的数据对无权存取这些数据的用户隐藏起来，对数据提供一定程度的安全保护 2. 间接地实现支持存取谓词的用户权限定义。					
审计（耗费系统资源）	1. 审计日志：启用一个专用的审计日志（Audit Log） 将用户对数据库的所有操作记录在上面，审计员利用审计日志监控数据库中的各种行为，找出非法存取数据的人、时间和内容。 ■ C2以上安全级别的DBMS必须具有审计功能					
审计功能的可选性	审计很费时间和空间 DBA可以根据应用对安全性的要求，灵活地打开或关闭审计功能 审计功能主要用于安全性要求较高的部门					
审计事件	服务器事件 审计数据库服务器发生的事件 系统权限 对系统拥有的结构或模式对象进行操作的审计 要求该操作的权限是通过系统权限获得的 语句事件 对SQL语句，如DDL、DML、DQL及DCL语句的审计 模式对象事件 对特定模式对象上进行的SELECT或DML操作的审计					
审计功能	基本功能 提供多种审计查阅方式 多套审计规则：一般在初始化设定 提供审计分析和报表功能 审计日志管理功能 防止审计员误删审计记录，审计日志必须先转储后删除 对转储的审计记录文件提供完整性和保密性保护 只允许审计员查阅和转储审计记录，不允许任何用户新增和修改审计记录等 提供查询审计设置及审计记录信息的专门视图					
审计分类	<table><tr><td>用户级审计</td><td>任何用户可设置的审计 主要是用户针对自己创建的数据库表和视图进行审计</td></tr><tr><td>系统级审计</td><td>只能由数据库管理员设置 监测成功或失败的登录要求、监测授权和收回操作以及其他数据库级权限下的操作</td></tr></table>		用户级审计	任何用户可设置的审计 主要是用户针对自己创建的数据库表和视图进行审计	系统级审计	只能由数据库管理员设置 监测成功或失败的登录要求、监测授权和收回操作以及其他数据库级权限下的操作
用户级审计	任何用户可设置的审计 主要是用户针对自己创建的数据库表和视图进行审计					
系统级审计	只能由数据库管理员设置 监测成功或失败的登录要求、监测授权和收回操作以及其他数据库级权限下的操作					
审计语句	AUDIT语句和NOAUDIT语句 ■ AUDIT语句：设置审计功能 ■ NOAUDIT语句：取消审计功能					
审计说明	【说明】 1. 审计设置和审计日志一般存储在数据字典中。 2. 必须打开审计开关，系统参数audit_trail设为true，才能在系统 看到审计息。 3. 数据库安全审计系统提供一种事后检查的安全机制。					



4.7数据加密

2022年1月28日

13:25

数据加密	防止数据库中数据在存储和传输中失密的有效手段										
思想	根据一定的算法将原始数据——明文（Plain text）变换为不可直接识别的格式——密文（Cipher text）,从而使得不知道解密算法的人无法获知数据的内容。										
加密方法	1. 存储加密 <table><tr><td>透明存储加密</td><td> ● 内核级加密保护方式，对用户完全透明 ● 将数据在写到磁盘时对数据进行加密，授权用户读取数据时再对其进行解密 ● 数据库的应用程序不需要做任何修改，只需在创建表语句中说明需加密的字段即可 </td></tr><tr><td>非透明存储加密</td><td> ● 通过多个加密函数实现。 </td></tr></table> 2. 传输加密 <table><tr><td>链路加密</td><td> ● 在链路层进行加密 ● 传输信息由报头和报文两部分组成 ● 报文和报头均加密 </td></tr><tr><td>端到端加密</td><td> ● 在发送端加密，接收端解密 ● 只加密报文不加密报头 ● 所需密码设备数量相对较少，容易被非法监听者发现并从中获取敏感信息 </td></tr></table>			透明存储加密	● 内核级加密保护方式，对用户完全透明 ● 将数据在写到磁盘时对数据进行加密，授权用户读取数据时再对其进行解密 ● 数据库的应用程序不需要做任何修改，只需在创建表语句中说明需加密的字段即可	非透明存储加密	● 通过多个加密函数实现。	链路加密	● 在链路层进行加密 ● 传输信息由报头和报文两部分组成 ● 报文和报头均加密	端到端加密	● 在发送端加密，接收端解密 ● 只加密报文不加密报头 ● 所需密码设备数量相对较少，容易被非法监听者发现并从中获取敏感信息
透明存储加密	● 内核级加密保护方式，对用户完全透明 ● 将数据在写到磁盘时对数据进行加密，授权用户读取数据时再对其进行解密 ● 数据库的应用程序不需要做任何修改，只需在创建表语句中说明需加密的字段即可										
非透明存储加密	● 通过多个加密函数实现。										
链路加密	● 在链路层进行加密 ● 传输信息由报头和报文两部分组成 ● 报文和报头均加密										
端到端加密	● 在发送端加密，接收端解密 ● 只加密报文不加密报头 ● 所需密码设备数量相对较少，容易被非法监听者发现并从中获取敏感信息										

5.1数据库完整性

2022年1月28日

15:14

数据库的完整性	数据的正确性 是指数据是符合现实世界语义，反映了当前实际状况的。 数据的相容性 是指数据库同一对象在不同关系表中的数据是符合逻辑的。
完整性与安全性区别	数据的完整性 防止数据库中存在不符合语义的数据，也就是防止数据库中存在不正确的数据。 防范对象：不合语义的、不正确的数据。 数据的安全性 保护数据库 防止恶意的破坏和非法的存取 防范对象：非法用户和非法操作
数据库管理系统为维护数据库的完整性	为维护数据库的完整性，提供如下功能： 1.提供定义完整性约束条件的机制 完整性约束条件也称为完整性规则，是数据库中的数据必须满足的语义约束条件 SQL标准使用了一系列概念来描述完整性，包括关系模型的实体完整性、参照完整性和用户定义完整性 这些完整性一般由SQL的数据定义语言语句来实现 2.提供完整性检查的方法 ● 数据库管理系统中检查数据是否满足完整性约束条件的机制称为完整性检查。 ● 一般在INSERT、UPDATE、DELETE语句执行后开始检查，也可以在事务提交时检查。 检查这些操作执行后数据库中数据是否违背了完整性约束条件。 3.违约处理 数据库管理系统若发现用户的操作违背了完整性约束条件，就采取一定的动作,如拒绝（NO ACTION）执行该操作、级连（CASCADE）执行其他操作，进行违约处理以保证数据的完整性。 关系数据库管理系统使得完整性控制成为其核心支持的功能，从而能够为所有用户和应用提供一致的数据库完整性。

5.2 实体完整性

2022年1月28日

15:25

实体完整性	★ 目的：设置主码，使得每一行都可以区分。 <i>实体完整性这项规则要求每个数据表都必须有主键，而为主键的所有字段，其属性必须是独一无二非空值</i> ❖ 单属性构成的码有两种说明方法 ■ 定义为列级约束条件 在建立列的时候设置是否为主码 ■ 定义为表级约束条件 建完表之后在设置主码 ❖ 对多个属性构成的码只有一种说明方法 ■ 定义为表级约束条件（只能建完表之后） 举例： (1) 在列级定义主码 <pre>CREATE TABLE Student (Sno CHAR(9) PRIMARY KEY, Sname CHAR(20) NOT NULL, Ssex CHAR(2), Sage SMALLINT, Sdept CHAR(20));</pre> (2) 在表级定义主码 <pre>CREATE TABLE Student (Sno CHAR(9), Sname CHAR(20) NOT NULL, Ssex CHAR(2), Sage SMALLINT, Sdept CHAR(20), PRIMARY KEY (Sno) //多个属性组成主码 PRIMARY KEY (Sno,Cno));</pre> <i>多个属性组成的</i>						
实体完整性检查内容	★ 插入或对主码列进行更新操作时，关系数据库管理系统按照实体完整性规则自动进行检查。包括： ■ 检查主码值是否唯一，如果不唯一则拒绝插入或修改。 ■ 检查主码的各个属性是否为空，只要有一个为空就拒绝插入或修改，从而保证实体完整性。 检查主码的方法 <table><tr><td>方法</td><td>检查记录中主码值是否唯一的一种方法是进行全表扫描。</td></tr><tr><td>步骤</td><td>依次判断表中每一条记录的主码值与将插入记录上的主码值（或者修改的新主码值）是否相同。</td></tr><tr><td>缺点</td><td>十分耗时</td></tr></table>	方法	检查记录中主码值是否唯一的一种方法是进行全表扫描。	步骤	依次判断表中每一条记录的主码值与将插入记录上的主码值（或者修改的新主码值）是否相同。	缺点	十分耗时
方法	检查记录中主码值是否唯一的一种方法是进行全表扫描。						
步骤	依次判断表中每一条记录的主码值与将插入记录上的主码值（或者修改的新主码值）是否相同。						
缺点	十分耗时						

缺点	十分耗时
优化	为避免对基本表进行全表扫描，RDBMS核心一般都在主码上自动建立一个索引

5.3参照完整性和用户定义完整性

2022年1月28日 17:58

写在表 Foreign KEY (Sno) REFERENCES Student (Sno)

参照完整性定义	■ 在CREATE TABLE中用FOREIGN KEY短语定义哪些列为外码，用REFERENCES短语指明这些外码参照哪些表的主码。 写在列后面用外码 references 下带外码 foreign key															
参照完整性检查	一个参照完整性将两个表中的相应元组联系起来，对被参照表和参照表进行增删改操作时有可能破坏参照完整性，必须进行检查以保证这两个表的相容性。 写在表级则还可以指定其它列															
参照完整性违约处理	(1) 拒绝 (NO ACTION) 执行 ●不允许该操作执行。该策略一般设置为默认策略。 ON DELETE CASCADE 级联删除 (2) 级联 (CASCADE) 操作 ●当删除或修改被参照表 (Student) 的一个元组造成了与参照表 (SC) 的不一致，则删除或修改参照表中的所有造成不一致的元组。 (3) 设置为空值 (SET-NULL) ●当删除或修改被参照表的一个元组时造成了不一致，则将参照表中的所有造成不一致的元组的对应属性设置为空值。 ON DELETE SET NULL 级联为空 <table><tr><th>被参照表 (例如Student)</th><th>参照表 (例如SC)</th><th>违约处理</th></tr><tr><td>可能破坏参照完整性</td><td>插入元组</td><td>拒绝</td></tr><tr><td>可能破坏参照完整性</td><td>修改外码值</td><td>拒绝</td></tr><tr><td>删除元组</td><td>可能破坏参照完整性</td><td>拒绝/级连删除/设置为空值</td></tr><tr><td>修改主码值</td><td>可能破坏参照完整性</td><td>拒绝/级连修改/设置为空值</td></tr></table>	被参照表 (例如Student)	参照表 (例如SC)	违约处理	可能破坏参照完整性	插入元组	拒绝	可能破坏参照完整性	修改外码值	拒绝	删除元组	可能破坏参照完整性	拒绝/级连删除/设置为空值	修改主码值	可能破坏参照完整性	拒绝/级连修改/设置为空值
被参照表 (例如Student)	参照表 (例如SC)	违约处理														
可能破坏参照完整性	插入元组	拒绝														
可能破坏参照完整性	修改外码值	拒绝														
删除元组	可能破坏参照完整性	拒绝/级连删除/设置为空值														
修改主码值	可能破坏参照完整性	拒绝/级连修改/设置为空值														
★ 用户定义完整性	针对某一具体应用的数据必须满足的语义要求，关系数据库管理系统提供了定义和检验用户定义完整性的机制，不必由应用程序承担。 约束条件的定义 <table><tr><th>属性上的约束条件</th><th>元组上约束条件</th></tr><tr><td>CREATE TABLE时定义属性上的约束条件 列值非空 (NOT NULL) 列值唯一 (UNIQUE) 检查列值是否满足一个条件表达式 (CHECK)</td><td>在CREATE TABLE时可以用CHECK短语定义元组上的约束条件，即元组级的限制同属性值限制相比，元组级的限制可以设置不同属性之间的取值的相互约束条件</td></tr></table>	属性上的约束条件	元组上约束条件	CREATE TABLE时定义属性上的约束条件 列值非空 (NOT NULL) 列值唯一 (UNIQUE) 检查列值是否满足一个条件表达式 (CHECK)	在CREATE TABLE时可以用CHECK短语定义元组上的约束条件，即元组级的限制同属性值限制相比，元组级的限制可以设置不同属性之间的取值的相互约束条件											
属性上的约束条件	元组上约束条件															
CREATE TABLE时定义属性上的约束条件 列值非空 (NOT NULL) 列值唯一 (UNIQUE) 检查列值是否满足一个条件表达式 (CHECK)	在CREATE TABLE时可以用CHECK短语定义元组上的约束条件，即元组级的限制同属性值限制相比，元组级的限制可以设置不同属性之间的取值的相互约束条件															
完整性约束命名子句	CONSTRAINT <完整性约束条件名> <完整性约束条件> ■ <完整性约束条件>包括NOT NULL、UNIQUE、PRIMARY KEY短语、FOREIGN KEY短语、CHECK短语等															
修改表中的完整性限制	使用ALTER TABLE语句修改表中的完整性限制 [例5.12]去掉例5.10 Student表中对性别的限制。 ALTER TABLE Student DROP CONSTRAINT C4; ■ 可以先删除原来的约束条件，再增加新的约束条件 ALTER TABLE Student DROP CONSTRAINT C1; ALTER TABLE Student ADD CONSTRAINT C1 CHECK (Sno BETWEEN 900000 AND 999999),															

5.4域、断言与触发器

2022年1月29日 8:58

域	域是一组具有相同数据类型的值的集合，即属性的取值范围。SQL可以用CREATE DOMAIN语句建立一个域以及域应该满足的完整性约束条件，然后用域定义属性。 创建一个性别域GenderDomain，约束条件名为GD <pre>CREATE DOMAIN GenderDomain_CHAR(2) CONSTRAINT GD CHECK(VALUE IN('男', '女'));</pre> 删除域GenderDomain的限制条件GD:ALTER DOMAIN GenderDomain <pre>DROP CONSTRAINT GD;</pre> ——增加在这里用 add 优点：数据库中不同的属性可以来自同一个域，当域上的完整性约束条件改变时只需要修改域的定义即可。
断言	创建断言的语句格式 ■ CREATE ASSERTION <断言名> <CHECK 子句> ■ 每个断言都被赋予一个名字，<CHECK 子句>中的约束条件与WHERE子句的条件表达式类似。 任何使断言不为真值的操作者都会被拒绝执行 [例5.18] 限制数据库课程最多60名学生选修 <pre>CREATE ASSERTION ASSE_SC_DB_NUM CHECK (60 >= (select count(*) /*此断言的谓词涉及聚集操作count的SQL语句*/ From Course,SC Where SC.Cno=Course.Cno and Course.Cname = '数据库'));</pre> [例5.20]限制每个学期每一门课程最多60名学生选修 首先需要修改SC表的模式，增加一个“学期（TERM）”属性ALTER TABLE SC ADD TERM DATE; 然后，定义断言：CREATE ASSERTION ASSE_SC_CNUM2 CHECK(60 >= ALL (SELECT count(*) FROM SC GROUP by cno,TERM)); 删除断言的语句格式为 ■ DROP ASSERTION <断言名>; ■ 如果断言很复杂，则系统在检测和维护断言的开销较高，这是在使用断言时应该注意的
触发器	❖触发器（Trigger）是用户定义在关系表上的一类由事件驱动的特殊过程 ■ 触发器保存在数据库服务器中 ■ 任何用户对表的增、删、改操作均由服务器自动激活相应的触发器 ■ 触发器可以实施更为复杂的检查和操作，具有更精细和更强大的数据控制能力 （类似于内事，但此约束更严格） 定义触发器 <pre>CREATE TRIGGER语法格式 CREATE TRIGGER <触发器名> {BEFORE AFTER} <触发事件> ON <表名> REFERENCING NEW OLD ROW AS<变量> FOR EACH {ROW STATEMENT} [WHEN <触发条件>]<触发动作体></pre> 每当触发事件发生时，该触发器被激活 指明触发器激活的时间是在执行触发事件前或后 当触发条件为真时触发触发动作 触发器又叫做事件-条件-动作（event-condition-action）规则。当特定的系统事件发生时，对规则的条件进行检查，如果条件成立则执行规则中的动作，否则不执行该动作。规则中的动作体可以很复杂，通常是一段SQL存储过程 定义触发器的语法说明 （1）表的拥有者才可以在表上创建触发器，表创建触发器的数量是有限的。 （2）触发器名 ●触发器名可以包含模式名，也可以不包含模式名 ●同一模式下，触发器名必须是唯一的，并且触发器名和表名必须在同一模式下 （3）表名 ●触发器只能定义在基本表上，不能定义在视图上 ●当基本表的数据发生变化时，将激活定义在该表上相应触发事件的触发器，该表称为触发器的目标表

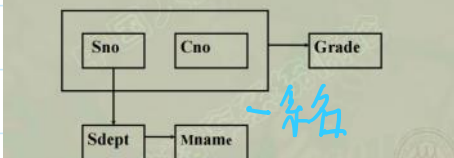
```
Create table s(sno char(5) not null unique,
Sname char(30) not null,
Status char(8),
City char(20),
Primary key(sno)
);
Create p( pno char(6)
Pname char(30) not null,
Color char(8) ,
Weight numeric(6,2),
City char(20),
Primary key (pno));
Create sp(
Sno char(5),
Pno char(6),
Status char(8),
Foreign key(sno) references s(sno)
Foreign key(pno) references p(pno)
);
```

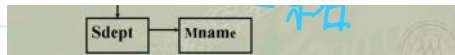
这里可以不用，因为主键自带防止重复。

★	(4) 触发事件
★	● 触发事件可以是INSERT、DELETE或UPDATE，也可以是这几个事件的组合
★	● 还可以UPDATE OF <触发列，...>，即进一步指明修改哪些列时激活触发器
	● AFTER/BEFORE是触发的时机
	➢ AFTER表示在触发事件的操作执行之后激活触发器
	➢ BEFORE表示在触发事件的操作执行之前激活触发器
	(5) 触发器类型
	➢ 行级触发器 (FOR EACH ROW)
	➢ 语句级触发器 (FOR EACH STATEMENT)
	例如,在例5.11的TEACHER表上创建一个AFTER UPDATE触发器, 触发事件是UPDATE语句: UPDATE TEACHER SET Deptno=5;假设表TEACHER有1000行
	● 如果是语句级触发器, 那么执行完该语句后, 触发动作只发生一次
	● 如果是行级触发器, 触发动作将执行1000次
	(6) 触发条件
	● 触发器被激活时, 只有当触发条件为真时触发动作体才执行; 否则触发动作体不执行。
	● 如果省略WHEN触发条件, 则触发动作体在触发器激活后立即执行
	(7) 触发动作体
激活触发器	✧ 触发器的执行, 是由触发事件激活的, 并由数据库服务器自动执行 ✧ 一个数据表上可能定义了多个触发器, 遵循如下的执行顺序: (1) 执行该表上的BEFORE触发器; (2) 激活触发器的SQL语句; (3) 执行该表上的AFTER触发器
删除触发器	DROP TRIGGER <触发器名> ON <表名>; ✧ 触发器必须是一个已经创建的触发器, 并且只能由具有相应权限的用户删除。

6.1 关系模型的表示

2022年2月7日 8:31

关系模式组成	由五部分组成，是一个五元组： $R(U, D, DOM, F)$ 关系名 R 是符号化的元组语义 U 为一组属性 D 为属性组 U 中的属性所来自的域 DOM 为属性到域的映射 F 为属性组 U 上的一组数据依赖 D、DOM 在表需要放数据时才考虑，在设计表时不用考虑。因此在本章中把关系模式看作一个三元组：$R<U, F>$。 与模式设计关系不大 当且仅当 U 上的一个关系 r 满足 F 时，r 称为关系模式 $R<U, F>$ 的一个关系 r 理解为实例，即是一个表，R 是模型，做表的模具。	
第一范式	★ 作为二维表，关系要符合一个最基本的条件：每个分量必须是不可分开的数据项。满足了这个条件的关系模式就属于第一范式（1NF）。所有关系表都要满足第一范式。	
数据依赖	★ 定义：是一个关系内部属性与属性之间的一种约束关系，是通过属性间值的相等与否体现出来的数据间相互联系。	
	分类：	
	函数依赖（Functional Dependency, 简记为FD） <i>某个属性决定另一个属性集时，称另一属性集依赖于该属性集。</i>	多值依赖（Multi-Valued Dependency, 简记为MVD）
	描述一个学生关系，可以有学号、姓名、系名等属性。 一个学号只对应一个学生，一个学生只在一个系中学习，“学号”值确定后，学生的姓名及所在系的值就被唯一确定。 ★ $Sname = f(Sno), Sdept = f(Sno)$ 。即 Sno 函数决定 $Sname$ ， Sno 函数决定 $Sdept$ 。记作 $Sno \rightarrow Sname$ ， $Sno \rightarrow Sdept$ 。也可以说姓名依赖于学号，系依赖于学号。	多值依赖（一对多） $sdept = f(sno)$ $sno \rightarrow sdept$ $sdept$ 就是 x
	● 一个系有若干学生，但一个学生只属于一个系； ● 一个系只有一名（正职）负责人； ● 一个学生可以选修多门课程，每门课程有若干学生选修； ● 每个学生学习每一门课程有一个成绩。	$\xrightarrow{+*}$ $sno + cno \rightarrow grade$ $\downarrow$ $sdept \rightarrow 姓名$ $sdept$ $sno + cno$ $name$
	■ 由此可得到属性组 U 上的一组函数依赖 F ： $F = \{Sno \rightarrow Sdept, Sdept \rightarrow Mname, (Sno, Cno) \rightarrow Grade\}$	
		



出现的问题:

(1) 数据冗余

- 浪费大量的存储空间
- 每一个系主任的姓名重复出现, 重复次数与该系所有学生的所有课程成绩出现次数相同。

(2) 更新异常 (Update Anomalies)

- 数据冗余, 更新数据时, 维护数据完整性代价大。
- 某系更换系主任后, 必须修改与该系学生有关的每一个元组。

(3) 插入异常 (Insertion Anomalies)

- 如果一个系刚成立, 尚无学生, 则无法把这个系及其系主任的信息存入数据库。

(4) 删除异常 (Deletion Anomalies)

- 如果某个系的学生全部毕业了, 则在删除该系学生信息的同时, 把这个系及其系主任的信息也丢掉了。

原因

- 由存在于模式中的某些数据依赖引起的。

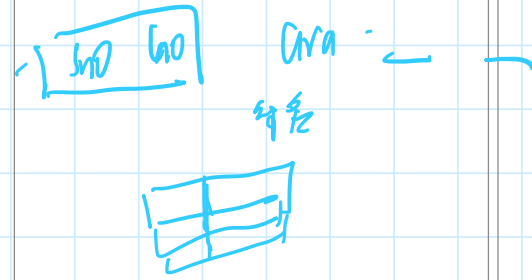
解决方法

- 用规范化理论改造关系模式来消除其中不合适的数据依赖

❖把这个单一的模式分成三个关系模式: 即三个表

- $S(Sno, Sdept, Sno \rightarrow Sdept)$;
- $SC(Sno, Cno, Grade, (Sno, Cno) \rightarrow Grade)$;
- $DEPT(Sdept, Mname, Sdept \rightarrow Mname)$;

❖这三个模式都不会发生插入异常、删除异常的问题, 数据的冗余也得到了控制。



6.2 函数依赖

2022年2月7日 9:05

函数依赖

给定一个X, 总
能找到一个
Y)
有重复值的
依赖于没有
重复值的

设 $R(U)$ 是一个属性集 U 上的关系模式, X 和 Y 是 U 的子集。若对于 $R(U)$ 的任意一个可能的关系 r , r 中不可能存在两个元组在 X 上的属性值相等, 而在 Y 上的属性值不等, 则称“ X 函数确定 Y ”或“ Y 函数依赖于 X ”, 记作 $X \rightarrow Y$ 。(简单的说, 通过学号值能确定唯一一个学生)。

X 相等 $\Rightarrow Y$ 相等

分类:

★ 非平凡的函数依赖	平凡的函数依赖
$X \rightarrow Y$, 但 $Y \not\subseteq X$ 则称 $X \rightarrow Y$ 是非平凡的函数依赖	$X \rightarrow Y$, 但 $Y \subseteq X$ 则称 $X \rightarrow Y$ 是平凡的函数依赖。
$(sno, Cno) \rightarrow (Grade)$	学号 $\rightarrow$ 学号
$x \rightarrow y, y$ 不在 x 的范围里。	对于任一关系模式, 平凡函数依赖都成立, 它反映新的语义

对于任一关系模式, 平凡函数依赖都是必然成立的, 它不反映新的语义。若不特别声明, 我们总是讨论非平凡函数依赖。

- ★ 若 $X \rightarrow Y$, 则 X 称为这个函数依赖的决定因素 (Determinant), X 称为这个函数依赖的属性组。
- 若 $X \rightarrow Y, Y \rightarrow X$, 则记作 $X \leftrightarrow Y$ 。若 Y 不函数依赖于 X , 则记作 $X \nrightarrow Y$ 。

完全函数依赖

在 $R(U)$ 中, 如果 $X \rightarrow Y$, 并且对于 X 的任何一个真子集 X' , 都有 $X' \nrightarrow Y$, 则称 Y 对 X 完全函数依赖,

记作 $X \xrightarrow{F} Y$ 。

一个X值对应一个Y值

部分函数依赖

若 $X \rightarrow Y$, 但 Y 不完全函数依赖于 X , 则称 Y 对 X 部分函数依赖,

记作 $X \xrightarrow{P} Y$

(学号课程) $\rightarrow$ 成绩, 但是学号自己就能推出成绩, 所以是部分依赖

传递函数依赖

在 $R(U)$ 中, 如果 $X \rightarrow Y (Y \not\subseteq X), Y \nrightarrow X, Y \rightarrow Z, Z \not\subseteq Y$, 则称 Z 对 X 传递函数依赖(transitive functional dependency)。

记为: $X \xrightarrow{传递} Z$ 。

例: $sno \rightarrow sdept, sdept \rightarrow mname$ 成立, 所以 $sno \xrightarrow{传递} mname$

- 注: 如果 $Y \rightarrow X$, 即 $X \leftrightarrow Y$, 则 Z 直接依赖于 X , 而不是传递函数依赖。

码

设 K 为 $R \langle U, F \rangle$ 中的属性或属性组合。若 $K \xrightarrow{F} U$, 则 K 称为 R 的一个候选码 (Candidate Key)。(即可以唯一确定一条记录)。

这组且完全依赖。

若关系模式 R 有多个候选码, 则选定其中的一个做为**主码**(Primary key)。

- 如果 U 部分函数依赖于 K , 即 $K \xrightarrow{P} U$, 则 K 称为**超码** (Surpkey)。候选码是最小的超码, 即 K 的任意一个真子集都不是候选码。(即含有候选码的集合)

超码为虽然肯定能唯一确定记录, 但是超码的字段可能有多余的

★ 主属性与非主属性

- 包含在任何一个候选码中的属性, 称为**主属性** (Prime attribute)

★主属性与非主属性

- 包含在任何一个候选码中的属性，称为**主属性** (Prime attribute)
- 不包含在任何码中的属性称为非主属性 (Nonprime attribute) 或非码属性 (Non-key attribute)

❖ **全码**：整个属性组是码，称为全码 (All-key) 即全部属性才能确定一条记录。

关系模式 R 中属性或属性组 X 并非 R 的码，但 X 是另一个关系模式的码，则称 X 是 R 的外部码 (Foreign key) 也称**外码**。

主码与外部码一起提供了表示关系间联系的手段。

主码和候选码都简称为码

6.3 范式

2022年2月7日

10:31

范式

范式是符合某一种级别的关系模式的集合。

关系数据库中的关系必须满足一定的要求。满足不同程度要求的为不同范式。

范式的种类：

1. 第一范式(1NF)

2. 第二范式(2NF)

3. 第三范式(3NF)

4. BC范式(BCNF)

5. 第四范式(4NF)

6. 第五范式(5NF)

关系中的属性不可再分，每列不能再分割

非主属性完全依赖于主属性

！无传递依赖

！左边是码

一个低一级范式的关系模式，通过模式分解 (schema decomposition) 可以转换为若

干个高一级范式的关系模式的集合，这种过程就叫规范化。通常按属性间依赖情况来区分规范化程度。

第二范式

若关系模式 $R \in 1NF$ ，并且每一个非主属性都完全函数依赖于任何一个候选码，则 $R \in 2NF$ 。

(表中不能有
部分依赖)

[例6.4] S-L-C(Sno,Sdept,Sloc,Cno,Grade), Sloc为学生的住处，并且每个系的学生住在同一个地方。S-L-C的码为(Sno,Cno)。

函数依赖有

■ $(Sno,Cno) \xrightarrow{F} Grade$

■ $Sno \rightarrow Sdept, (Sno,Cno) \xrightarrow{P} Sdept$

■ $Sno \rightarrow Sloc, (Sno,Cno) \xrightarrow{P} Sloc$

■ $Sdept \rightarrow Sloc$

(完全)

部分

学号、课号 $\rightarrow$ 成绩

学号 $\rightarrow$ 系名 学号、课号 $\rightarrow$ 系名

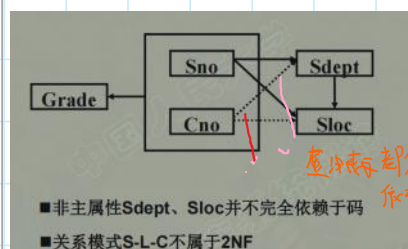
学号、课号 $\rightarrow$ 姓名

学号 $\rightarrow$ 住处

系名 $\rightarrow$ 住处

解决方法把
部分依赖单独
拆成另外一个表。

(第二范式
发生的条件
就是因为
主属性有多
个字段)



部分依赖: $(Sno, Cno) \xrightarrow{P} Sdept$

$(Sno, Cno) \xrightarrow{P} Sloc$

出现了非主属性没有完全
依赖于候选码

(让每个非主属性的依赖对候选码从部份依赖变为完全依赖)

❖ 一个关系模式不属于2NF，会产生以下问题：

■ 插入异常

● 如果插入一个新学生，但该生未选课，即该生无Cno，由于插入元组时，必须给定码值，因此插入失败。

■ 删除异常

● 如果S4只选了一门课C3，现在他不再选这门课，则删除C3后，整个元组的其他信息也被删除了。

■ 修改复杂

● 如果一个学生选的多门课，则Sdept, Sloc被存储了多次。如果该生转系，则需要修改所有相关的Sdept和Sloc，造成修改的复杂化。

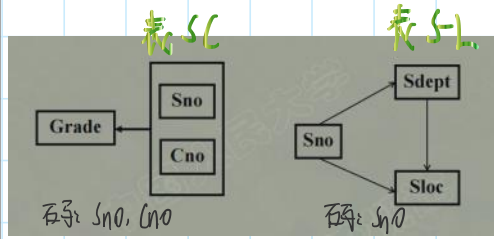
解决方法：

SC和S-L, 造成修改的冗余性。

解决方法:

- 用投影分解把关系模式S-L-C分解成两个关系模式
 - SC(Sno,Cno,Grade)
 - S-L(Sno,Sdept,Sloc)

能用于确定就别用两个



第三范式

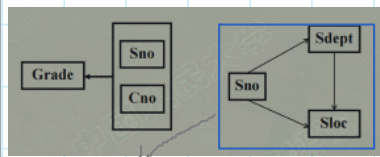
设关系模式 $R<U,F>\in 1NF$,若 R 中不存在这样的码 X 、属性组 Y 及非主属性 Z ($Z\notin Y$),使得 $X\rightarrow Y$, $Y\rightarrow Z$ 成立, $Y\rightarrow X$ 不成立,则称 $R<U,F>\in 3NF$ 。(即要满足第一范式,且不能满足传递函数依赖。)

不成立,则了不成立

若 $R\in 3NF$,则每一个非主属性即不传递

依赖于码,也符合部分依赖

所有子段只能依赖于主键



不符合第三范式。

- SC没有传递依赖,因此 $SC\in 3NF$

- S-L中 $Sno\rightarrow Sdept$ ($Sdept\rightarrow Sno$), $Sdept\rightarrow Sloc$, 可得 $Sno\rightarrow Sloc$ 。

- 解决的办法是将S-L分解成

- S-D(Sno,Sdept) $\in 3NF$
- D-L(Sdept,Sloc) $\in 3NF$

3NF的“不彻底”性表现在可能存在主属性对码的部分依赖和传递依赖。

BCNF

由Boyce和Codd提出,比3NF更进了一步,通常认为BCNF是修正的第三范式,有时也称为扩充的第三范式。

设关系模式 $R<U,F>\in 1NF$,若 $X\rightarrow Y$ 且 $Y\subseteq X$ 时 X 必是码(什么码都行,y是什么都行),则 $R<U,F>\in BCNF$ 。换言之,在关系模式 $R<U,F>$ 中,如果每一个决定属性集都包含候选码,则 $R\in BCNF$ 。(即 x 的属性为码)

主码 $\Rightarrow$ 任意属性

❖BCNF的关系模式所具有的性质

- 所有非主属性(不是码)都完全函数依赖于每个候选码
- 所有主属性都完全函数依赖于每个不包含它的候选码 $Sno\rightarrow Sname$
- 没有任何属性完全函数依赖于非码的任何一组属性

❖如果一个关系数据库中的所有关系模式都属于BCNF,那么在函数依赖范畴内,它已实现了模式的彻底分解,达到了最高的规范化程度,消除了插入异常和删除异常

关系模式 $S(Sno,Sname,Sdept,Sage)$,假定 $Sname$ 也具有唯一性,那么 S 就有两个码,这两个码都由单个属性组成,彼此不相交。

- 其他属性不存在对码的传递依赖与部分依赖,所以 $S\in 3NF$ 。
- 同时 S 中除 Sno , $Sname$ 外没有其他决定因素,所以 S 也属于BCNF

$Sno\rightarrow Sname$ $Sname\rightarrow Sno$ 左边位置都是码

$Sno\rightarrow Sage$ $Sno\rightarrow Sdept$ $Sname\rightarrow Sage$ $Sname\rightarrow Sdept$

多值依赖(一对

设 $R(U)$ (关系模型)是属性集 U 上的一个关系模式。 X,Y,Z (三组属性)是 U 的子集,并且 $Z=U-X-Y$ 。关系

多)

模式R(U)中多值依赖 (用两个箭头表示) $X \twoheadrightarrow Y$ 成立, 当且仅当对R(U)的任一关系r, 给定的一对(x,z)值, 有一组Y的值, 这组值仅仅决定于x值而与z值无关。

(有一张表, 属性为x,y,z, z可以由x和y表示出来, 但是z给了x后就能表示出来, 如下图, $(x,z) \rightarrow y$, 但是这组值仅仅决定于x值而与z值无关)

设学校中某一门课程由多个教师讲授, 他们使用相同的一套参考书。每个教员可以讲授多门课程, 每种参考书可以供多门课程使用用关系模式Teaching(C,T,B)来表示课程C、教师T和参考书B之间的关系

课程 C	教员 T	参考书 B
物理	李勇 王军	普通物理学 光学原理 物理习题集
数学	李勇 张平	数学分析 微分方程 高等代数
计算数学	张平 周峰	数学分析 ...

课程 C	教员 T	参考书 B
物理	李勇	普通物理学
物理	李勇	光学原理
物理	李勇	物理习题集
物理	王军	普通物理学
物理	王军	光学原理
物理	王军	物理习题集
数学	李勇	普通物理学
数学	李勇	光学原理
数学	李勇	物理习题集
数学	张平	普通物理学

别管参考书B如何取值,

Teaching (C, T, B) 对于C的每一个值, T有一组值与之对应, 而不论B取何值。因此T多值依赖于C, 即 $C \twoheadrightarrow T$

多值依赖的另一个等价的定义在R(U)的任一关系r中, 如果存在元组t, s使得 $t[X]=s[X]$ (两行都是物理), 那么就必然存在元组w, v ∈ r, (w, v可以与s, t相同), 使得 $w[X]=v[X]=t[X]$, 而 $w[Y]=t[Y]$, $w[Z]=s[Z]$, $v[Y]=s[Y]$, $v[Z]=t[Z]$ (即交换s, t元组的Y值所得的两个新元组必在r中则Y多值依赖于X, 记为 $X \twoheadrightarrow Y$ 。这里X, Y是U的子集, $Z=U-X-Y$ 。(交换彼此的y, 还是在原先表里的元素)

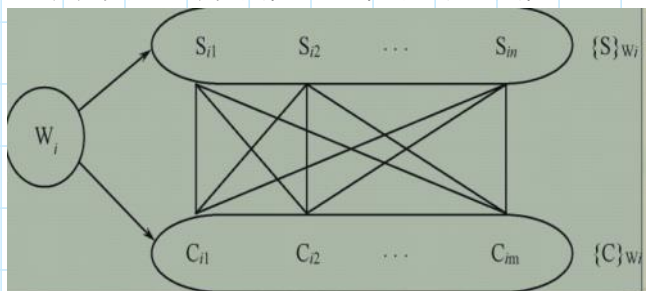
平凡多值依赖和非平凡的多值依赖

■ 若 $X \twoheadrightarrow Y$, 而 $Z = \Phi$, 即Z为空, 则称 $X \twoheadrightarrow Y$ 为平凡的多值依赖。否则称 $X \twoheadrightarrow Y$ 为非平凡的多值依赖

关系模式WSC(W,S,C)中, W表示仓库, S表示保管员, C表示商品。假设每个仓库有若干个保管员, 有若干种商品。每个保管员保管所在仓库的所有商品, 每种商品被所有保管员保管

W	S	C
W1	S1	C1
W1	S1	C2
W1	S1	C3
W1	S2	C1
W1	S2	C2
W1	S2	C3
W2	S3	C4
W2	S3	C5
W2	S4	C4
W2	S4	C5

s与c无关, 但w可以多值推出s。w和s在一起也能推c



多值依赖的性质

- (1) 多值依赖具有对称性。即若 $X \twoheadrightarrow Y$, 则 $X \twoheadrightarrow Z$, 其中 $Z = U - X - Y$
 - 多值依赖的对称性可以用完全二分图直观地表示出来。
 - 从[例6.10] 容易看出, 因为每个保管员保管所有商品, 同时每种商品被所有保管员保管, 显然若 $W \twoheadrightarrow S$, 必然有 $W \twoheadrightarrow C$ 。
- (2) 多值依赖具有传递性。即若 $X \twoheadrightarrow Y$, $Y \twoheadrightarrow Z$, 则 $X \twoheadrightarrow Z - Y$ 。
- (3) 函数依赖是多值依赖的特殊情况。(一对多的特殊情况是一对一) 即若 $X \rightarrow Y$, 则 $X \twoheadrightarrow Y$ 。
- (4) 若 $X \twoheadrightarrow Y$, $X \twoheadrightarrow Z$, 则 $X \twoheadrightarrow YZ$ 。(Y U Z)
- (5) 若 $X \twoheadrightarrow Y$, $X \twoheadrightarrow Z$, 则 $X \twoheadrightarrow Y \cap Z$ 。
- (6) 若 $X \twoheadrightarrow Y$, $X \twoheadrightarrow Z$, 则 $X \twoheadrightarrow Y - Z$, $X \twoheadrightarrow Z - Y$ (在Z里面不在Y里面)。

多值依赖与函数依赖的区别

- (1) 多值依赖的有效性~~与属性集的范围有关~~
 - 若 $X \twoheadrightarrow Y$ 在U上成立, 则在W ($XY \subseteq W \subseteq U$) 上一定成立; 反之则不然, 即 $X \twoheadrightarrow Y$ 在W ($W \subset$ 真包含U) 上成立, 在U上并不一定成立。
 - 原因: 多值依赖的定义中不仅涉及属性组X和Y, 而且涉及U中其余属性Z。
- (2) 若函数依赖 $X \rightarrow Y$ 在R (U)上成立, 则对于任何 $Y' \subset Y$ 均有 $X \rightarrow Y'$ 成立。多值依赖 $X \twoheadrightarrow Y$ 若在R(U)上成立, 不能断言对于任何 $Y' \subset Y$ 有 $X \twoheadrightarrow Y'$ 成立

4NF

关系模式 $R \langle U, F \rangle \in 1NF$, 如果对于R的每个非平凡多值依赖 $X \twoheadrightarrow Y$ ($Y \not\subseteq X$), X都含有码, 则 $R \langle U, F \rangle \in 4NF$ 。(多值依赖左边的必须是码)

1. 4NF就是限制关系模式的属性之间不允许有非平凡且非函数依赖的多值依赖。4NF所允许的非平凡多值依赖实际上是函数依赖。
1. 如果一个关系模式是4NF, 则必为BCNF。
1. 在[例6.10]的WSC中, $W \twoheadrightarrow S$, $W \twoheadrightarrow C$, 他们都是非平凡多值依赖。而W不是码, 关系模式WSC的码是(W,S,C), 即All-key, 因此 $WSC \notin 4NF$ 。
1. 可以把WSC分解成 $WS(W,S)$, $WC(W,C)$, $WS \in 4NF$, $WC \notin 4NF$ 。

规范化小结

- ❖在关系数据库中, 对关系模式的基本要求是满足第一范式。
- ❖规范化程度过低的关系不一定能够很好地描述现实世界
 - 可能存在插入异常、删除异常、修改复杂、数据冗余等问题
 - 解决方法就是对其进行规范化, 转换成高级范式。
- ❖一个低一级范式的关系模式, 通过模式分解可以转换为若干个高一级范式的关系模式集合, 这个过程就叫关系模式的规范化。
- ❖关系数据库的规范化理论是数据库逻辑设计的工具。

思想:

1. 是逐步消除数据依赖中不合适的部分, 使模式中的各关系模式达到某种程度的“分离”。
2. 即采用“一事一地”的模式设计原则

●让一个关系描述一个概念、一个实体或者实体间的一种联系。

●若多于一个概念就把它“分离”出去。

3. 因此 规范化实质上是概念的单一化。

4. 不能说规范化程度越高的关系模式就越好



6.4数据依赖的公理系统

2022年2月8日 15:28

导航	$\subseteq \supseteq \subsetneq \not\subset \not\supseteq$
红色 重点 橙色 色：会背 蓝色 了解 绿色：可能	
公理系统	对于满足一组函数依赖F的关系模式 $R \langle U, F \rangle$，其任何一个关系r，若函数依赖 $X \rightarrow Y$ 都成立（即r中任意两组t、s，若 $t[X]=s[X]$，则 $t[Y]=s[Y]$），则称F逻辑蕴涵 $X \rightarrow Y$。 $x \rightarrow A$，$x \rightarrow B$这两个能推出 $x \rightarrow A \cup B$，则称F逻辑蕴涵 $x \rightarrow A \cup B$。u就是所有属性的集合。
Armstrong公理系统	■ 一套推理规则，是模式分解算法的理论基础 ■ 用途 ● 求给定关系模式的码 ● 从一组函数依赖求得蕴涵的函数依赖 Armstrong公理系统 设U为属性集总体，F是U上的一组函数依赖，于是有关系模式 $R \langle U, F \rangle$。对 $R \langle U, F \rangle$ 来说有以下的推理规则： ■ A1 自反律 (reflexivity rule)：若 $Y \subseteq X \subseteq U$，则 $X \rightarrow Y$ 为F所蕴涵。 (Sno,Cno) $\rightarrow$ Sno 为平凡函数依赖 ■ A2 增广律 (augmentation rule)：若 $X \rightarrow Y$ 为F所蕴涵，且 $Z \subseteq U$，则 $XZ \rightarrow YZ$ 为F所蕴涵。 ■ A3 传递律 (transitivity rule)：若 $X \rightarrow Y$ 及 $Y \rightarrow Z$ 为F所蕴涵，则 $X \rightarrow Z$ 为F所蕴涵。 注意：由自反律所得到的函数依赖均是平凡的函数依赖，自反律的使用并不依赖于F。

8.1 嵌入式sql

2022年2月12日

9:54

导航

红色 重点 橙色：会背 蓝色：了解 绿色：可能

≡ ≧ ≦ ≠ ≠

嵌入式SQL

主语言： 嵌入式SQL是将SQL语句嵌入程序设计语言中，被嵌入程序设计语言，如C、C++、Java，称为宿主语言，简称主语言。

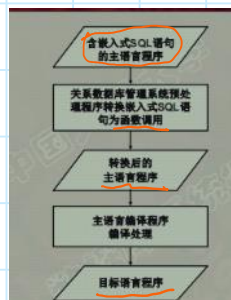
为了区分SQL语句与主语言语句，所有SQL语句必须加前缀EXEC SQL，
主语言为C语言时，语句格式：

EXEC SQL <SQL语句>;

主语言为Java语言时，语句格式：

#SQL <SQL语句>;

处理过程：



d

嵌入式SQL语句与主语言之间的通信

将SQL嵌入到高级语言中混合编程，程序中会含有两种不同计算模型的语句

■ SQL语句

- 描述性的面向集合的语句
- 负责操纵数据库

■ 高级语言语句

- 过程性的面向记录的语句
- 负责控制逻辑流程

数据库工作单元与源程序工作单元之间的通信：

- (1) 向主语言传递SQL语句的执行状态信息，使主语言能够据此控制程序流程，主要用SQL通信区(SQLCA)实现
- (2) 主语言向SQL语句提供参数，主要用主变量实现
- (3) 将SQL语句查询数据库的结果交主语言处理，主要用主变量和游标实现

SQL通信区

SQLCA： SQL语句执行后，系统反馈给应用程序信息，描述系统当前工作状态，描述运行环境这些信息将送到SQL通信区中。应用程序从SQL通信区中取出这些状态信息，据此决定接下来执行的语句。

定义SQLCA： 用EXEC SQL INCLUDE SQLCA定义

使用SQLCA：

1. SQLCA中有一个存放每次执行SQL语句后返回代码的变量SQLCODE
2. 如果SQLCODE等于预定义的常量SUCCESS，则表示SQL语句成功，否则表示出错
3. 应用程序每执行完一条SQL 语句之后都应该测试一下SQLCODE的值，以了解该SQL语句执行情况并做相应处理

主变量

定义： 嵌入式SQL语句中可以使用主语言的程序变量来输入或输出数据。在SQL语句中使用的主语言程序变量简称为主变量（Host Variable）。

类型:

1. 输入主变量： 由应用程序对其赋值， SQL语句引用
2. 输出主变量： 由SQL语句对其赋值或设置状态信息， 返回给应用程序
3. 指示变量： 是一个整型变量， 用来“指示”所指主变量的值或条件,一个主变量可以附带一个指示变量（Indicator Variable）， 指示变量的用途。指示输入主变量是否为空值， 检测输出变量是否为空值， 值是否被截断

7.1数据库设计

2022年2月9日 11:26

导航

红色 重点 橙色
色：会背 蓝色
了解 绿色：可能

⊆ ⊇ ⊂ ⊄ ⊅

★ 数据库设计

数据库设计是指对于一个给定的应用环境，构造（设计）优化的数据库逻辑模式和物理结构，并据此建立数据库及其应用系统，使之能够有效地存储和管理数据，满足各种用户的应用需求，包括信息管理要求和数据操作要求。

信息管理要求：在数据库中应该存储和管理哪些数据对象。

数据操作要求：对数据对象需要进行哪些操作，如查询、增、删、改、统计等操作。

设计目标

数据库设计的目标是为用户和各种应用系统提供一个信息基础设施和高效率的运行环境。

高效率的运行环境

1. 数据库数据的存取效率高
2. 数据库存储空间的利用率高
3. 数据库系统运行管理的效率高

数据库设计方法

它要求多方面的知识和技术。主要包括：

1. 计算机的基础知识
2. 软件工程的原理和方法
3. 程序设计的方法和技巧
4. 数据库的基本知识
5. 数据库设计技术
6. 应用领域的知识

规范设计法

1. 手工设计方法

基本思想：过程迭代和逐步求精

2. 典型方法

1. 新奥尔良（New Orleans）方法
2. 基于E-R模型的数据库设计方法
3. 3NF（第三范式）的设计方法
4. 面向对象的数据库设计方法
5. 统一建模语言（UML）方法

数据库设计步骤

数据库设计分6个阶段

1. 需求分析
2. 概念结构设计

1. 需求分析
2. 概念结构设计
3. 逻辑结构设计
4. 物理结构设计
5. 数据库实施
6. 数据库运行和维护

将概念结构转换为某个数据库管理系统所支持的数据模型

1. 需求分析和概念设计独立于任何数据库管理系统
2. 逻辑设计和物理设计与选用的数据库管理系统密切相关

参加数据库设计的人员

1. 系统分析人员和数据库设计人员：自始至终参与数据库设计，其水平决定了数据库系统的质量
2. 数据库管理员和用户代表：主要参加需求分析与数据库的运行和维护
3. 应用开发人员：包括程序员和操作员，在实施阶段参与进来，分别负责编制程序和准备软硬件环境

各阶段任务

1. 需求分析阶段

是否做得充分与准确，决定了构建数据库的速度和质量

2. 概念结构设计阶段 (ER)

通过对用户需求进行综合、归纳与抽象，形成一个独立于具体数据库管理系统的概念模型

3. 逻辑结构设计阶段

将概念结构转换为某个数据库管理系统所支持的数据模型，并对其进行优化

4. 物理结构设计阶段

为逻辑数据结构选取一个最适合应用环境的物理结构，包括存储结构和存取方法

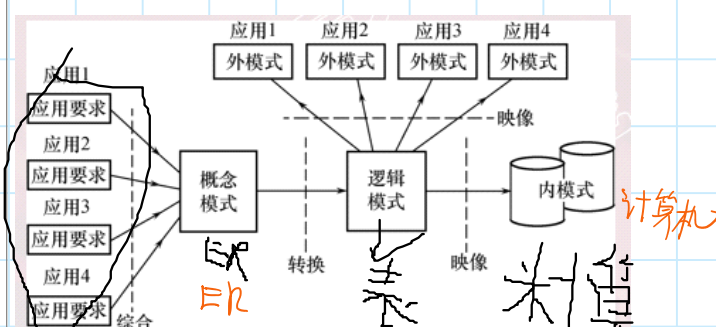
5. 数据库实施阶段

根据逻辑设计和物理设计的结果构建数据库，编写与调试应用程序，组织数据入库并进行试运行

6. 数据库运行和维护阶段

经过试运行后即可投入正式运行，在运行过程中必须不断对其进行评估、调整与修改

设计阶段	设计描述
需求分析	数字字典、全系统中数据项、数据结构、数据流、数据存储的描述 概念模型 (E-R 图)
概念结构设计	数据字典 某种数据模型 关系 非关系
逻辑结构设计	存储安排 存取方法选择 存取路径建立
物理结构设计	创建数据库模式 装入数据 数据库试运行
数据库实施	性能监测、转储恢复、数据库重组和重构





7.2需求分析

2022年2月9日 11:26

导航	$\subseteq \supseteq \subsetneq \not\subset \not\supseteq$
红色 重点 橙色：会背 蓝色 了解 绿色：可能	
需求分析	略
数据字典	数据字典是关于数据库中数据的描述，即元数据，不是数据本身，数据字典在需求分析阶段建立，在数据库设计过程中不断修改、充实、完善，数据字典是进行详细的数据收集和数据分析所获得的主要结果。 数据字典的内容 - 数据项 数据项是不可再分的数据单位， 数据项描述={数据项名,数据项含义说明,别名,数据类型,长度,取值范围,取值含义,与其他数据项的逻辑关系,数据项之间的联系} “取值范围”、“与其他数据项的逻辑关系”定义了数据的完整性约束条件，是设计数据检验功能的依据，可以用关系规范化理论为指导，用数据依赖的概念分析和表示数据项之间的联系 - 数据结构 一个数据结构可以由若干个数据项组成，也可以由若干个数据结构组成，或由若干个数据项和数据结构混合组成。 数据结构描述={数据结构名, 含义说明, 组成:{数据项或数据结构}} - 数据流 数据流是数据结构在系统内传输的路径。 数据流描述={数据流名,说明,数据流来源,数据流去向,组成:{数据结构},平均流量,高峰期流量} 数据流来源：说明该数据流来自哪个过程 数据流去向：说明该数据流将到哪个过程去 平均流量：在单位时间（每天、每周、每月等）里的传输次数 高峰期流量：在高峰时期的数据流量 - 数据存储 数据存储是数据结构停留或保存的地方，也是数据流的来源和去向之一。 对数据存储的描述 数据存储描述={数据存储名,说明,编号,输入的数据流,输出的数据流,

组成:{数据结构},数据量,
存取频度,存取方式}

存取频度: 每小时、每天或每周存取次数, 每次存取的数据量等信息

存取方法: 批处理 / 联机处理; 检索 / 更新; 顺序检索 / 随机检索

输入的数据流: 数据来源

输出的数据流: 数据去向

5. 处理过程

处理过程的**具体处理逻辑一般用判定表或判定树来描述**。数据字典中只需要描述处理过程的说明性信息

处理过程说明性信息的描述

处理过程描述={处理过程名,说明,输入:{数据流},
输出:{数据流},处理:{简要说明}}

简要说明: 说明该处理过程的功能及处理要求

功能: 该处理过程用来做什么

处理要求: 处理频度要求, 如单位时间里处理多少事务, 多少数据量、响应时间要求等

处理要求是后面物理设计的输入及性能评价的标准

7.3 概念结构设计

2022年2月9日

16:41

导航

红色 重点 橙色
色：会背 蓝色
了解 绿色：可能

$\subseteq \supseteq \subset \subsetneq \neq$

概念模型

将需求分析得到的用户需求抽象为信息结构（即概念模型）的过程就是概念结构设计。

概念模型的特点

- (1) 能真实、充分地反映现实世界，是现实世界的一个真实模型。
- (2) 易于理解，从而可以用它和不熟悉计算机的用户交换意见。
- (3) 易于更改，当应用环境和应用要求改变时，容易对概念模型修改和扩充。
- (4) 易于向关系、网状、层次等各种数据模型转换

描述概念模型的

1. 实体之间的联系

工具：E-R模型

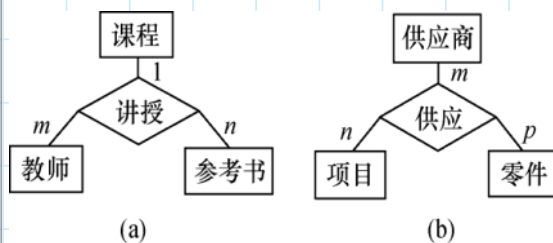
(1) 两个实体型之间的联系：

- ① 一对一联系 (1:1) 班级与班长
- ② 一对多联系 (1:n) 班级与学生
- ③ 多对多联系 (m:n) 课程与学生

(2) 两个以上的实体型之间的联系

一般地，两个以上的实体型之间也存在着一对一、一对多、多对多联系。

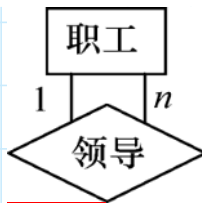
例：对于课程、教师与参考书3个实体型，如果一门课程可以有若干个教师讲授，使用若干本参考书，而每一个教师只讲授一门课程，每一本参考书只供一门课程使用，则课程与教师、参考书之间的联系是一对多的，如图7.7(a)所示。



(3) 单个实体型内的联系

同一个实体集内的各实体之间也可以存在一对一、一对多、多对多的联系。

例：职工实体型内部具有领导与被领导的联系，即某一职工（干部）“领导”若干名职工，而一个职工仅被另外一个职工直接领导，因此这是一对多的联系，如图7.8所示。



联系的度：参与联系的实体的数目

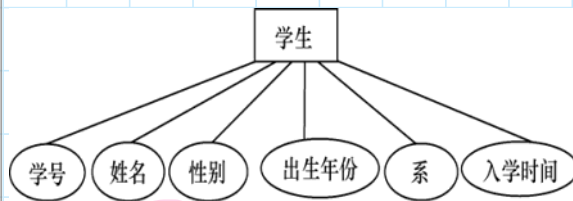
1. 2个实体型之间的联系度为2，也称为二元联系；
2. 3个实体型之间的联系度为3，称为三元联系；
3. N个实体型之间的联系度为N，也称为N元联系

E-R图

E-R图提供了表示实体型、属性和联系的方法：

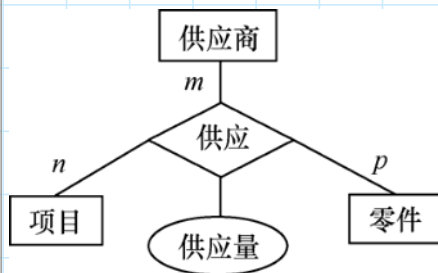
实体型：用矩形表示，矩形框内写明实体名。

属性：用椭圆形表示，并用无向边将其与相应的实体型连接起来。



联系：用菱形表示，菱形框内写明联系名，并用无向边分别与有关实体型连接起来，同时无向边旁标上联系类型（1:1, 1:n或m:n）。**联系可以具有属性**

如果一个联系具有属性，则这些属性也用无向边与该联系连接起来



举例：

某个工厂物资管理的概念模型。物资管理涉及的实体有：

仓库：属性有仓库号、面积、电话号码

零件：属性有零件号、名称、规格、单价、描述

供应商：属性有供应商号、姓名、地址、电话号码、账号

项目：属性有项目号、预算、开工日期

职工：属性有职工号、姓名、年龄、职称

联系：

(1) 一个仓库可以存放多种零件，一种零件可以存放在多个仓库中，因此仓库和零件具有多对多联系。用库存量来表示某种零件在某个仓库中的数量。

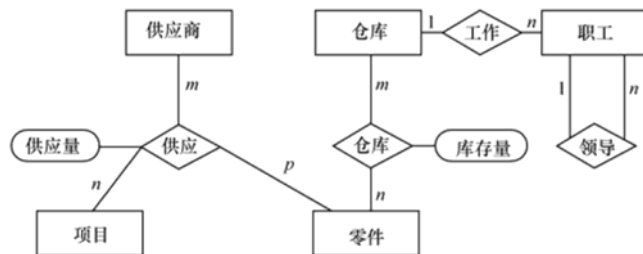
(2) 一个仓库有多个职工当仓库保管员，一个职工只能在一个仓库工作，因此仓库和职工之间是一对多的联系。

(3) 职工之间具有领导与被领导关系。即仓库主任领导若干保管员，因此职工实体型中具有一对多的联系。

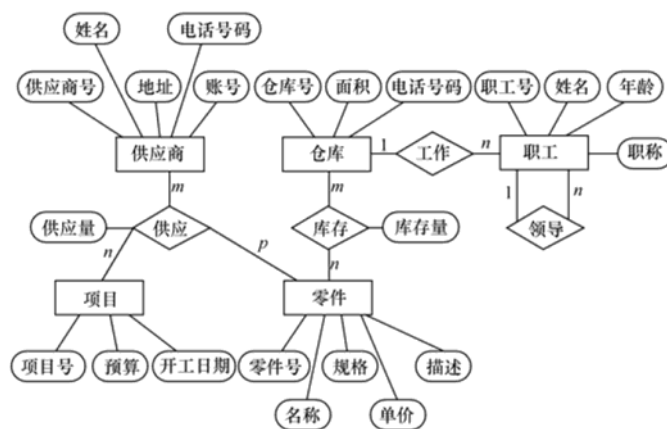
(4) 供应商、项目和零件三者之间具有多对多的联系。即一个供应商可以供给若干项目多种零件，每个项目可以使用不同供应商供应的零件，每种零件可由不同供应商供给。



(a) 实体及其属性图



(b) 实体及其联系图



(c) 完整的实体-联系图

概念结构设计

1. 实体与属性的划分原则

为了简化E-R图的处置，现实世界的事物能作为属性对待的，尽量作为属性对待。

两条准则：

- (1) 作为属性，不能再具有需要描述的性质。属性必须是不可分的数据项，不能包含其他属性。
- (2) 属性不能与其他实体具有联系，即E-R图中所表示的联系是实体之间的联系。

[例1] 职工是一个实体，职工号、姓名、年龄是职工的属性。

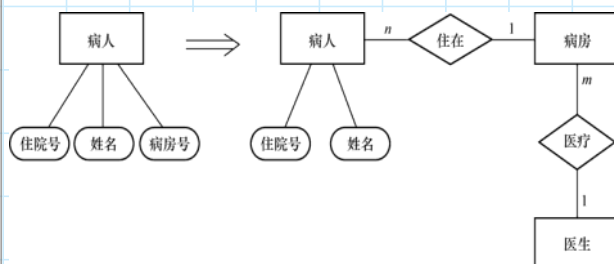
职称如果没有与工资、福利挂钩，根据准则（1）可以作为职工实体的属性

如果不同的职称有不同的工资、住房标准和不同的附加福利，则职称作为一个实体更恰当



[例2] 在医院中，一个病人只能住在一个病房，病房号可以作为病人实体的一个属性；

如果病房还要与医生实体发生联系，即一个医生负责几个病房的病人的医疗工作，则根据准则（2）病房应作为一个实体。



E-R集成

E-R图的集成一般需要分两步

1. 合并。解决各分E-R图之间的冲突，将分E-R图合并起来生成初步E-R图。

各个局部应用所面向的问题不同，各个子系统的E-R图之间必定会存在许多不一致的地方，称之为冲突。子系统E-R图之间的冲突主要有三类：

①属性冲突

1.属性域冲突，即属性值的类型、取值范围或取值集合不同。

例如零件号，有的部门把它定义为整数，有的部门把它定义为字符型。

年龄，某些部门以出生日期形式表示职工的年龄，而另一些部门用整数表示职工的年龄。

2.属性取值单位冲突。

例如，零件的重量有的以公斤为单位，有的以斤为单位，有的以克为单位。

②命名冲突

1.同名异义，即不同意义的对象在不同的局部应用中具有相同的名字。

2.异名同义（一义多名），即同一意义的对象在不同的局部应用中具有不同的名字。

如对科研项目，财务科称为项目，科研处称为课题，生产管理处称为工程。

③结构冲突

1.同一对象在不同应用中具有不同的抽象。

例如，职工在某一局部应用中被当作实体，而在另一局部应用中则被当作属性。

解决方法：把属性变换为实体或把实体变换为属性，使同一对象具有相同的抽象。

2.同一实体在不同子系统的E-R图所包含的属性个数和属性排列次序不完全相同。

解决方法：使该实体的属性取各子系统的E-R图中属性的并集，再适当调整属性的次序。

通过讨论、协商等行政手段加以解决

3.实体间的联系在不同的E-R图中为不同的类型。

实体E1与E2在一个E-R图中是多对多联系，在另一个E-R图中是一对多联系

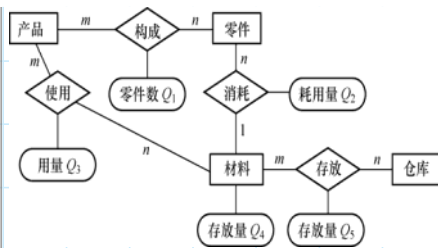
解决方法是根据应用的语义对实体联系的类型进行综合或调整。

2.修改和重构。消除不必要的冗余，生成基本E-R图。

所谓冗余的数据是指可由基本数据导出的数据，冗余的联系是指可由其他联系导出的联系。

消除冗余主要采用分析方法，即以数据字典和数据流图为依据，根据数据字典中关于数据项之间逻辑关系的说明来消除冗余。

如图， $Q3=Q1 \times Q2$ ， $Q4=\sum Q5$ 。所以Q3和Q4是冗余数据，可以消去。并且由于Q3消去，产品与材料间m:n的冗余联系也应消去。



并不是所有的冗余数据与冗余联系都必须加以消除，有时为了提高效率，不得不以冗余信息作为代价。

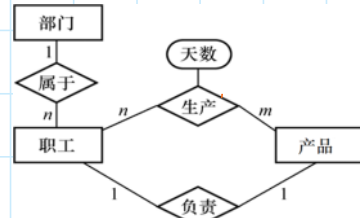
用规范化理论来消除冗余

①确定分E-R图实体之间的数据依赖。

实体之间一对一、一对多、多对多的联系可以用实体码之间的函数依赖来表示。于是有函数依赖集FL。

例：

部门和职工之间一对多的联系可表示为职工号→部门号 职工和产品之间多对多的联系可表示为 (职工号，产品号) →工作天数等。



②求FL的最小覆盖GL，差集为 $D=FL-GL$ 。

逐一考察D中的函数依赖，确定是否是冗余的联系，若是，就把它去掉。

注意：

冗余的联系一定在D中，而D中的联系不一定是冗余的；

当实体之间存在多种联系时，要将实体之间的联系在形式上加以区分

7.4逻辑结构设计

2022年2月10日

10:05

导航

红色 重点 橙

色：会背 蓝色

了解 绿色：可能

$\subseteq \supseteq \subsetneq \not\subset \not\supseteq$

逻辑结构设计的任务

把概念结构设计阶段设计好的基本E-R图转换为与选用数据库管理系统产品所支持的数据模型相符合的逻辑结构

E-R图向关系模型的转换

E-R图由实体型、实体的属性和实体型之间的联系三个要素组成。关系模型的逻辑结构是一组关系模式的集合。将E-R图转换为关系模型：将实体型、实体的属性和实体型之间的联系转化为关系模式。

转换原则

1. 一个实体型转换为一个关系模式。

关系的属性：实体的属性

关系的码：实体的码

2. 实体型间的联系有以下不同情况

(1) 一个1:1联系可以转换为一个独立的关系模式，也可以与任意一端对应的关系模式合并。

① 转换为一个独立的关系模式

关系的属性：与该联系相连的各实体的码以及联系本身的属性

关系的候选码：每个实体的码均是该关系的候选码

② 与某一端实体对应的关系模式合并

合并后关系的属性：加入对应关系的码和联系本身的属性

合并后关系的码：不变

(2) 一个1:n联系可以转换为一个独立的关系模式，也可以与n端对应的关系模式合并。

① 转换为一个独立的关系模式

关系的属性：与该联系相连的各实体的码以及联系本身的属性

关系的码：n端实体的码

② 与n端对应的关系模式合并

合并后关系的属性：在n端关系中**加入1端关系的码和联系本身的属性**

合并后关系的码：不变

可以减少系统中的关系个数，一般情况下更倾向于采用这种方法

(3) 一个m:n联系转换为一个关系模式

关系的属性：与该联系相连的各实体的码以及联系本身的属性

关系的码：各实体码的组合

(4) 三个或三个以上实体间的一个多元联系转换为一个关系模式。

关系的属性：与该多元联系相连的各实体的码以及联系本身的属性

关系的码：各实体码的组合

(5) 具有相同码的关系模式可合并 比如都有学生学号为主码

目的：减少系统中的关系个数

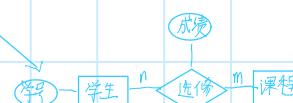
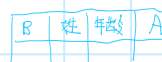
合并方法：

将其中一个关系模式的全部属性加入到另一个关系模式中



方法一：
B做为主码

方法二：
在n端加主码



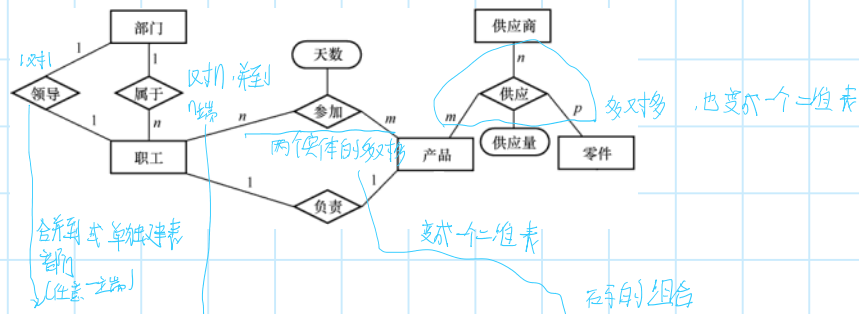
选修(学生, 课号, 成绩)

为学生选课

将其中一个关系模式的全部属性加入到另一个关系模式中
 然后去掉其中的同义属性（可能同名也可能不同名）
 适当调整属性的次序

□-□
 □-□
 □-□

举例:



- 部门 (部门号, 部门名, 经理的职工号, ...)
- 职工 (职工号, 部门号, 职工名, 职务, ...)
- 产品 (产品号, 产品名, 产品组长的职工号, ...)
- 供应商 (供应商号, 姓名, ...)
- 零件 (零件号, 零件名, ...)
- 职工工作 (职工号, 产品号, 工作天数, ...)
- 供应 (产品号, 供应商号, 零件号, 供应量)

数据模型的优化
 (不重要)

数据库逻辑设计的结果不是唯一的。得到初步数据模型后, 还应该适当地修改、调整数据模型的结构, 以进一步提高数据库应用系统的性能, 这就是数据模型的优化。

优化数据模型的方法:

(1) 确定数据依赖

按需求分析阶段所得到的语义, 分别写出每个关系模式内部各属性之间的数据依赖以及不同关系模式属性之间数据依赖。

(2) 对于各个关系模式之间的数据依赖进行极小化处理, 消除冗余的联系。

(3) 按照数据依赖的理论对关系模式进行分析, 考察是否存在部分函数依赖、传递函数依赖、多值依赖等, 确定各关系模式分别属于第几范式。

(4) 按照需求分析阶段得到的各种应用对数据处理的要求, 分析对于这样的应用环境这些模式是否合适, 确定是否要对它们进行合并或分解。

(5) 对关系模式进行必要分解, 提高数据操作效率和存储空间利用率。

常用分解方法

水平分解

垂直分解

水平分解

把(基本)关系的元组分为若干子集合, 定义每个子集合为一个子关系, 以提高系统的效率。

分解方法

对符合80/20的, 把经常被使用的数据 (约20%), 水平分解出来, 形成一个子关系。

水平分解为若干子关系, 使每个事务存取的数据对应一个子关系。

垂直分解

把关系模式R的属性分解为若干子集合, 形成若干子关系模式。

垂直分解的原则

经常在一起使用的属性从R中分解出来形成一个子关系模式

垂直分解的优点

可以提高某些事务的效率

垂直分解的缺点

可能使另一些事务不得不执行连接操作，降低了效率

定义用户外模式

- (1) 使用更符合用户习惯的别名
- (2) 针对不同级别的用户定义不同的视图，以保证系统的安全性
- (3) 简化用户对系统的使用

10.1事务的基本概念

2022年2月12日

9:57

导航

红色 重点 橙色

色：会背 蓝色

了解 绿色：可能

≡ ≧ ≡ ≠

事务

1. 事务(Transaction)是用户定义的一个数据库操作序列，这些操作要么全做，要么全不做，是一个不可分割的工作单位。是恢复和并发控制的基本单位。

事务和程序区别：

- 在关系数据库中，一个事务可以是一条SQL语句，一组SQL语句或整个程序
- 一个程序通常包含多个事务

2. 事务的定义

1. 显式定义方式

事务正常结束方式

COMMIT

- 事务正常结束
- 提交事务的所有操作（读+更新）
- 事务中所有对数据库的更新写回到磁盘上的物理数据库中

格式：

BEGIN TRANSACTION

SQL 语句1

SQL 语句2

.....

COMMIT

事务异常结束方式

ROLLBACK

- 事务异常终止
- 事务运行的过程中发生了故障，不能继续执行
- 系统将事务中对数据库的所有已完成的操作全部撤销
- 事务滚回到开始时的状态

格式：

BEGIN TRANSACTION

SQL 语句1

SQL 语句2

.....

ROLLBACK

定义事务的语句

BEGIN TRANSACTION;

COMMIT; 提交

ROLLBACK; 回滚

事物的特性
(ACID特性)

1. 原子性 (Atomicity)

事务是数据库的逻辑工作单位，事务中包括的诸操作要么都做，要么都不做

2. 一致性 (Consistency)

事务执行的结果必须是使数据库从一个一致性状态变到另一个一致性状态
一致性状态

- 数据库中只包含成功事务提交的结果

不一致状态

- 数据库系统运行中发生故障，有些事务尚未完成就被迫中断，这些未完成事务对数据库所做的修改有一部分已写入物理数据库，这时数据库就处于一种不正确的状态，

银行转帐：从帐号A中取出一万元，存入帐号B。

- 定义一个事务，该事务包括两个操作

A	B
$A = A - 1$	$B = B + 1$

- 这两个操作要么全做，要么全不做
 - 全做或者全不做，数据库都处于一致性状态。
 - 如果只做一个操作，用户逻辑上就会发生错误，少了一万元，数据库就处于不一致性状态。

3. 隔离性 (Isolation)

一个事务的执行不能被其他事务干扰

- 一个事务内部的操作及使用的数据对其他并发事务是隔离的，并发执行的各个事务之间不能互相干扰

4. 持续性 (Durability)，也称永久性

- 一个事务一旦提交，它对数据库中数据的改变就应该是永久性的。接下来的其他操作或故障不应该对其执行结果有任何影响。

保证事务ACID特性是事务处理的任务

破坏ACID特性的因素

(1) 多个事务并行运行时，不同事务的操作交叉执行

- 数据库管理系统必须保证多个事务的交叉运行不影响这些事务的隔离性

(2) 事务在运行过程中被强行停止

- 数据库管理系统必须保证被强行终止的事务对数据库和其他事务没有任何影响

10.2数据库恢复

2022年2月12日

11:02

导航

红色 重点 橙色
色：会背 蓝色
了解 绿色：可能

≡ ≧ ≡ ♀

数据库恢复

故障包括：

- 计算机硬件故障
- 软件的错误
- 操作员的失误
- 恶意的破坏

故障的影响

- 运行事务非正常中断，影响数据库中数据的正确性
- 破坏数据库，全部或部分丢失数据

数据库的恢复

- 数据库管理系统必须具有把数据库从错误状态恢复到某一已知的正确状态(亦称为一致状态或完整状态)的功能，这就是数据库的恢复管理系统对故障的对策，恢复子系统是数据库管理系统的一个重要组成部分，恢复技术是衡量系统优劣的重要指标

数据库中故障的种类

1.事务内部的故障

- 有的是可以通过事务程序本身发现的(见下面转账事务的例子)
 - 有的是非预期的（预测不到），不能由事务程序处理的。
 - 运算溢出
 - 并发事务发生死锁而被选中撤销该事务
 - 违反了某些完整性限制而被终止等
- 以后，事务故障仅指这类非预期的故障

事务故障的恢复：事务撤消（UNDO）

- 强行回滚（ROLLBACK）该事务，撤销该事务已经作出的任何对数据库的修改，使得该事务像根本没有启动一样

2.系统故障

称为软故障，是指造成系统停止运转的任何事件，使得系统要重新启动。

表现：

- 整个系统的正常运行突然被破坏
- 所有正在运行的事务都非正常终止
- 不破坏数据库
- 内存中数据库缓冲区的信息全部丢失

常见原因：

- 特定类型的硬件错误（如CPU故障）
- 操作系统故障

c. 数据库管理系统代码错误

d. 系统断电

恢复策略:

表现1: 发生系统故障时, 一些尚未完成的事务的结果可能已送入物理数据库, 造成数据库可能处于不正常的状态。

方法: 系统重新启动时, 恢复程序让所有非正常终止的事务回滚, 强行撤消 (UNDO) 所有未完成事务。

表现2: 发生系统故障时, 有些已完成的事务可能有一部分甚至全部留在缓冲区, 尚未写入磁盘上的物理数据库中, 系统故障使得这些事务对数据库的修改部分或全部丢失。

方法: 系统重新启动时, 恢复程序需要重做 (REDO) 所有已提交的事务

3. 介质故障

称为硬故障, 指外存故障, 如磁盘损坏、磁头碰撞、瞬时强磁场干扰

表现: 介质故障破坏数据库或部分数据库, 并影响正在存取这部分数据的所有事务, 介质故障比前两类故障的可能性小得多, 但破坏性大得多

4. 计算机病毒

一种人为的故障或破坏, 是一些恶作剧者研制的一种计算机程序可以繁殖和传播, 造成对计算机系统包括数据库的危害。

计算机病毒的特点: 隐蔽性、潜伏性、传染性、破坏性、寄生性。

10.3恢复的实现技术

2022年2月12日

14:41

导航

红色 重点 橙色
蓝色
会背
了解 绿色: 可能

≡ ≧ ≦ ≠ ≠

恢复机制的关键问题

恢复操作的基本原理: 冗余

利用存储在系统别处的冗余数据来重建数据库中已被破坏或不正确的那部分数据
复的实现技术: 复杂

一个大型数据库产品, 恢复子系统的代码要占全部代码的10%以上

1. 如何建立冗余数据

■数据转储 (backup)

■登记日志文件 (logging)

2. 如何利用这些冗余数据实施数据库恢复

数据转储

1. 定义:

转储是指数据库管理员定期地将整个数据库复制到磁带、磁盘或其他存储介质上保存起来的过程, 备用的数据文本称为后备副本(backup)或后援副本。

说明: 数据库遭到破坏后可以将后备副本重新装入, 重装后备副本只能将数据库恢复到转储时的状态, 要想恢复到故障发生时的状态, 必须重新运行自转储以后的所有更新事务

2. 转储方法

(1) 静态转储与动态转储

从状态分类

静态转储: 在系统中无运行事务时进行的转储操作, 转储开始时数据库处于一致性状态, 转储期间不允许对数据库的任何存取、修改活动, 得到的一定是一个数据一致性的副本。

优点: 实现简单

缺点: 降低了数据库的可用性, 转储必须等待正运行的用户事务结束, 新的事务必须等转储结束

动态转储: 转储操作与用户事务并发进行, 转储期间允许对数据库进行存取或修改。

优点: 不用等待正在运行的用户事务结束, 不会影响新事务的运行。

缺点: 不能保证副本中的数据正确有效。

例: 在转储期间的某时刻 T_c , 系统把数据 $A=100$ 转储到磁带上, 而在下一时刻 T_d , 某一事务将 A 改为200。后备副本上的 A 过时了。

说明:

利用动态转储得到的副本进行故障恢复, 需要把动态转储期间各事务对数据库的修改活动登记下来, 建立日志文件, 后备副本加上日志文件就能把数据库恢复到某一时刻的正确状态

(2) 海量转储与增量转储

从内容分类

海量转储: 每次转储全部数据库。

增量转储: 只转储上次转储后更新过的数据。

比较:

- 从恢复角度看, 使用海量转储得到的后备副本进行恢复往往更方便
- 如果数据库很大, 事务处理又十分频繁, 则增量转储方式更实用更有效

总结:

❖ 转储方法分类		
转储方式	转储状态	
	动态转储	静态转储
海量转储	动态海量转储	静态海量转储
增量转储	动态增量转储	静态增量转储

日志文件

定义: 日志文件(log file)是用来记录事务对数据库的更新操作的文件。

格式:

- 以记录为单位的日志文件

日志文件内容:

各个事务的开始标记(BEGIN TRANSACTION)

各个事务的结束标记(COMMIT或ROLLBACK)

各个事务的所有更新操作

以上均作为日志文件中的一个日志记录(log record)

日志记录的内容:

事务标识 (标明是哪个事务)

操作类型 (插入、删除或修改)

操作对象 (记录ID、记录内部标识)

更新前数据的旧值 (对插入操作而言, 此项为空值)

更新后数据的新值 (对删除操作而言, 此项为空值)

- 以数据块为单位的日志文件

日志记录内容:

事务标识

被更新的数据块

用途

- 进行事务故障恢复
- 进行系统故障恢复
- 协助后备副本进行介质故障恢复

作用

- 事务故障恢复和系统故障恢复必须用日志文件。
- 在动态转储方式中必须建立日志文件, 后备副本和日志文件结合起来才能有效地恢复数据库。
- 在静态转储方式中, 也可以建立日志文件。当数据库毁坏后可重新装入后备副本把数据库恢复到转储结束时刻的正确状态, 利用日志文件, 把已完成的事务进行重做处理, 对故障发生时尚未完成的事务进行撤销处理

登记记录文件

为保证数据库是可恢复的，登记日志文件时必须遵循两条原则

1. 登记的次序严格按并发事务执行的时间次序。
2. 必须先写日志文件，后写数据库。
 - 写日志文件操作：把表示这个修改的日志记录写到日志文件中
 - 写数据库操作：把对数据的修改写到数据库中

10.4恢复策略

2022年2月13日

9:41

导航

红色 重点
橙色 会背
蓝色 了解
绿色 可能

≡ ≧ ≡ ≠ ≠

事务故障

定义：事务在运行至正常终止点前被终止。

恢复方法：

恢复子系统利用日志文件撤消（UNDO）此事务已对数据库进行的修改。

恢复步骤：

- （1）反向扫描文件日志（即从最后向前扫描日志文件），查找该事务的更新操作。
- （2）对该事务的更新操作执行逆操作（即回滚）。即将日志记录中“更新前的值”写入数据库。
 - 插入操作，“更新前的值”为空，则相当于做删除操作
 - 删除操作，“更新后的值”为空，则相当于做插入操作
 - 修改操作，则相当于用修改前值代替修改后值
- （3）继续反向扫描日志文件，查找该事务的其他更新操作，并做同样处理。
- （4）如此处理下去，直至读到此事务的开始标记，事务故障恢复就完成了（即查到begin）。

系统故障恢复

系统故障造成数据库不一致状态的原因：

1. 未完成事务对数据库的更新可能已写入数据库（即写入到物理磁盘）
2. 已提交事务对数据库的更新可能还留在缓冲区来不及写入数据库

恢复方法：

1. Undo（撤销）故障发生时未完成的事务
2. Redo（重做）已完成的事务

系统故障的恢复由系统在重新启动时自动完成，不需要用户干预（即对用户来说是透明的）

恢复步骤：

- （1）正向扫描日志文件（即从头扫描日志文件）
在故障发生前已经提交的事务，加入到重做队列。
故障发生时尚未完成的事务，加入到撤销队列。
- （2）对撤销(UNDO)队列事务进行撤销(UNDO)处理
反向扫描日志文件，对每个撤销事务的更新操作执行逆操作，即将日志记录中“更新前的值”写入数据库。
- （3）对重做(REDO)队列事务进行重做(REDO)处理
正向扫描日志文件，对每个重做事务重新执行登记的操作，即将日志记录中“更新后的值”写入数据库。

介质故障恢复	定义：介质故障后磁盘上的物理数据和日志文件被破坏，这是最严重的一种故障。 恢复方法： 重装数据库，然后重做已完成得到事务 恢复步骤： (1) 装入最新的后备数据库副本(离故障发生时刻最近的转储副本)，使数据库恢复到最近一次转储时的一致性状态。 对于静态转储的数据库副本，装入后数据库即处于一致性状态。 对于动态转储的数据库副本，还须同时装入转储时刻的日志文件副本，利用恢复系统故障的方法，(即REDO+UNDO)，才能将数据库恢复到一致性状态。 (2) 装入有关的日志文件副本(转储结束时刻的日志文件副本)，重做已完成的事务。 首先扫描日志文件，找出故障发生时已提交的事务的标识，将其记入重做队列。然后正向扫描日志文件，对重做队列中的所有事务进行重做处理。即将日志记录中“更新后的值”写入数据库。
--------	--

10.5带有检查点的恢复技术

2022年2月13日

10:17

导航

红色 重点
橙色：会背
蓝色
了解 绿色：可能

⊆ ⊇ ⊂ ⊄

检查点技术

问题提出：搜索整个日志将耗费大量的时间，重做处理：重新执行，浪费了大量时间。

解决方法：具有检查点 (checkpoint) 的恢复技术

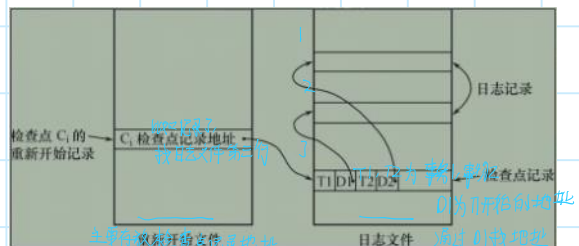
1. 在日志文件中增加检查点记录 (checkpoint)
2. 增加重新开始文件
3. 恢复子系统在登录日志文件期间动态地维护日志

检查点记录的内容：

建立检查点时刻所有正在执行的事务清单
这些事务最近一个日志记录的地址

重新开始文件的内容：

记录各个检查点记录在日志文件中的地址



动态维护日志文件的方法：

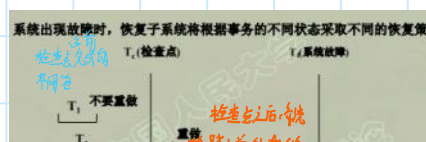
周期性地执行如下操作：建立检查点，保存数据库状态。

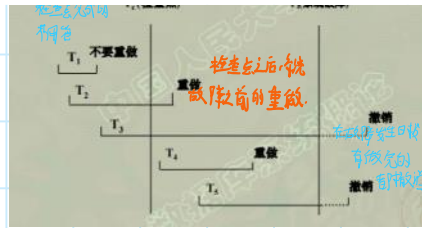
具体步骤是：

- (1) 将当前日志缓冲区中的所有日志记录写入磁盘的日志文件上
- (2) 在日志文件中写入一个检查点记录
- (3) 将当前数据缓冲区的所有数据记录写入磁盘的数据库中
- (4) 把检查点记录在日志文件中的地址写入一个重新开始文件

利用检查点的恢复策略：

1. 当事务T在一个检查点之前提交，T对数据库所做的修改已写入数据库
2. 写入时间是在这个检查点建立之前或在这个检查点建立之时
3. 在进行恢复处理时，没有必要对事务T执行重做操作





利用检查点的恢复步骤:

- (1) 从重新开始文件中找到最后一个检查点记录, 在日志文件中的地址, 由该地址在日志文件中找到最后一个检查点记录。
- (2) 由该检查点记录得到检查点建立时刻所有正在执行的事务清单ACTIVE-LIST。建立两个事务队列 UNDO-LIST 和REDO-LIST, 把ACTIVE-LIST暂时放入UNDO-LIST队列, REDO队列暂为空。
- (3) 从检查点开始正向扫描日志文件, 直到日志文件结束。
 - 1.如有新开始的事务 T_i , 把 T_i 暂时放入UNDO-LIST队列
 2. 如有提交的事务 T_j , 把 T_j 从UNDO-LIST队列移到REDO LIST队列;直到日志文件结束
- (4) 对UNDO-LIST中的每个事务执行UNDO操作, 对REDO-LIST中的每个事务执行REDO操作

10.6数据库镜像

2022年2月13日

11:15

导航

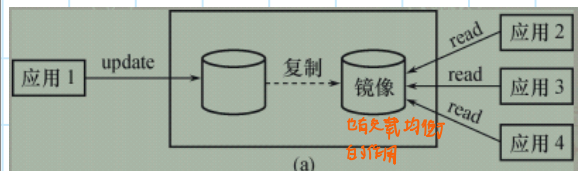
红色 重点 橙色
色：会背 蓝色
了解 绿色：可能

≡ ≡ ≡ ≠ ≠

数据库镜像

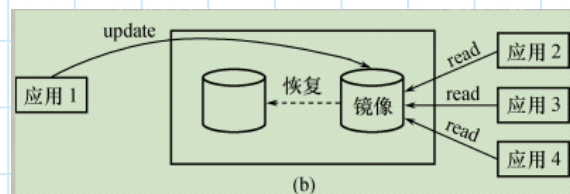
为预防介质故障，提高数据库可用性，提出数据库镜像（Mirror）解决方案。

定义：数据库管理系统自动把整个数据库或其中的关键数据复制到另一个磁盘上。数据库管理系统自动保证镜像数据与主数据的一致性每当主数据库更新时，数据库管理系统自动把更新后的数据复制过去。



出现介质故障时

1. 可由镜像磁盘继续提供使用
2. 同时数据库管理系统自动利用镜像磁盘数据进行数据库的恢复
3. 不需要关闭系统和重装数据库副本



没有出现故障时

1. 可用于并发操作
2. 一个用户对数据加排他锁修改数据，其他用户可以读镜像数据库上的数据，而不必等待该用户释放锁

11.1并发控制概述

2022年2月13日

13:30

导航

红色 重点 橙色
蓝色
色：会背
了解 绿色：可能

⊆ ⊇ ⊂ ⊄

多用户数据库系统

定义：允许多个用户同时使用的数据库系统,其特点在同一时刻并发运行的事务数可达数百上千个。

像飞机订票系统

多事务执行方式

(1) 事务串行执行

每个时刻只有一个事务运行，其他事务必须等到这个事务结束以后方能运行。

评价：不能充分利用系统资源，发挥数据库共享资源的特点

(2) 交叉并发方式（本章讲的）

在单处理机系统（可以理解为一台电脑）中，事务的并行执行是这些并行事务的并行操作轮流交叉运行。
并没有真正地并行运行。

评价：单处理机系统中的并行事务并没有真正地并行运行，但能够减少处理机的空闲时间，提高系统的效率

(3) 同时并发方式

多处理机系统中，每个处理机可以运行一个事务，多个处理机可以同时运行多个事务，实现多个事务真正的并行运行

评价：最理想的并发方式，但受制于硬件环境，更复杂的并发方式机制

并发控制

事务是并发控制的基本单位，并发控制机制的任务：

1. 对并发操作进行正确调度
2. 保证事务的隔离性（执行该事务，别的事务别打扰他）
3. 保证数据库的一致性

举例：

[例11.1]飞机订票系统中的一个活动序列，① 甲售票点(事务T1)读出某航班的机票余额A，设A=16；② 乙售票点(事务T2)读出同一航班的机票余额A，也为16；③ 甲售票点卖出一张机票，修改余额 $A \leftarrow A-1$ ，所以A为15，把A写回数据库；④ 乙售票点也卖出一张机票，修改余额 $A \leftarrow A-1$ ，所以A为15，把A写回数据库 结果明明卖出两张机票，数据库中机票余额只减少1。

原因：这种情况称为数据库的不一致性，是由并发操作引起的。在并发操作情况下，对T1、T2两个事务的操作序列的调度是随机的。在并发操作情况下，对T1、T2两个事务的操作序列的调度是随机的。若按上面的调度序列执行，T1事务的修改就被丢失。

并发操作带来的数据不一致性，R(x):读数据x，W(x):写数据x

1.丢失修改（Lost Update）

两个事务T1和T2读入同一数据并修改，T2的提交结果破坏了T1提交的结果，导致T1的修改被丢失。

2.不可重复读 (Non-repeatable Read)

事务T1读取数据后, 事务T2执行更新操作, 使T1无法再现前一次读取结果

不可重复读包括三种情况:

- (1) 事务T1读取某一数据后, 事务T2对其做了修改, 当事务T1再次读该数据时, 得到与前一次不同的值。
别人修改
- (2) 事务T1按一定条件从数据库中读取了某些数据记录后, 事务T2删除了其中部分记录, 当T1再次按相同条件读取数据时, 发现某些记录神秘地消失了。
别人删除
- (3) 事务T1按一定条件从数据库中读取某些数据记录后, 事务T2插入了一些记录, 当T1再次按相同条件读取数据时, 发现多了一些记录。
别人插入

后两种不可重复读有时也称为幻影现象。

3.读“脏”数据 (Dirty Read)

即不正确的数据

举例:

1. 事务T1修改某一数据, 并将其写回磁盘
2. 事务T2读取同一数据后, T1由于某种原因被撤销
3. 这时T1已修改过的数据恢复原值, T2读到的数据就与数据库中的数据不一致
4. T2读到的数据就为“脏”数据, 即不正确的数据

数据不一致原因:

数据不一致性: 由于并发操作破坏了事务的隔离性。不一致性原因

并发控制就是要用正确的方式调度并发操作, 使一个用户事务的执行不受其他事务的干扰, 从而避免造成数据的不一致性。

并发控制的主要技术

1. 封锁(Locking)
2. 时间戳(Timestamp)
3. 乐观控制法
4. 多版本并发控制(MVCC)

11.2封锁

2022年2月13日

15:11

导航

红色 重点 橙

色：会背 蓝色

了解 绿色：可能

⊆ ⊇ ⊈ ⊉

封锁

定义：封锁就是事务T在对某个数据对象（例如表、记录等）操作之前，先向系统发出请求，对其加锁。加锁后事务T就对该数据对象有了一定的控制，在事务T释放它的锁之前，其它的事务不能更新（有时候可以读）此数据对象。

基本封锁类型：

一个事务对某个数据对象加锁后究竟拥有什么样的控制由封锁的类型决定。

1. 排它锁（Exclusive Locks，简记为X锁）

又称为写锁，若事务T对数据对象A加上X锁，则只允许T读取和修改A，其它任何事务都不能再对A加任何类型的锁，直到T释放A上的锁，保证其他事务在T释放A上的锁之前不能再读取和修改A。

2. 共享锁（Share Locks，简记为S锁）

又称为读锁，若事务T对数据对象A加上S锁，则事务T可以读A但不能修改A，其它事务只能再对A加S锁，而不能加X锁，直到T释放A上的S锁。保证其他事务可以读A，但在T释放A上的S锁之前不能对A做任何修改。

排它锁与共享锁的控制方式

$T_1 \backslash T_2$	X	S	-
X	N	N	Y
S	N	Y	Y
-	Y	Y	Y

Y=Yes，相容的请求
N=No，不相容的请求

见X后不能加任何锁

加S后不能加X锁

11.3封锁协议

2022年2月13日 15:44

导航

红色 重点 橙色
色：会背 蓝色
了解 绿色：可能

封锁协议

定义：在运用X锁和S锁对数据对象加锁时，需要约定一些规则，这些规则为封锁协议（Locking Protocol）。例如何时申请X锁或S锁、持锁时间、何时释放。对封锁方式规定不同的规则，就形成了各种不同的封锁协议，它们分别在不同的程度上为并发操作的正确调度提供一定的保证。

常用封锁协议：

三级封锁协议

1.一级封锁协议

修改要加排他锁

事务T在修改数据R之前必须先对其加X锁，直到事务结束才释放。事务结束包括正常结束和非正常结束。

作用：可防止丢失修改，并保证事务T是可恢复的。

问题：如果仅仅是读数据不对其进行修改，是不需要加锁的，所以它不能保证可重复读和不读

“脏”数据。

2.二级封锁协议

即加X锁，也加S锁

一级封锁协议加上事务T在读取数据R之前必须先对其加S锁，读完后即可释放S锁。

作用：二级封锁协议可以防止丢失修改和读“脏”数据。

问题：由于读完数据后即可释放S锁，所以它不能保证可重复读。

3.三级封锁协议

一级封锁协议加上事务T在读取数据R之前必须先对其加S锁，直到事务结束才释放。协议时间不同

作用：可防止丢失修改、读脏数据和不可重复读。

T₁ ① Xlock A ② R(A)=14 ③ A←A-1 ④ W(A)=15 ⑤ Commit ⑥ Unlock A	T₂ Xlock A 等待 等待 等待 获得Xlock A R(A)=15 A←A-1 W(A)=14 Commit Unlock A	没有丢失修改 ■ 事务T₁在读A进行修改之前先对A加X锁 ■ 当T₂再请求对A加X锁时被拒绝 ■ T₂只能等待T₁释放A上的锁后再获得对A的X锁 ■ 这时T₂读到的A已经是T₁更新过的值15 ■ T₂按此新的A值进行运算，并将结果是A=14写回到磁盘，避免了丢失T₁的更新。
T₁ ① Xlock C R(C)=100 C←C+2 W(C)=200 ② ROLLBACK (C恢复为100) Unlock C ③ ④ ⑤	T₂ Slock C 等待 等待 获得Slock C R(C)=100 Commit C Unlock C	不读“脏”数据 ■ 事务T₁在对C进行修改之前，先对C加X锁，修改其值后写回磁盘 ■ T₂请求在C上加S锁，因T₁已在C上加X锁，T₂只能等待 ■ T₁因某种原因被撤销，C恢复为原值100 ■ T₂释放C上的X锁后T₁获得C上的S锁，读C=100，避免了T₂读“脏”数据
T₁ ① Xlock A ② Xlock B ③ Xlock C ④ Xlock A ⑤ Xlock B ⑥ Xlock C ⑦ Xlock A ⑧ Xlock B ⑨ Xlock C ⑩ Xlock A ⑪ Xlock B ⑫ Xlock C ⑬ Xlock A ⑭ Xlock B ⑮ Xlock C ⑯ Xlock A ⑰ Xlock B ⑱ Xlock C ⑲ Xlock A ⑳ Xlock B ㉑ Xlock C ㉒ Xlock A ㉓ Xlock B ㉔ Xlock C ㉕ Xlock A ㉖ Xlock B ㉗ Xlock C ㉘ Xlock A ㉙ Xlock B ㉚ Xlock C ㉛ Xlock A ㉜ Xlock B ㉝ Xlock C ㉞ Xlock A ㉟ Xlock B ㊱ Xlock C ㊲ Xlock A ㊳ Xlock B ㊴ Xlock C ㊵ Xlock A ㊶ Xlock B ㊷ Xlock C ㊸ Xlock A ㊹ Xlock B ㊺ Xlock C ㊻ Xlock A ㊼ Xlock B ㊽ Xlock C ㊾ Xlock A ㊿ Xlock B	T₂ Xlock A 等待 等待 等待 获得Xlock A ① Xlock B ② Xlock C ③ Xlock A ④ Xlock B ⑤ Xlock C ⑥ Xlock A ⑦ Xlock B ⑧ Xlock C ⑨ Xlock A ⑩ Xlock B ⑪ Xlock C ⑫ Xlock A ⑬ Xlock B ⑭ Xlock C ⑮ Xlock A ⑯ Xlock B ⑰ Xlock C ⑱ Xlock A ⑲ Xlock B ⑳ Xlock C ㉑ Xlock A ㉒ Xlock B ㉓ Xlock C ㉔ Xlock A ㉕ Xlock B ㉖ Xlock C ㉗ Xlock A ㉘ Xlock B ㉙ Xlock C ㉚ Xlock A ㉛ Xlock B ㉜ Xlock C ㉝ Xlock A ㉞ Xlock B ㉟ Xlock C ㊱ Xlock A ㊲ Xlock B ㊳ Xlock C ㊴ Xlock A ㊵ Xlock B ㊶ Xlock C ㊷ Xlock A ㊸ Xlock B ㊹ Xlock C ㊺ Xlock A ㊻ Xlock B ㊼ Xlock C ㊽ Xlock A ㊾ Xlock B ㊿ Xlock C	可重复读 ■ 事务T₁在读A、B之前，先对A、B加S锁 ■ 其他事务只能再对A、B加S锁，而不能加X锁，即其他事务只能读A、B，而不能修改 ■ 当T₂为修改B而申请对B的X锁时被拒绝只能等待T₁释放B上的锁 ■ T₂为计算B值，B，这时读出的B仍是100，求和结果仍为150，即可重复读 ■ T₁结束才释放A、B上的S锁，T₂才获得对B的X锁

三级协议的主要区别：

什么操作需要申请封锁以及何时释放锁（即持锁时间）

不同的封锁协议使事务达到的一致性级别不同：

封锁协议级别越高，一致性程度越高

	X锁		S锁		一致性保证		
	操作结束释放	事务结束释放	操作结束释放	事务结束释放	不丢失修改	不读“脏”数据	可重复读
一级封锁协议		√			√		
二级封锁协议		√	√		√	√	
三级封锁协议		√		√	√	√	√

11.4活锁和死锁

2022年2月13日

17:33

导航

红色 重点 橙

色：会背 蓝色

了解 绿色：可能

≡ ≧ ≡ ≠ ≠

活锁

例子：事务T1封锁了数据R，事务T2又请求封锁R，于是T2等待。T3也请求封锁R，当T1释放了R上的封锁之后系统首先批准了T3的请求，T2仍然等待。T4又请求封锁R，当T3释放了R上的封锁之后系统又批准了T4的请求……T2有可能永远等待，这就是活锁的情形。（即无限等待）活锁就是无限等待

避免死锁：采用先来先服务的策略，当多个事务请求封锁同一数据对象时，按请求封锁的先后次序对这些事务排队，该数据对象上的锁一旦释放，首先批准申请队列中第一个事务获得锁。

死锁

例子：事务T1封锁了数据R1，T2封锁了数据R2，T1又请求封锁R2，因T2已封锁了R2，于是T1等待T2释放R2上的锁，接着T2又申请封锁R1，因T1已封锁了R1，T2也只能等待T1释放R1上的锁，这样T1在等待T2，而T2又在等待T1，T1和T2两个事务永远不能结束，形成死锁。

产生死锁的原因：

是两个或多个事务都已封锁了一些数据对象，然后又都请求对已为其他事务封锁的数据对象加锁，从而出现死等待。

解决死锁的方法：

1. 死锁的预防（在数据库中不可行）

预防死锁的发生就是要破坏产生死锁的条件

预防死锁的方法：

(1) 一次封锁法

要求每个事务必须一次将所有要使用的数据全部加锁，否则就不能继续执行。

问题：

1.降低系统并发度。

2.难于事先精确确定封锁对象（即最初很难知道自己需要什么事务）。

(2) 顺序封锁法

顺序封锁法是预先对数据对象规定一个封锁顺序，所有事务都按这个顺序实行封锁（按顺序封锁）。

问题：

1.维护成本高

数据库系统中封锁的数据对象极多，并且随数据的插入、删除等操作而不断地变化，要维护这样的资源的封锁顺序非常困难，成本很高

2.难以实现

事务的封锁请求可以随着事务的执行而动态地决定，很难事先确定每一个事务要封锁哪些对象，因此也就很难按规定的顺序去施加封锁

2. 死锁的诊断与解除

死锁的诊断：

(1) 超时法

如果一个事务的等待时间超过了规定的时限，就认为发生了死锁。

优点：实现简单

缺点：有可能误判死锁，时限若设置得太长，死锁发生后不能及时发现

(2) 等待图法

用事务等待图动态反映所有事务的等待情况。事务等待图是一个有向图 $G=(T, U)$ ，其中 T 为结点的集合，每个结点表示正运行的事务， U 为边的集合，每条边表示事务等待的情况。若 T_1 等待 T_2 ，则 T_1, T_2 之间划一条有向边，从 T_1 指向 T_2 。

并发控制子系统周期性地（比如每隔数秒）生成事务等待图，检测事务。如果发现图中存在回路，则表示系统中出现了死锁。

解除死锁：

选择一个处理死锁代价最小的事务，将其撤消，释放此事务持有的所有的锁，使其它事务能继续运行下去

11.5 并发调度的可串行性

2022年2月14日

15:42

导航

红色 重点 橙色
色：会背 蓝色
了解 绿色：可能

$\subseteq \supseteq \subsetneq \not\subsetneq$

可串行化调度

定义：多个事务的并发执行是正确的，当且仅当其结果与按某一次序串行地执行这些事务时的结果相同。

并行运行刚结束和串行相同。

可串行性(Serializability)：是并发事务正确调度的准则，一个给定的并发调度，当且仅当它是可串行化的，才认为是正确调度。

冲突可串行化调度

冲突操作：是指不同的事务对同一数据的读写操作和写写操作：

1. $R_i(x)$ 与 $W_j(x)$ /*事务 T_i 读 x , T_j 写 x , 其中 $i \neq j$ */
2. $W_i(x)$ 与 $W_j(x)$ /*事务 T_i 写 x , T_j 写 x , 其中 $i \neq j$ */
3. 其他操作是不冲突操作

说明：

1. 除读写操作和写写操作外，其他操作是不冲突操作
2. 不同事务的冲突操作和同一事务的两个操作是不能交换的。
 - a. $R_i(x)$ 与 $W_j(x)$ 改变执行次序，则 $T_i(x)$ 的数据身将发生变化。
 - b. $W_i(x)$ 与 $W_j(x)$ 改变执行次序，则 x 的数据由等于 T_j 的结果，变为 T_i 的结果。

定义：

一个调度 Sc 在保证冲突操作的次序不变的情况下，通过交换两个事务不冲突操作的次序得到另一个调度 Sc' ，如果 Sc' 是串行的，称调度 Sc 是冲突可串行化的调度。

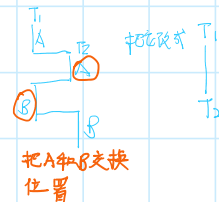
通过交换变为串行操作

说明：若一个调度是冲突可串行化，则一定是可串行化的调度。

可用这种方法判断一个调度是否是冲突可串行化的。

举例：

【例】今有调度
 $Sc_1 = r_1(A) w_1(A) r_2(A) w_2(A) r_1(B) w_1(B) r_2(B) w_2(B)$
把 $w_2(A)$ 与 $r_1(B) w_1(B)$ 交换，得到：
 $r_1(A) w_1(A) r_2(A) r_1(B) w_1(B) w_2(A) r_2(B) w_2(B)$
再把 $r_2(A)$ 与 $r_1(B) w_1(B)$ 交换，得到：
 $Sc_2 = r_1(A) w_1(A) r_1(B) w_1(B) r_2(A) w_2(A) r_2(B) w_2(B)$
 $\quad \quad \quad T_1 \quad \quad \quad T_2$
 Sc_2 等价于串行调度 T_1, T_2 ，所以 Sc_1 冲突可串行化的调度。



Sc_2 等价于一个串行调度 T_1, T_2 。所以 Sc_1 冲突可串行化的调度。

冲突可串行化调度是可串行化调度的充分条件，不是必要条件。还有不满足冲突可串行化条件的可串行化调度。

[例11.4]有3个事务

$T_1=W_1(Y)W_1(X)$, $T_2=W_2(Y)W_2(X)$, $T_3=W_3(X)$

- 调度 $L_1=W_1(Y)W_1(X)W_2(Y)W_2(X)W_3(X)$ 是一个串行调度。 $T_1 \rightarrow T_2 \rightarrow T_3$, 最后X中W的值由T3决定, Y中W的值由T2决定
- 调度 $L_2=W_1(Y)W_2(Y)W_2(X)W_1(X)W_3(X)$ 不满足冲突可串行化。但是调度 L_2 是可串行化的, 因为 L_2 执行的结果与调度 L_1 相同, Y的值都等于 T_2 的值, X的值都等于 T_3 的值

先执行T1, X的值最终由W3(X)决定, Y的值由W2(X)决定
按位置, 属冲突操作。

11.6两段锁协议

2022年2月15日

9:39

导航

红色 重点 橙色
色：会背 蓝色
了解 绿色：可能

≡ ≧ ≦ ≠

两段锁协议

简称 2PL

定义：指所有事务必须分两个阶段对数据项加锁和解锁，在对任何数据进行读、写操作之前，事务首先要获得对该数据的封锁。在释放一个封锁之后，事务不再申请和获得任何其他锁。

两段”锁的含义：

事务分为两个阶段

- 第一阶段是获得封锁，也称为扩展阶段（即只能加锁）
 - 事务可以申请获得任何数据项上的任何类型的锁，但是不能释放任何锁
- 第二阶段是释放封锁，也称为收缩阶段（即只能解锁）
 - 事务可以释放任何数据项上的任何类型的锁，但是不能再申请任何锁。

遵守两段锁协议的，一定是一个可行化调度。

说明：

1. 事务遵守两段锁协议是可串行化调度的充分条件，而不是必要条件。
2. 若并发事务都遵守两段锁协议，则对这些事务的任何并发调度策略都是可串行化的
3. 若并发事务的一个调度是可串行化的，不一定所有事务都符合两段锁协议。

两段锁协议与防止死锁的一次封锁法的区别：

一次封锁法要求每个事务必须一次将所有要使用的数据全部加锁，否则就不能继续执行，因此一次封锁法遵守两段锁协议，但是两段锁协议并不要求事务必须一次将所有要使用的数据全部加锁，因此遵守两段锁协议的事务可能发生死锁。

封锁的粒度

定义：封锁对象的大小称为封锁粒度(Granularity)

封锁的对象:逻辑单元，物理单元。

逻辑单元: 属性值、属性值的集合、元组、关系、索引项、整个索引、整个数据库等

物理单元: 页（数据页或索引页）、物理记录等

说明：

1. 封锁的粒度越大，数据库所能够封锁的数据单元就越少，并发度就越小，系统开销也越小；
2. 封锁的粒度越小，并发度较高，但系统开销也就越大

例

若封锁粒度是数据页（表），事务T1需要修改元组L1，则T1必须对包含L1的整个数据页A加锁。如果T1对A加锁后事务T2，要修改A中元组L2，则T2被迫等待，直到T1释放A。

如果封锁粒度是元组，则T1和T2可以同时L1和L2加锁，不需要互相等待，提高了系统的并行度。

多粒度封锁

定义：在一个系统中同时支持多种封锁粒度供不同的事务选择。

选择封锁粒度原则：封锁开销和并发度两个因素。

具体：

1. 需要处理多个关系的大量元组的用户事务：以数据库为封锁单位
2. 需要处理大量元组的用户事务：以关系为封锁单元
3. 只处理少量元组的用户事务：以元组为封锁单位

多粒度树：以树形结构来表示多级封锁粒度，根结点是整个数据库，表示最大的数据粒度。叶结点表示最小的数据粒度。



多粒度封锁协议：

允许多粒度树中的每个结点被独立地加锁，对一个结点加锁意味着这个结点的所有后裔结点也被加以同样类型的锁（即如果对表加锁了，则这个表里的每一行也会被加锁）。

在多粒度封锁中一个数据对象可能以两种方式封锁：显式封锁和隐式封锁。

显式封锁：直接加到数据对象上的封锁

隐式封锁：是该数据对象没有独立加锁，是由于其上级结点加锁而使该数据对象加上了锁。

译成汉语了

显式封锁和隐式封锁的效果是一样的，系统检查封锁冲突时，不仅要检查显式封锁，还要检查隐式封锁。

例子：

例如事务T要对关系R1加X锁

1. 系统必须搜索其上级结点数据库、关系R1 往上
2. 还要搜索R1的下级结点，即R1中的每一个元组 往下
3. 如果其中某一个数据对象已经加了不相容锁，则T必须等待

对某个数据对象加锁，系统要检查：

1. 该数据对象：有无显式封锁与之冲突
2. 所有上级结点：检查本事务的显式封锁是否与该数据对象上的隐式封锁冲突：（由上级结点已加的封锁造成的）
3. 所有下级结点：看上面的显式封锁是否与本事务的隐式封锁（将加到下级结点的封锁）冲突

意向锁

引进意向锁（intention lock）目的：提高对某个数据对象加锁时系统的检查效率。

有了意向锁，数据库管理系统就无须逐个检查下级结点的显式封锁。

意向锁含义：如果对一个结点加意向锁，则说明该结点的下层结点正在被加锁。对任一结点加基本锁，必须先对它的上层结点加意向锁。

例如：对任一元组加锁时，必须对它所在的数据库和关系加意向锁。

常用意向锁：

意向共享锁(Intent Share Lock, 简称IS锁)	1. 意向排它锁(Intent Exclusive Lock, 简称IX锁)	共享意向排它锁(Share Intent Exclusive Lock, 简称SIX锁)
如果对一个数据对象加IS锁，表示它的后裔（下一级）结点拟（意向）加S锁。	如果对一个数据对象加IX锁，表示它的后裔结点拟（意向）加X锁。	如果对一个数据对象加SIX锁，表示对它加S锁，再加IX锁，即 $SIX = S + IX$ 。
例如：事务T1要对R1中某个元组加S锁，则要首先对关系R1和数据库加IS锁。	例如：事务T1要对R1中某个元组加X锁，则要首先对关系R1和数据库加IX锁。	例：对某个表加SIX锁，则表示该事务要读整个表（所以要对该表加锁），同时会更新个别元组（所以要对该表加IX锁）

意向锁的相容矩阵

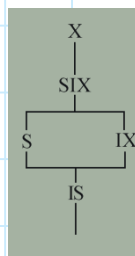
share Exclusive

T ₁ \ T ₂	S	X	IS	IX	SIX	-
S	Y	N	Y	N	N	Y
X	N	N	N	N	N	Y
IS	Y	N	Y	Y	Y	Y
IX	N	N	Y	Y	N	Y
SIX	N	N	Y	N	N	Y
-	Y	Y	Y	Y	Y	Y

Y=Yes, 表示相容的请求 N=No, 表示不相容的请求

(a) 数据锁的相容矩阵

锁的强度：锁的强度是指它对其他锁的排斥程度，一个事务在申请封锁时以强锁代替弱锁是安全的，反之则不然。



具有意向锁的多粒度封锁方法：

1. 申请封锁时应该按自上而下的次序进行
2. 释放封锁时则应该按自下而上的次序进行

例如：事务T1要对关系R1加S锁
要首先对数据库加IS锁

检查数据库和R1是否已加了不相容的锁(x或ix) 不再需要搜索和检查R1中的元组是否加了不相容的锁(x锁)
--

第一节：查询处理

2022年1月13日 10:40

查询处理步骤	关系数据库管理系统查询处理可以分为4个阶段：		
	- 查询分析 - 查询检查 - 查询优化 - 查询执行		
	查询分析	任务： 对查询语句进行扫描，分析 词法 、 语法 是否符合SQL语法规则 - 如果没有语法错误转入下一步 - 如果有语法错误则在报告中显示错误	
	查询检查	- 对合法的查询语句进行语义检查，即根据数据字典中有关的模式定义检查语句中的数据库对象，如关系名、属性名是否存在和有效 - 如果是对视图的操作，则要用视图消解方法把对视图的操作转换成对基本表的操作 - 还要对权限、完整性约束进行检查，如果违反则拒绝查询 - 检查通过后，把SQL查询语句转化为内部表示，也即等价的关系代数表达式 - 在此过程中，要把数据库对象的外部名称换为内部表示 - RDBMS一般用查询树（又称为语法分析树）来表示扩展的关系代数表达式	
	查询优化	任务： 每个查询都会有许多可供选择的执行策略和操作算法，查询优化就是选择一个高效执行的 查询处理策略 。按照优化的层次一般可以将查询优化分为 代数优化：是指关系代数表达式的优化，也即按照一定规则，通过对关系代数表达式进行等价变换，改变代数表达式中操作的次序和组合，使查询更高效 物理优化：是指存取路径和底层操作算法的选择。选择依据可以是基于规则的(rule based)、基于代价的(cost based)、基于语义的(semantic based)	
	查询执行	依据优化器得到的执行策略生成查询执行计划，由 代码生成器(code generator) 生成执行这个查询计划的代码，然后加以执行，回送查询结果。	
实现查询操作的算法示例	选择操作的实现	全表扫描	思想 思想：假设可以使用的内存块为M块 - 按照物理次序读 Student 的M块到内存 - 检查内存的每个元组t，如果t满足选择条件，则输出t - 如果 Student 还有其他块未被处理，重复即可 优缺点 优点：只需要用很少的内存（最少为1块）就可以运行，且控制简单。适用于规模较小的表 缺点：对于规模大的表进行顺序扫描，当选择率低时会使效率很低
		索引扫描	思想： 如果选择条件中的属性上有索引（例如 B+树索引 或 hash索引 ），可以用索引扫描。通过索引先找到满足条件的元组指针，再通过元组指针在查询的基本表中找到元组。一般来说，当选择率低于10%时建立索引才有意义
	连接操作的实现	嵌套循环方法（nested loop）	思想： 对外层循环(Student 表)的每一个元组，检索内层循环(SC 表)中的每一个元组，并检查这两个元组在连接属性(Sno) 上是否相等。如果满足连接条件，则串接后作为结果输出，直到外层循环表中的元组处理完为止

		排序-合并方法 (sort-merge join)	1. 如果参与连接的表没有排好序，首先对 Student 表和 SC 表按连接属性 Sno 排序 2. 取Student表中第一个 Sno ,依次扫描 SC 表中具有相同 Sno 的元组，把它们连接起来 3. 当扫描到 Sno 不相同的第一个SC元组时，返回 Student 表扫描它的下一个元组，再扫描 SC 表中具有相同 Sno 的元组，把它们连接起来 重复上述步骤直至 Student 扫描完毕
		索引连接 (index join)	• 在 SC 表上已经建立了属性 Sno 的索引 • 对 Student 中每一个元组，由 Sno 值通过 SC 的索引查找相应的 SC 元组 • 把这些 SC 元组和 Student 元组连接起来 循环执行第二步和第三步，直至 Student 中的元组处理完毕
		哈希连接 (hash join)	思想： 它把连接属性作为hash码，用同一个hash函数把 Student 表和 SC 表中的元组散列到hash表中 • 划分阶段（创建阶段）： 即创建hash表。对包含较少元组的表(如 Student 表)进行一遍处理，把它的元组按hash函数(hash码是连接属性)分散到hash表的桶中 • 试探阶段（连接阶段）： 对另一个表(SC 表)进行一遍处理，把 SC 表的元组也按同一个hash函数(hash 码是连接属性)进行散列，找到适当的hash桶，并把 SC 元组与桶中来自 Student 表并与之相匹配的元组连接起来。

第二节：查询优化

2022年2月23日 16:16

重点在于明白查询优化的重要性

查询优化的地位和重要性	关系系统的查询优化既是关系数据库管理系统实现的关键技术，又是关系系统的优点所在。用户只要提出“干什么”，而不必指出“怎么干”。 在非关系系统中，用户必须了解存取路径，系统提供用户选择存取路径的手段，查询的效率由用户的存取策略决定，且系统是无法加以优化的。这就要求用户需要具有较高的数据库技术和程序设计水平 查询优化的优点不仅在于用户不必考虑如何最好地表达查询以获得较高的效率，而且在于系统可以比用户程序的“优化”做得更好。这是因为： • 优化器可以从数据字典中获得很多统计信息，但是用户程序难以获得 • 即便数据库物理统计信息改变，系统也可以进行优化从而选择相应的执行计划，但是对于非关系系统则必须要重写程序 • 优化器可以考虑数百种不同的执行计划，但程序员一般仅能考虑有限的几种可能性 • 优化器包含了很多复杂的优化技术，这样就等同于所用的使用者间接拥有了这些技术
执行代价	目前关系数据库管理系统通过某种代价模型计算出各种查询执行策略的执行代价，然后选取代价最小的执行方案。一般来说：总代价=I/O代价+CPU代价+内存代价+通信代价 • 计算查询代价时一般用查询处理读写的块数作为衡量单位

第三节：查询优化之代数优化

2022年2月23日 16:23

关系代数表达式等价变换规则	连接、笛卡尔积、并、交的交换律	笛卡尔积	$R \times S \equiv S \times R$
		并	$R \cup S \equiv S \cup R$
		交	$R \cap S \equiv S \cap R$
		连接	$R \bowtie_F S \equiv S \bowtie_F R, R \bowtie S \equiv S \bowtie R$
	连接、笛卡尔积、并、交的结合律	笛卡尔积	$(R \times S) \times T \equiv R \times (S \times T)$
		并	$(R \cup S) \cup T \equiv R \cup (S \cup T)$
		交	$(R \cap S) \cap T \equiv R \cap (S \cap T)$
		连接	$(R \bowtie_F S) \bowtie_F T \equiv R \bowtie_F (S \bowtie_F T)$ $(R \bowtie S) \bowtie T \equiv R \bowtie (S \bowtie T)$
投影的串接定律	关系的两次投影操作可以合并为一次完成（反过来就是分解） $\Pi_{A_1, A_2, \dots, A_n}(\Pi_{B_1, B_2, \dots, B_m}(E)) \equiv \Pi_{A_1, A_2, \dots, A_n}(E)$ E是关系代数表达式 $A_i(i = 1, 2, \dots, n), B_j(j = 1, 2, \dots, m)$是属性名。并且$\{A_1, A_2, \dots, A_n\}$构成$\{B_1, B_2, \dots, B_m\}$的子集		
选择的串接定律	选择的两次投影操作可以合并为一次完成（反过来就是分解） $\sigma_{F_1}(\sigma_{F_2}(E)) \equiv \sigma_{F_1 \wedge F_2}(E)$		
选择与投影的交换律	$\sigma_F(\Pi_{A_1, A_2, \dots, A_n}(E)) \equiv \Pi_{A_1, A_2, \dots, A_n}(\sigma_F(E))$ - 假设：选择条件F只涉及属性$A_1, A_2, \dots, A_n$ $\Pi_{A_1, A_2, \dots, A_n}(\sigma_F(E)) \equiv \Pi_{A_1, A_2, \dots, A_n}(\sigma_F(\Pi_{A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_m}(E)))$ - 假设：F中有不属于$A_1, A_2, \dots, A_n$的属性$B_1, B_2, \dots, B_m$		
选择与笛卡尔积的交换律	$\sigma_F(E_1 \times E_2) \equiv \sigma_F(E_1) \times E_2$ - 假设：选择条件只与其中的一个关系有关，应该对那个关系先做选择，然后再做笛卡尔积。例如上面F中涉及的属性都是E_1中的属性 $\sigma_F(E_1 \times E_2) \equiv \sigma_{F_1}(E_1) \times \sigma_{F_2}(E_2)$ - 假设：选择条件与两个关系都有关，应该先分别做选择，然后再做笛卡尔积。例如上面$F = F_1 \wedge F_2$，并且F_1中只涉及E_1中的属性，F_2中只涉及E_2中的属性		

		$\sigma_F(E_1 \times E_2) \equiv \sigma_{F_2}(\sigma_{F_1}(E_1) \times E_2)$ • 假设：如果选择条件与某一部分关系有关，那么也应该先对那个关系做部分选择，然后做笛卡尔积，最后做选择。例如上面$F = F_1 \wedge F_2$，并且F_1中只涉及E_1中的属性，F_2中涉及E_1和E_2中的属性
选择与并的分配律		$\sigma(E_1 \cup E_2) \equiv \sigma_F(E_1) \cup \sigma_F(E_2)$ • 假设：$E = E_1 \cup E_2$，E_1和E_2有相同的属性名
选择与差运算的分配律		$\sigma(E_1 - E_2) \equiv \sigma_F(E_1) - \sigma_F(E_2)$
选择对自然连接的分配律		$\sigma_F(E_1 \bowtie E_2) \equiv \sigma_F(E_1) \bowtie \sigma_F(E_2)$ • F只涉及E_1和E_2的公共属性
投影与笛卡尔积的分配律		$\Pi_{A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_m}(E_1 \times E_2) \equiv \Pi_{A_1, A_2, \dots, A_n}(E_1) \times \Pi_{B_1, B_2, \dots, B_m}(E_2)$ • $A_1, A_2, \dots, A_n$是E_1的属性 • $B_1, B_2, \dots, B_m$是E_2的属性
投影与并的分配律		$\Pi_{A_1, A_2, \dots, A_n}(E_1 \cup E_2) \equiv \Pi_{A_1, A_2, \dots, A_n}(E_1) \cup \Pi_{A_1, A_2, \dots, A_n}(E_2)$

查询树的启发式优化	
典型的启发式规则	• 【规则1】选择运算应尽可能先做：这是为了减少中间结果的规模 • 【规则2】投影和选择运算同时进行：这是为了避免重复扫描 • 【规则3】将投影运算与其前后的双目运算结合起来：这是为了避免重复扫描 • 【规则4】把某些选择运算和其前面的笛卡尔积结合起来成为一个连接运算：这是为了减少中间结果的规模 • 【规则5】提取公共子表达式（公因子）：这是为了保存计算结果，避免重复计算
实现算法	• 【步骤1】分解选择运算：这是为了便于不同的选择运算沿树的不同分枝向树叶移动，一直移动到与这个选择条件相关的关系处，使选择尽可能先做。$\sigma_{F_1 \wedge F_2 \wedge \dots \wedge F_n}(E) \Rightarrow \sigma_{F_1}(\sigma_{F_2}(\dots(\sigma_{F_n}(E))\dots))$ • 【步骤2】通过交换选择运算，将每个选择运算尽可能移动到叶端：利用规则4~9尽可能把选择移动到树的叶端 • 【步骤3】通过交换投影运算，将每个投影运算尽可能移动到叶端：利用规则3、11、10、5尽可能把投影移动到树的叶端 • 【步骤4】合并选择和投影的串接：利用规则3~5把选择和投影的串接合并成单个选择、单个投影或一个选择后面跟一个投影。这是为了使多个选择或投影能同时进行，或在一次扫描中全部完成 • 【步骤5】对内结点分组：每一双目运算（$\times$、$\bowtie$、$\cup$、$-$）和它所有的直接祖先的一元运算结点（σ或Π）分为一组（如果其后代直到叶子全是单目运算，则也将他们并入该组）；注意当双目运算是笛卡尔积（$\times$），而且其后的选择不能与它结合为等值连接时，则不能将选择与这个$\times$并为一组
实例演示	https://zhangxing-tech.blog.csdn.net/article/details/122741529#3_195

第四节：查询优化之物理优化

2022年2月23日 19:17

基于启发式规则的存取 路径选择优化（定性）	选择操作的启发式规则	<table><tr><td>小关系</td><td>对于小关系，使用全表顺序扫描，即使选择列上有索引</td></tr><tr><td>大关系</td><td> - 对于选择条件是“主码=值”的查询，查询结果最多是一个元组，可以选择主码索引。一般的RDBMS会自动建立主码索引 - 对于选择条件是“非主属性=值”的查询，并且选择列上有索引，同样要估算查询结果的元组数目，如果选择率<10%可以使用索引扫描方法，否则还是使用全表扫描 - 对于用AND连接的合取选择条件，如果有涉及这些属性的组合索引，则优先采用组合索引扫描方法 - 对于用OR连接的析取选择条件，一般使用全表顺序扫描 </td></tr></table>	小关系	对于小关系，使用 全表顺序扫描 ，即使选择列上有索引	大关系	- 对于选择条件是“主码=值”的查询，查询结果最多是一个元组，可以选择主码索引。一般的RDBMS会自动建立主码索引 - 对于选择条件是“非主属性=值”的查询，并且选择列上有索引，同样要估算查询结果的元组数目，如果选择率<10%可以使用索引扫描方法，否则还是使用全表扫描 - 对于用AND连接的合取选择条件，如果有涉及这些属性的组合索引，则优先采用组合索引扫描方法 - 对于用OR连接的析取选择条件，一般使用全表顺序扫描
	小关系	对于小关系，使用 全表顺序扫描 ，即使选择列上有索引				
大关系	- 对于选择条件是“主码=值”的查询，查询结果最多是一个元组，可以选择主码索引。一般的RDBMS会自动建立主码索引 - 对于选择条件是“非主属性=值”的查询，并且选择列上有索引，同样要估算查询结果的元组数目，如果选择率<10%可以使用索引扫描方法，否则还是使用全表扫描 - 对于用AND连接的合取选择条件，如果有涉及这些属性的组合索引，则优先采用组合索引扫描方法 - 对于用OR连接的析取选择条件，一般使用全表顺序扫描					
连接操作的启发式规则	- 如果2个表都已经按照连接属性排序，则选用排序-合并算法 - 如果一个表在连接属性上有索引，则可以选择索引连接算法 - 如果上面2个规则都不适用，其中一个表较小，则可以选择用hash join算法 - 最后可以选择嵌套循环算法，并选择其中较小的表，确切地讲是占用的块数(B)较少的表，作为外表(外循环的表)（具体原因在第二节已经讲过）					
基于代价估算的优化 （定量）	统计信息	基本表：该表的元组总数（N）、元组长度（l）、占用的块数（B）、占用的溢出块数（BQ）等 基本表每个列：该列不同值的个数（m）、该列最大值、最小值，该列上是否已经建立了索引，如果有，又是哪一种索引等 索引：索引的层数（L）、不同索引值的个数、索引的选择基数S等				
	代价估算示例	https://zhangxing-tech.blog.csdn.net/article/details/122746430				